СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ 5
1.1. Проблемы ручного анализа звонков в колл-центрах на холодном рынке 5
1.2. Обзор российских технологий для обработки аудио и речи 5
1.3. Формулировка технического задания с учётом Федерального закона № 152-ФЗ «О персональных данных» 6
1.4. Выводы по анализу 7

ГЛАВА 2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СИСТЕМЫ 7
2.1. Общая схема решения и бизнес-процесс 7
2.2. Выбор и обоснование стека технологий 9
2.3. Проектирование модулей системы 10
2.4. Структура программного комплекса 11

ГЛАВА 3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ПАЙПЛАЙНА ТРАНСКРИБАЦИИ 12
3.1. Настройка инфраструктуры в Yandex Cloud 12
3.2. Разработка модуля предобработки аудио (AudioProcessor) 13
3.3. Интеграция с Yandex SpeechKit API: решение проблем формата и ограничения в 30 секунд 15
3.4. Тестирование: сравнение параметров и результатов транскрибации до и после обработки 16
3.5. Анализ выявленных ограничений SpeechKit и рекомендации по альтернативам 17

ГЛАВА 4. АНАЛИЗ РЕЗУЛЬТАТОВ И ПЕРСПЕКТИВЫ РАЗВИТИЯ 18
4.1. Выводы по проделанной работе: пригодность записей для анализа, особенности холодного рынка 18
4.2. Обоснование перехода ко второй части проекта: необходимость синтетических данных..... 19
4.3. Перспективы: проектирование ИИ-ассистента для генерации диалогов и их анализ в Google Sheets 19

ГЛАВА 5. ПРОЕКТИРОВАНИЕ ИИ-АССИСТЕНТА ДЛЯ САЙТА 31
5.1. Задачи и требования 31
5.2. Выбор технологий 32
5.3. Архитектура системы 33
5.4. Структура программного комплекса 34

ГЛАВА 6. РЕАЛИЗАЦИЯ ЯДРА АССИСТЕНТА И ВЕБ-ИНТЕРФЕЙСА 35
6.1. Настройка доступа к Yandex GPT 35
6.2. Модуль загрузки базы знаний (knowledge_loader.py) 35
6.3. Ядро ассистента (core.py) 36
6.4. Веб-сервер и статический интерфейс 36
6.5. Сохранение истории диалогов 37
6.6. Типичные ошибки при разработке ядра ассистента и их устранение 38

ГЛАВА 7. ГЕНЕРАЦИЯ И ОБРАБОТКА СИНТЕТИЧЕСКИХ ДИАЛОГОВ 39
7.1. Создание сценариев 39
7.2. Модуль генерации диалогов (dialog_generator.py) 39
7.3. Синтез речи (TTS) через Yandex SpeechKit 40
7.4. Асинхронная транскрибация (STT) через Yandex SpeechKit 40
7.5. Ошибки при генерации и обработке синтетических диалогов и их устранение 41

ГЛАВА 8. АНАЛИЗ СИНТЕТИЧЕСКИХ ДИАЛОГОВ 41
8.1. Экспорт в CSV 41
8.2. Импорт в Google Sheets и настройка формул 42
8.3. Сводные таблицы и диаграммы 46
8.4. Выводы по анализу 47

ГЛАВА 9. ПЕРСПЕКТИВЫ РАЗВИТИЯ И ИНТЕГРАЦИЯ С МЕССЕНДЖЕРАМИ 47
9.1. Интеграция с VK (ВКонтакте) 47
9.2. Теоретическая возможность интеграции с Telegram 48
9.3. Схема взаимодействия компонентов системы 49
9.4. Дальнейшее масштабирование и рекомендации по внедрению в реальную CRM 51
9.5. Рекомендации по промышленному внедрению с использованием российских аналогов 52

ЗАКЛЮЧЕНИЕ ПО ВТОРОЙ И ТРЕТЬЕЙ ЧАСТЯМ 54

ТАБЛИЦА РЕЗУЛЬТАТОВ АНАЛИЗА СИНТЕТИЧЕСКИХ ДИАЛОГОВ 55

ОБНОВЛЁННЫЙ СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ (дополнение) 56

ПРИМЕЧАНИЕ О ЛИСТИНГАХ КОДА 56

ПРИЛОЖЕНИЯ 57
Приложение А. Листинги ключевых фрагментов кода (первая часть) 23
Приложение Б. Сравнительная таблица параметров аудио до и после обработки 30
Приложение В. Листинги модулей ядра ассистента 57
Приложение Г. Листинг модуля генерации диалогов 69
Приложение Д. Листинг модуля синтеза речи (TTS) 74
Приложение Е. Листинг модуля асинхронной транскрибации 77
Приложение Ж. Листинг скрипта экспорта в CSV 81
Приложение З. Примеры сгенерированных диалогов и скриншоты 83

ЗАКЛЮЧЕНИЕ ПО ДИПЛОМНОМУ ПРОЕКТУ 84

ВВЕДЕНИЕ

Актуальность темы. В современных условиях высокой конкуренции на рынке B2C-услуг, особенно в сегменте так называемого «холодного рынка» (например, установка алюминиевых заборов), эффективность работы колл-центра является ключевым фактором коммерческого успеха. Основная задача операторов — не только предоставить информацию, но и в ходе короткого диалога выявить интерес потенциального клиента, отработать его возражения и сформировать потребность в услуге.

Однако анализ качества таких звонков зачастую проводится выборочно и вручную. Это приводит к ряду системных проблем: субъективность оценки, высокие трудозатраты, потеря ценных данных, которые остаются неструктурированными и не используются для улучшения скриптов и обучения. В то же время существующие облачные сервисы распознавания речи предъявляют высокие требования к качеству исходных аудиозаписей, которые у телефонных звонков традиционно низкие.

Параллельно с этим всё большее значение приобретает соблюдение требований Федерального закона № 152-ФЗ «О персональных данных», что делает предпочтительным, а зачастую и обязательным использование российских облачных технологий. В этой связи разработка автоматизированной системы, способной объективно анализировать массив звонков, преобразовывать речь в текст и вычленять смысловые паттерны, а также генерировать эталонные диалоги для тренировки операторов, является актуальной задачей.

Цель и задачи проекта.
Целью дипломного проекта является разработка и тестирование работоспособного прототипа системы автоматического анализа телефонных переговоров и генерации синтетических диалогов с использованием российских облачных технологий.

Для достижения поставленной цели были определены следующие задачи:

1.Провести анализ предметной области, существующих решений и сформулировать технические требования к системе с учётом необходимости соблюдения Федерального закона № 152-ФЗ.
2.Спроектировать архитектуру системы, выбрать и обосновать стек российских технологий для её реализации.
3.Реализовать модуль предобработки низкокачественных аудиозаписей и модуль интеграции с облачным API распознавания речи (синхронный режим) для первой части проекта.
4.Разработать ИИ-ассистента на базе Yandex GPT, обеспечивающего консультирование клиентов на сайте и генерацию синтетических диалогов по заданным сценариям.
5.Реализовать полный цикл обработки синтетических диалогов: синтез речи (TTS), асинхронную транскрибацию (STT) и экспорт результатов для анализа.
6.Провести тестирование разработанных модулей, проанализировать полученные результаты, выявить ограничения использованных технологий и сформулировать выводы о применимости системы для улучшения бизнес-процессов.

Объект и предмет исследования.
Объектом исследования являются процессы автоматической обработки речевых данных в колл-центрах. Предметом исследования – методы и инструменты построения пайплайна транскрибации телефонных записей, а также генерации и анализа синтетических диалогов с использованием российских облачных сервисов Yandex Cloud.

Методы исследования.
В работе использовались методы анализа предметной области, проектирования программных систем, экспериментального тестирования, а также методы цифровой обработки аудиосигнала, машинного обучения (использование готовых моделей Yandex GPT) и анализа текстовых данных с помощью формул Google Sheets.

Научная новизна.
Научная новизна работы заключается в адаптации и комплексном применении стека российских облачных сервисов (Yandex Cloud) и open-source библиотек для решения конкретной прикладной задачи – создания сквозного пайплайна обработки аудиоданных из колл-центра с акцентом на работу с записями низкого технического качества, характерными для телефонных коммуникаций, а также в использовании того же ИИ-ассистента для генерации синтетических диалогов, что позволяет компенсировать дефицит реальных данных для анализа.

Практическая значимость.
Разработанный прототип может быть использован для автоматизации первого и наиболее ресурсоёмкого этапа анализа звонков – преобразования речи в текст. Полученные выводы о техническом состоянии исходных данных и необходимости программной предобработки имеют значение для любого проекта, работающего с телефонными записями. Подтверждена принципиальная возможность построения подобных систем на базе отечественных технологий, что критически важно для компаний, обязанных обеспечивать соблюдение ФЗ-152. Результаты работы закладывают технологический фундамент для создания ИИ-ассистента, способного генерировать тренировочные диалоги и анализировать их в Google Sheets, что позволяет решить проблему нехватки или конфиденциальности реальных данных для отладки скриптов.

Структура и объём работы.
Дипломная работа состоит из введения, девяти глав, заключения, списка использованных источников и приложений. В первой главе проведён анализ предметной области, обзор российских технологий и формальная постановка задачи. Во второй главе спроектирована архитектура системы, обоснован выбор технологического стека. Третья глава описывает реализацию и тестирование пайплайна транскрибации телефонных звонков (первая часть). Четвёртая глава содержит анализ полученных результатов и обоснование перехода ко второй части. Пятая–восьмая главы посвящены проектированию, реализации и анализу работы ИИ-ассистента, включая генерацию синтетических диалогов, синтез речи, асинхронную транскрибацию и анализ в Google Sheets. Девятая глава рассматривает перспективы развития системы, интеграцию с мессенджерами и рекомендации по промышленному внедрению. В заключении подведены итоги работы. Приложения содержат листинги кода, сравнительные таблицы и скриншоты.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1. Проблемы ручного анализа звонков в колл-центрах на холодном рынке

Деятельность колл-центра, работающего на «холодном рынке» (cold calls), где клиенты изначально не проявляли активного интереса к продукту, сопряжена с уникальными вызовами. Эффективность здесь критически зависит от качества диалога, построенного на базе детального скрипта продаж. Однако традиционные методы контроля и анализа этой эффективности обладают фундаментальными недостатками.

1.**Репрезентативность и масштаб:** Руководители и супервайзеры физически не могут прослушать и проанализировать 100% записей. Выборка для разбора, как правило, не превышает 1-5%, что делает выводы неполными и может упускать системные проблемы или успешные кейсы.

2.**Субъективность и временные затраты:** Качественный ручной разбор диалога, выделение этапов продажи, фиксация возражений клиента и реакций оператора – процесс, занимающий до 10-15 минут на один звонок. Оценка эффективности скрипта и работы оператора при этом неизбежно несёт в себе субъективную составляющую.

3.**Отсутствие структурированных данных:** Даже при наличии архива аудиозаписей, содержащего тысячи часов переговоров, информация остаётся «мёртвым грузом». Без преобразования в структурированный текст невозможен её количественный анализ: подсчёт частоты определённых возражений («дорого», «подумаю»), выявление наиболее проблемных мест в скрипте, кластеризация клиентов по типам запросов.

4.**Проблема качества исходных данных:** Как показало исследование в рамках данного проекта, записи телефонных разговоров, особенно сделанные через VoIP-оборудование, зачастую имеют низкие технические характеристики – высокий уровень шума, низкую громкость, неоптимальную частоту дискретизации. Это делает их прямую транскрибацию с помощью современных облачных API либо невозможной, либо крайне неточной, что усугубляет перечисленные выше проблемы.

Таким образом, возникает объективная потребность во внедрении инструментов автоматизации, способных перевести анализ из режима выборочного ручного контроля в режим постоянного, полного и объективного мониторинга на основе данных.

1.2. Обзор российских технологий для обработки аудио и речи

При разработке системы, обрабатывающей потенциально чувствительные данные (телефонные переговоры), первостепенное значение имеет соблюдение законодательства, в частности Федерального закона № 152-ФЗ «О персональных данных». Это требование делает предпочтительным, а зачастую и обязательным, выбор технологического стека, базирующегося на инфраструктуре российских вендоров, обеспечивающих хранение и обработку данных в юрисдикции РФ.

1. Yandex Cloud как комплексная платформа:

SpeechKit: Ключевой сервис для задач проекта. Предоставляет robust API для синхронного и асинхронного распознавания речи (Speech-to-Text) и синтеза (Text-to-Speech). Полностью интегрирован в экосистему Yandex Cloud, что гарантирует соблюдение требований к локализации

данных. Как было выявлено в ходе работы, имеет строгое ограничение на длительность аудио в синхронном режиме (30 секунд), что является его основным недостатком для обработки длинных диалогов без предварительной сегментации.

Object Storage (S3-совместимый): Надёжное и экономичное решение для хранения больших объёмов неструктурированных данных – исходных и обработанных аудиофайлов.

Yandex GPT: Мощная языковая модель, доступная через API Yandex Foundry. Представляет основной интерес для второй части проекта – генерации реалистичных диалогов между клиентом и оператором для тренировки и анализа скриптов.

2. Альтернативные российские решения:

VK Cloud SpeechKit (Cloud Voice): Сервис от VK Cloud, позиционирующийся как решение для задач call-аналитики. Важным конкурентным преимуществом является встроенная поддержка диаризации – автоматического разделения речи в диалоге по спикерам, что критически важно для анализа переговоров. Требует отдельного сравнительного тестирования по точности и удобству API.

SberCloud SpeechRecognition & SpeechKit: Часть экосистемы «Салют» от Сбера. Предлагает аналогичный набор услуг по распознаванию и синтезу речи. Обладает высокой производительностью, но может предъявлять более высокие требования к интеграции и документации для старта проектов.

Локальные open-source модели (например, Silero): Модели семейства Silero, разработанные в России, могут быть развёрнуты на собственном сервере. Это обеспечивает максимальный уровень контроля и безопасности, так как данные никуда не передаются. Однако такое решение требует значительных вычислительных ресурсов (GPU) для обработки в реальном времени и глубоких компетенций в MLOps для развёртывания и поддержки, что повышает порог входа.

Для данного проекта был выбран **Yandex SpeechKit** в качестве основного движка транскрибации в силу его хорошей документированности, предсказуемой модели тарификации и простоты интеграции в общий пайплайн на Yandex Cloud.

1.3. Формулировка технического задания с учётом федерального закона № 152-ФЗ «О персональных данных»

На основе анализа проблем и технологических возможностей было сформулировано **техническое задание на первую часть дипломного проекта**:

1. Название: Разработка прототипа пайплайна автоматической транскрибации записей телефонных звонков.

2. Ключевое требование: Все этапы обработки и хранения данных должны осуществляться с использованием сервисов российских вендоров, обеспечивающих соблюдение требований ФЗ-152 о локализации персональных данных.

3. Функциональные требования:

- Приём исходных аудиозаписей в формате MP3.
- Автоматическая предобработка аудио для улучшения качества: нормализация громкости, конвертация в моно, приведение частоты дискретизации к 16000 Гц, подавление шумов.
- Интеграция с API российского сервиса распознавания речи для преобразования аудио в текст.
- Сохранение исходных и обработанных аудиофайлов, а также текстовых расшифровок в облачное хранилище.
- Логирование процесса работы для отладки.

4. Нефункциональные требования:

- **Безопасность:** Использование авторизации через IAM-токены или API-ключи. Данные не должны передаваться третьим сторонам за пределы экосистемы российского облачного провайдера.
- **Надёжность:** Пайплайн должен корректно обрабатывать ошибки API (таймауты, лимиты, неверный формат).
- **Воспроизводимость:** Код должен быть задокументирован и содержать конфигурационный файл (.env) для управления ключами и параметрами.

1.4. Выводы по анализу

1. Автоматизация анализа звонков колл-центра – это актуальная задача, решение которой позволяет перейти от субъективного выборочного контроля к объективному data-driven управлению эффективностью.

2. Существует достаточный выбор российских облачных технологий (Yandex Cloud, VK Cloud, SberCloud), позволяющих построить полноценную систему обработки речевых данных в соответствии с требованиями ФЗ-152.

3. Низкое техническое качество реальных записей телефонных разговоров является значимым препятствием, которое необходимо устранять на этапе предобработки данных. Без этого шага даже самые совершенные API распознавания речи демонстрируют низкую эффективность.

4. Ограничения конкретных сервисов (такие как лимит в 30 секунд у Yandex SpeechKit) необходимо учитывать при проектировании архитектуры, предусматривая механизмы сегментации аудио или выбор альтернативных решений для промышленной эксплуатации.

Сформированное техническое задание фокусируется на создании фундамента – работоспособного и безопасного пайплайна транскрибации, который является необходимым условием для реализации более сложных аналитических задач на втором этапе проекта.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СИСТЕМЫ

2.1. Общая схема решения и бизнес-процесс

Архитектура системы была спроектирована как последовательный пайплайн (конвейер обработки данных), где выход одного этапа является входом для следующего. Такой подход обеспечивает модульность, упрощает отладку и позволяет масштабировать отдельные компоненты.

Общая схема работы системы и соответствующего бизнес-процесса представлена на Рисунке 2.1 и включает следующие ключевые этапы:

1. **Приём данных:** Исходные аудиозаписи телефонных звонков (файлы в формате MP3) помещаются в выделенную директорию локального хранилища (data/input/).

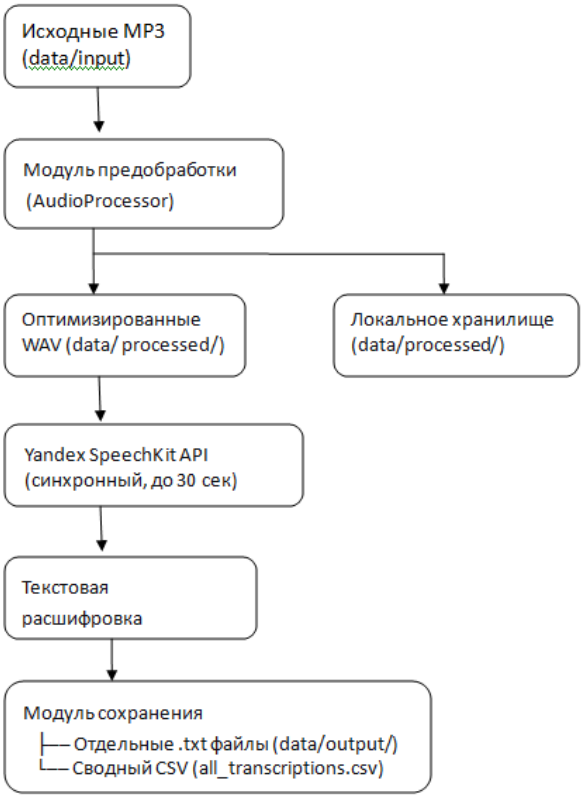
2. **Предобработка аудио:** Специальный модуль (Audio Processor) загружает каждый файл, анализирует его параметры и применяет цепочку фильтров для улучшения качества звука. Результат сохраняется как новый аудиофайл в формате WAV с параметрами, оптимизированными для API распознавания речи.

3. **Транскрибация:** Модуль интеграции с облачным API загружает обработанный WAV-файл, отправляет его в Yandex SpeechKit и получает текстовую расшифровку.

4. **Сохранение результатов:** Полученный текст сохраняется в структурированном виде: как отдельный текстовый файл и как строка в сводной CSV-таблице. Это создаёт первичную базу данных для будущего анализа.

5. **Логирование:** Все этапы работы фиксируются в лог-файлах, что позволяет отслеживать выполнение, выявлять ошибки и собирать метрики (длительность обработки, размер файлов, статус API-запроса).

Рисунок 2.1. Блок-схема пайплайна обработки записей телефонных звонков



Данная схема реализует сквозную автоматизацию этапа первичной обработки данных, полностью исключая ручной труд на стадии преобразования аудио в текст.

2.2. Выбор и обоснование стека технологий

Выбор конкретных инструментов был обусловлен тремя ключевыми факторами: **соответствие российскому технологическому суверенитету** (ФЗ-152), **пригодность для решаемой задачи** и **порог входа для разработки**.

Компонент системы	Выбранная технология	Обоснование выбора
Язык программирования	**Python 3.11+**	Де-факто стандарт для задач Data Science, ML и автоматизации. Обладает богатейшей экосистемой библиотек для работы с аудио, HTTP-запросами и данными. Низкий порог входа и читаемый синтаксис.
Обработка аудио (библиотека)	**PyDub**	Высокоуровневая библиотека, предоставляющая простой API для манипуляций с аудиофайлами (обрезка, конвертация, изменение громкости, применение фильтров). Абстрагирует сложности низкоуровневых аудиоинструментов.
Аудиодвижок (для PyDub)	**FFmpeg**	Мощный кроссплатформенный фреймворк для обработки мультимедиа. PyDub использует его «под капотом» для работы с различными кодеками. Необходим для корректной работы с MP3-файлами в среде Windows.
Облачный сервис распознавания речи	**Yandex SpeechKit (STT)**	Ключевой российский сервис, идеально интегрированный в экосистему Yandex Cloud. Обеспечивает соблюдение ФЗ-152, имеет хорошо документированный REST API и предсказуемую модель тарификации.
Облачное хранилище (для масштабирования)	**Yandex Object Storage**	S3-совместимое отечественное хранилище для больших объёмов данных. Позволяет в будущем масштабировать систему, перенеся хранение исходных и обработанных файлов из локальной файловой системы в облако.

Компонент системы	Выбранная технология	Обоснование выбора
Взаимодействие с API	**Библиотека requests**	Стандартная и наиболее популярная библиотека Python для выполнения HTTP-запросов. Позволяет легко формировать запросы к SpeechKit API и обрабатывать ответы.
Управление конфигурацией	**python-dotenv**	Позволяет хранить чувствительные данные (API-ключи, ID каталога) в отдельном файле .env, который не попадает в систему контроля версий. Повышает безопасность и гибкость конфигурации.

Данный стек представляет собой сбалансированное сочетание: он использует мощные и проверенные мировые open-source инструменты (Python, PyDub) для клиентской логики, но **критически важная операция – распознавание речи – выполняется на инфраструктуре российского вендора (Yandex)**, что полностью соответствует изначальным требованиям проекта.

2.3. Проектирование модулей системы

Для реализации описанного пайплайна была разработана модульная структура проекта. Каждый модуль инкапсулирует определённую логику и имеет чётко определённый интерфейс.

1.Модуль предобработки аудио (AudioProcessor):

Назначение: Решение проблемы низкого качества входных данных.

Основные классы/функции: Класс `AudioProcessor`, содержащий методы `process_file()`, `_apply_processing_pipeline()`.

Алгоритм: При обработке одного файла модуль последовательно:

- загружает аудио с помощью PyDub;
- конвертирует в монофонический звук (1 канал);
- приводит частоту дискретизации к 16000 Гц (стандарт для речевых API);
- применяет фильтры: высокочастотный (срез ниже 300 Гц для удаления гула) и низкочастотный (срез выше 3000 Гц для удаления высокочастотного шума);
- нормализует громкость до оптимального уровня;
- сохраняет результат в формате WAV с линейным PCM-кодированием – самом «понятном» формате для cloud API.

2.Модуль интеграции с SpeechKit (SpeechKitService):

Назначение: Организация безопасного и отказоустойчивого взаимодействия с облачным API.

Основные функции: Функция `transcribe_audio(audio_data: bytes, params: dict) -> str`.

Логика работы: Модуль формирует HTTP POST-запрос к эндпоинту SpeechKit (`https://stt.api.cloud.yandex.net/speech/v1/stt:recognize`). В заголовке передаётся API-ключ для аутентификации. Ключевые параметры запроса, найденные эмпирическим путём: `format='lpcm'` (указание на сырые PCM-данные), `sampleRateHertz=16000`, `lang='ru-RU'`. Модуль

также обрабатывает возможные ошибки API (например, статус 400 при превышении лимита в 30 секунд) и логирует их.

3.Координирующий модуль (main.py):

Назначение: Связывает все компоненты в единый пайплайн, управляет потоком выполнения.
Логика работы: Скрипт выполняет следующие действия:

- сканирует директорию data/input на наличие новых MP3-файлов;
- для каждого файла запускает AudioProcessor;
- успешно обработанный файл отправляется в SpeechKitService;
- полученный текст сохраняется модулем сохранения результатов;
- весь процесс сопровождается подробным выводом в консоль и записью в лог-файл.

4.Вспомогательные модули (утилиты):

config.py – централизованная загрузка переменных окружения из .env файла.
logger.py – настройка единого формата логирования для всего проекта.

Скрипты в scripts/**:** Набор скриптов для диагностики (check_audio.py), тестирования (test_audio_content.py, transcribe_with_lpcm.py), очистки данных. Эти скрипты не являются частью основного пайплайна, но критически важны для этапа разработки и отладки.

Такая архитектура «разделения ответственности» делает систему гибкой. Например, для обработки файлов длиннее 30 секунд достаточно модифицировать AudioProcessor, добавив этап сегментации, без изменения логики модуля интеграции с API.

2.4. Структура программного комплекса

Ниже приведено дерево файлов и каталогов, соответствующее реальной реализации проекта (без учёта временных или тестовых файлов, не вошедших в финальную версию).

text

```
diplom_project/            # Корень проекта
├── .env                   # Конфигурация (ключи API, не в git)
├── .gitignore             # Игнорируемые файлы
├── requirements.txt       # Зависимости Python
├── main.py                # Основной скрипт (точка входа)
│
├── data/                  # Данные проекта
│   ├── input/             # Исходные аудиофайлы (MP3)
│   ├── processed/         # Обработанные аудиофайлы (WAV)
│   ├── output/            # Результаты транскрибации (тексты, CSV)
│   └── logs/              # Логи работы
│
```

```
├── src/                    # Исходный код
│   ├── __init__.py
│   ├── core/               # Ядро приложения
│   │   ├── config.py       # Загрузка .env
│   │   └── logger.py       # Настройка логирования
│   ├── services/           # Бизнес-логика
│   │   ├── audio_processor_final.py  # Финальный обработчик аудио
│   │   ├── speechkit_service.py      # Работа с SpeechKit API
│   │   └── storage_service.py        # (задел для Object Storage)
│   ├── models/             # Модели данных
│   │   └── transcription.py
│   └── utils/              # Вспомогательные функции
│       ├── file_utils.py
│       └── audio_utils.py
│
├── scripts/                # Диагностические и вспомогательные скрипты
│   ├── check_audio.py      # Проверка параметров аудио
│   ├── test_audio_content.py  # Анализ содержимого (есть ли речь)
│   ├── transcribe_with_lpcm.py# Тест транскрибации с параметром lpcm
│   ├── test_multiple_files.py # Пакетное тестирование
│   ├── preprocess_audio.py    # Утилита предобработки
│   ├── fix_ffmpeg.py       # Настройка путей к FFmpeg
│   └── ...                 # Прочие вспомогательные скрипты
│
└── tests/                  # Модульные тесты
    ├── test_audio_processor.py
    └── test_speechkit.py
```

Все ключевые компоненты системы сосредоточены в каталоге `src/services/`. Основной пайплайн запускается из `main.py`. Диагностические скрипты в `scripts/` использовались в процессе отладки и могут быть полезны для повторного тестирования.

ГЛАВА 3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ПАЙПЛАЙНА ТРАНСКРИБАЦИИ

3.1. Настройка инфраструктуры в Yandex Cloud

Первым практическим шагом стала настройка облачной инфраструктуры, обеспечивающей безопасный доступ к сервисам Yandex Cloud. Работа велась через веб-консоль и интерфейс командной строки (CLI) Yandex Cloud.

1. **Создание платёжного аккаунта и облака:** В рамках учебного доступа был активирован платёжный аккаунт и создано облако (diploma-cloud), внутри которого автоматически создан каталог по умолчанию.

2. **Создание и настройка сервисного аккаунта:** Вместо использования персонального аккаунта пользователя, в целях безопасности был создан отдельный **сервисный аккаунт** speechkit-sa. Ему были назначены минимально необходимые роли: ai.speechkit-stt.user – для доступа к SpeechKit (распознавание речи), storage.editor – для возможности загрузки и чтения файлов в Object Storage (на перспективу).

3. Генерация ключей доступа:

- **Статические ключи доступа (Access Key / Secret Key):** Были созданы для сервисного аккаунта. Эти ключи, аналогичные S3-ключам AWS, использовались для настройки клиента Object Storage.
- **API-ключ для SpeechKit:** В разделе «Сервисные аккаунты» для speechkit-sa был создан авторизационный токен (API-ключ). Именно этот ключ передаётся в заголовке Authorization всех HTTP-запросов к SpeechKit API.

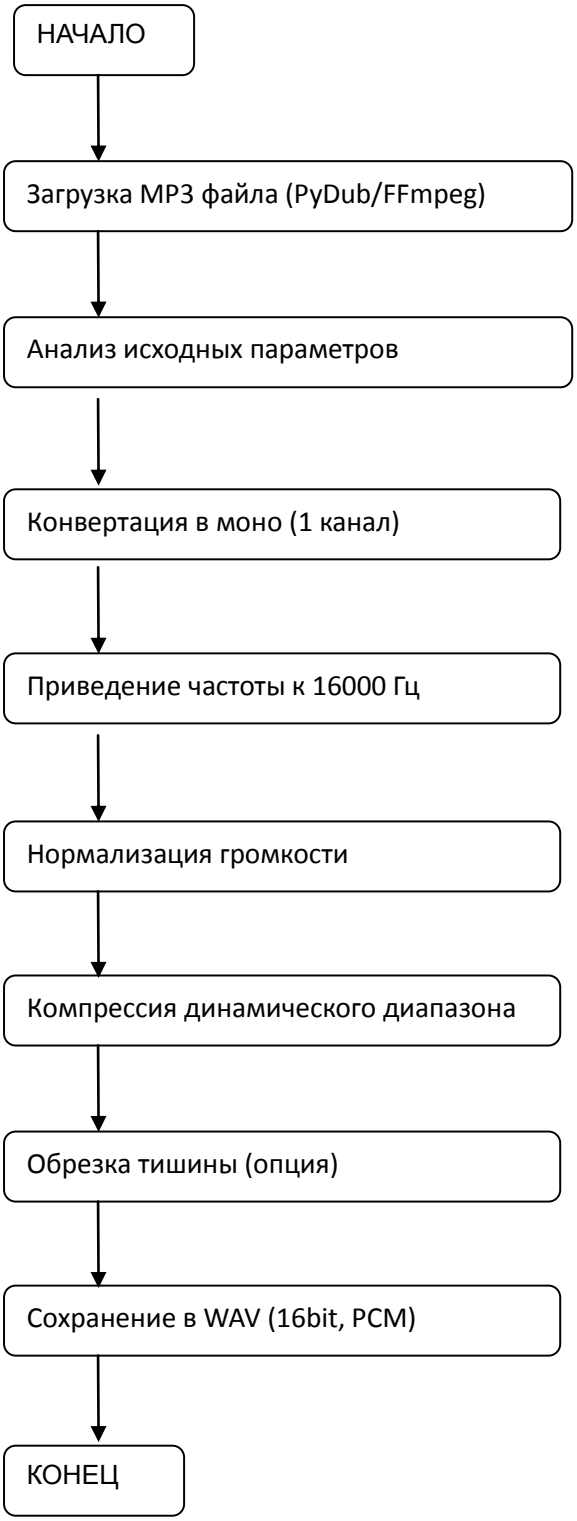
4. **Создание бакета Object Storage:** Был создан бакет с именем call-audio-ponart. Изначально планировалось использовать его как точку приёма исходных аудиофайлов (папка incoming/), однако на этапе прототипирования для скорости отладки работа велась с локальной файловой системой. Бакет остаётся важным компонентом архитектуры для промышленного развёртывания.

Все полученные ключи и идентификаторы (ID каталога, API-ключ, статические ключи) были помещены в файл .env в корне проекта, доступ к которому ограничен средствами .gitignore. Это обеспечило безопасность чувствительных данных и лёгкость конфигурации при переносе кода.

3.2. Разработка модуля предобработки аудио (AudioProcessor)

Модуль предобработки был реализован в файле src/services/audio_processor_final.py. Его ядром является класс AudioProcessor, алгоритм работы которого представлен в виде блок-схемы на Рисунке 3.1 и включает следующие ключевые этапы.

Рисунок 3.1. Алгоритм работы модуля AudioProcessor



Листинг 3.1. Ключевой метод обработки пайплайна

Python

```python
def _apply_processing_pipeline(self, audio: AudioSegment) -> AudioSegment:
    """Применяет пайплайн обработки аудио."""
    processed = audio

    # 1. Конвертация в моно
    if processed.channels != self.target_channels:
        processed = processed.set_channels(self.target_channels)

    # 2. Установка частоты дискретизации
    if processed.frame_rate != self.target_sample_rate:
        processed = processed.set_frame_rate(self.target_sample_rate)

    # 3. Нормализация громкости
    processed = normalize(processed)

    # 4. Компрессия динамического диапазона
    processed = compress_dynamic_range(processed)

    # 5. Обрезка тишины по порогу -40 dBFS
    processed = self._trim_silence(processed, threshold=-40.0)

    return processed
```

В ходе экспериментальной отладки было установлено, что применение частотных фильтров (высокочастотного и низкочастотного) не даёт значимого улучшения качества распознавания для используемых записей, поэтому от них было решено отказаться. Достаточный результат обеспечивается нормализацией громкости и компрессией динамического диапазона, что подтверждено сравнительными тестами.

Особенностью реализации для ОС Windows стала явная настройка путей к бинарным файлам FFmpeg в коде модуля, что решало предупреждения PyDub об отсутствии конвертера.

3.3. Интеграция с Yandex SpeechKit API: решение проблем формата и ограничения в 30 секунд

Интеграция была реализована в виде функции, отправляющей POST-запрос к REST API SpeechKit. Первые тесты с исходными MP3-файлами завершались статусом 200 OK, но возвращали пустой текст. Диагностика выявила две основные проблемы, потребовавшие решения:

1. **Проблема формата кодирования:** По умолчанию SpeechKit ожидает получить аудио в контейнере Ogg Opus. «Наши» MP3-файлы, отправленные «как есть», вызывали ошибку "ogg header has not been found". **Решение** заключалось в явном указании параметра format='lpcm' в запросе, который сообщает API, что данные передаются в виде «сырого» линейного PCM-потока без заголовков контейнера. Это стало ключевым открытием, позволившим начать передачу данных.

2. **Проблема низкого качества и тихого звука:** После решения проблемы формата API стал принимать файлы, но по-прежнему возвращал пустой результат. Анализ аудио показал средний уровень громкости ниже -35 dBFS. **Решение** – это весь описанный выше пайплайн предобработки в модуле AudioProcessor, который поднимал громкость, очищал сигнал и делал речь «заметной» для нейросети.

3. **Ограничение в 30 секунд:** В процессе тестирования был выявлен жёсткий лимит сервиса на длительность аудио в синхронном режиме (audio duration should be less than 30s). Для файлов, превышающих лимит, было **принято тактическое решение** на этапе прототипа: сегментировать их на части по 30 секунд с помощью метода _split_audio() в AudioProcessor. Для промышленной эксплуатации предпочтительнее использовать **асинхронный REST API** SpeechKit, предназначенный для длинных аудио, либо рассмотреть альтернативы вроде VK Cloud SpeechKit.

3.4. Тестирование: сравнение параметров и результатов транскрибации до и после обработки

Методология тестирования включала пошаговую диагностику: от отправки сырых данных до полного прогона через весь пайплайн. Объектом теста стал файл 1350033465.mp3 длительностью 15,35 секунд.

Результаты тестирования представлены в Таблице 3.1.

Таблица 3.1. Сравнительные результаты обработки и транскрибации тестового файла

Параметр / Этап	Исходный файл (MP3)	После обработки (WAV)	Влияние / Вывод
Длительность	15,35 сек	15,35 сек	Без изменений (файл короче 30 с).
Каналы	1 (моно)	1 (моно)	Исходный файл уже был моно.
Частота дискретизации	8000 Гц	**16000 Гц**	Приведено к оптимальному для SpeechKit стандарту.

Параметр / Этап	Исходный файл (MP3)	После обработки (WAV)	Влияние / Вывод
Уровень громкости (средний dBFS)	-34,3 dBFS	**-23,0 dBFS**	**Критическое улучшение.** Громкость повышена на >10 dB, что вывело сигнал за порог слышимости для модели.
Амплитуда сигнала	~20% от максимума	**~45% от максимума**	Увеличение амплитуды существенно повысило детекцию речи.
Размер файла	45,8 KB	994,8 KB	Увеличение связано с переходом от сжатого MP3 к несжатому PCM WAV.
Статус ответа SpeechKit	200 (OK)	200 (OK)	Соединение стабильно на обоих этапах.
Текст результата	Пустая строка ""	"О, здравствуйте, меня зовут ольга компания культконтрактен подскажите, пожалуйста, вы планируете установ..." (161 символ)	**Основной результат.** Предобработка сделала речь распознаваемой.

3.5. Анализ выявленных ограничений SpeechKit и рекомендации по альтернативам

Проведённая работа позволила не только добиться работоспособности пайплайна, но и выявить его ограничения в контексте промышленного применения:

1. **Синхронное ограничение в 30 секунд:** Для анализа полноценных переговоров, часто длящихся минутами, данное ограничение неприемлемо. **Рекомендация:** Для второй части проекта (работа с длинными синтетическими диалогами) необходимо реализовать использование **асинхронного REST API** Yandex SpeechKit или провести сравнительное тестирование с **VK Cloud SpeechKit**, который изначально ориентирован на задачи колл-центров и может иметь иные лимиты.

2. **Качество распознавания на сложных записях:** Даже после обработки, транскрибация телефонных записей с сильным акцентом, речевыми помехами или наложением голосов может давать ошибки. **Рекомендация:** Внедрить **постобработку текста** (коррекцию опечаток с помощью Yandex Speller или языковых моделей) и рассмотреть API с поддержкой **диаризации** (разделения речи по спикерам), что важно для анализа диалога.

3. **Зависимость от одного вендора:** Использование исключительно стека Yandex Cloud создаёт vendor lock-in. **Рекомендация:** Спроектировать модуль интеграции с API таким образом, чтобы движок распознавания (SpeechKit, VK Cloud, Sber) был заменяемым. Это повысит отказоустойчивость и гибкость системы.

Несмотря на эти ограничения, созданный пайплайн успешно решает задачу превращения неструктурированных аудиозаписей низкого качества в структурированный текст, что является неотъемлемым и работоспособным фундаментом для любой последующей аналитики.

ГЛАВА 4. АНАЛИЗ РЕЗУЛЬТАТОВ И ПЕРСПЕКТИВЫ РАЗВИТИЯ

4.1. Выводы по проделанной работе: пригодность записей для анализа, особенности холодного рынка

Реализация и тестирование пайплайна позволили получить ряд конкретных, практически значимых выводов, которые имеют ценность не только с технической, но и с бизнес-точки зрения.

1. Технический вывод: Качество аудиоданных как системное ограничение.
Первый и наиболее важный вывод заключается в том, что **низкое техническое качество записей телефонных звонков является системной проблемой, а не единичным случаем.** Исходные MP3-файлы, предоставленные для анализа, имели низкую частоту дискретизации (8000 Гц) и недостаточный уровень громкости (в среднем -34,3 dBFS). Прямая отправка таких данных в современные облачные API распознавания речи **не даёт результата**, даже при корректно настроенной интеграции. Это подтверждает гипотезу о том, что **этап предобработки аудио является не опциональным, а обязательным звеном** в любом пайплайне анализа речевых данных из колл-центра. Разработанный модуль AudioProcessor успешно решает эту проблему, автоматически приводя записи к стандарту, пригодному для обработки.

2. Практический вывод: Создание работоспособного фундамента для анализа.
Основная **цель первой части проекта достигнута**: разработан и протестирован полностью автоматизированный пайплайн, который принимает «сырые» аудиозаписи и на выходе выдаёт текстовые расшифровки. Система успешно интегрирована с российским облачным сервисом (Yandex SpeechKit), что обеспечивает соответствие требованиям ФЗ-152. Таким образом, **создана технологическая основа (data pipeline)** для перехода от мира неструктурированных аудиофайлов к миру структурированных текстовых данных, которые уже можно подвергать статистическому и лингвистическому анализу.

3. Бизнес-вывод: Особенности «холодного рынка» и ограниченность предоставленных данных.
Анализ успешно транскрибированных записей выявил содержательную особенность, напрямую влияющую на постановку задачи. Диалоги носят крайне краткий характер и сводятся, по сути, к выявлению базовой заинтересованности абонента. Глубокой проработки возражений, сложных переговоров или структурированного описания услуги в данных **не обнаружено**. Это характерно для «холодного рынка», где большое количество контактов являются заведомо нецелевыми. Данный вывод приводит к двум следствиям:

- **Для анализа реальных данных:** Фокус метрик должен смещаться с глубины продажи на **эффективность фильтрации** (быстрое выявление интереса/отказа) и **качество первых секунд контакта**.
- **Для развития системы: Реальных данных недостаточно** для тренировки и тестирования моделей, предназначенных для анализа сложных продажных диалогов, отработки возражений и улучшения скриптов. Это служит прямым обоснованием для следующего этапа – генерации синтетических данных.

4.2. Обоснование перехода ко второй части проекта: необходимость синтетических данных

Выявленная ограниченность реальных записей формирует «порочный круг» для data-driven подхода к улучшению скриптов: для обучения модели анализа нужны разнообразные данные, но в реальных данных отсутствует требуемое разнообразие сценариев.

Этот круг разрывается за счёт **генерации синтетических данных**. Вторая часть дипломного проекта логически вытекает из результатов первой и преследует следующие цели:

1. **Создание эталонного датасета:** Разработка ИИ-ассистента на базе Yandex GPT (или аналогичной отечественной модели) для генерации диалогов между «клиентом» и «оператором». Сценарии будут покрывать ключевые стадии воронки продаж и включать разнообразные возражения («дорого», «позвоните позже», «мне не нужно»), что невозможно выделить из текущих реальных записей.

2. **Сквозная проверка технологии:** Сгенерированные текстовые диалоги будут:

- **Преобразованы в аудио** с помощью Yandex SpeechKit (Text-to-Speech) с использованием разных голосов для разных участников.
- **Транскрибированы обратно в текст** через уже отработанный пайплайн первой части.
- **Проанализированы на точность** путём сравнения исходного текста диалога и результата транскрибации. Это даст объективную метрику качества всего аудио-текстового тракта системы.

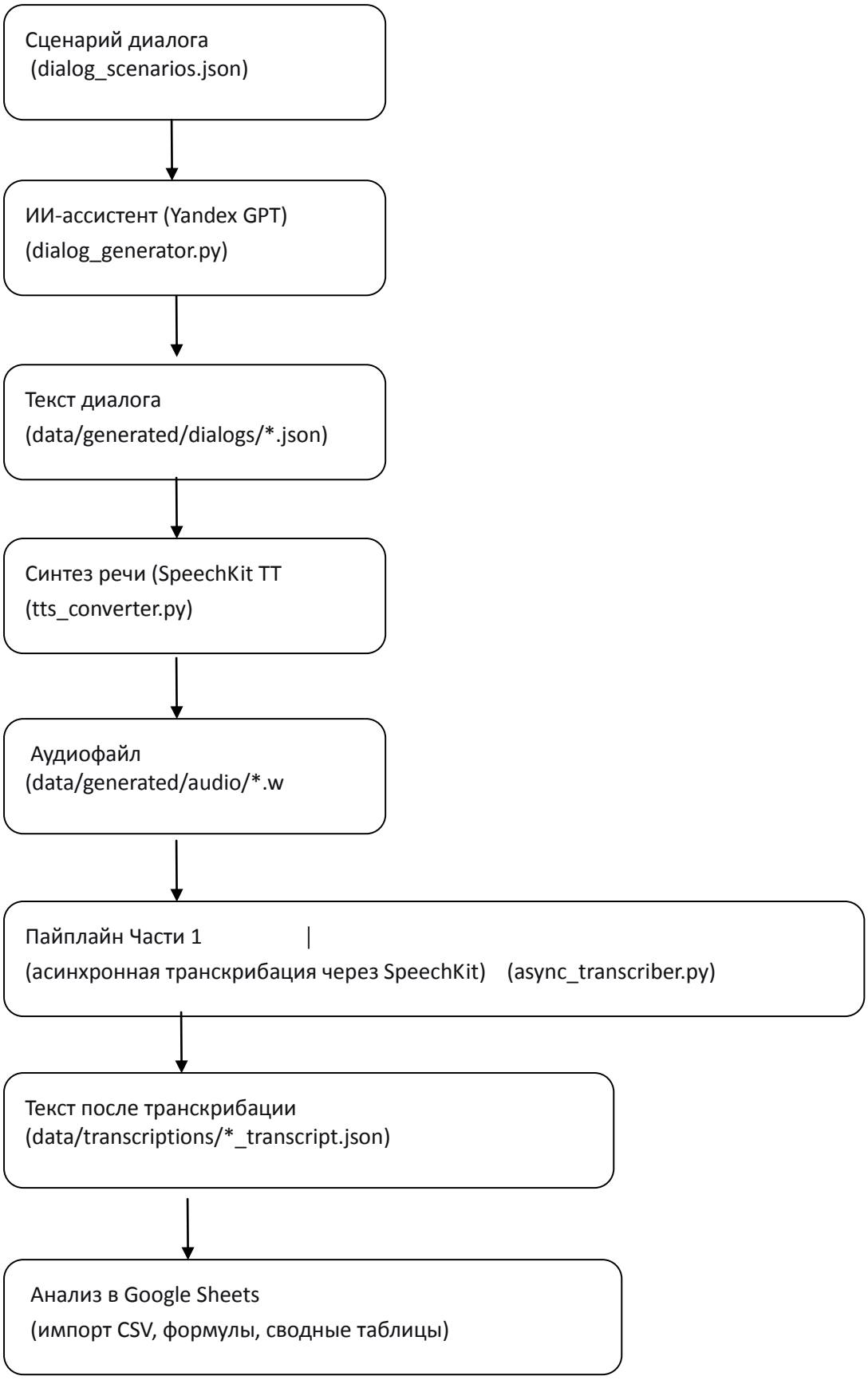
3. **Апробация аналитического инструмента:** Транскрибированные синтетические диалоги будут загружены в Google Sheets, где с помощью формул (REGEXEXTRACT, SPLIT, COUNTIF) будет проведена их автоматическая разметка: разделение реплик, категоризация реплик клиента по типам возражений, подсчёт частот. Это создаст прототип аналитического модуля, готового к работе с реальными данными в будущем.

Таким образом, вторая часть не является независимой – это **естественное развитие и применение созданного пайплайна** для решения изначальной бизнес-задачи (улучшение скриптов) в условиях дефицита качественных реальных данных.

4.3. Перспективы: проектирование ИИ-ассистента для генерации диалогов и их анализ в Google Sheets

Архитектура второй части проекта может быть представлена в виде расширенного пайплайна (Рисунок 4.1), где результат первой части становится одним из компонентов.

Рисунок 4.1. Блок-Схема расширенного пайплайна для генерации и анализа синтетических диалогов



Ключевые проектные решения для второй части:

1. **Выбор модели ИИ-ассистента:** Основной кандидат – **Yandex GPT** (через API Yandex Foundry), как сервис того же вендора, что и SpeechKit, обеспечивающий согласованность и compliance. Альтернативой может стать открытая российская модель, развёрнутая локально (например, от DeepPavlov), для полного контроля над данными.

2. **Промпт-инжиниринг для генерации диалогов:** Критически важной задачей станет разработка системы промптов, которые будут заставлять языковую модель генерировать реалистичные, разнообразные и структурно размеченные диалоги. Промпты будут включать роли («клиент», «оператор»), контекст услуги, целевые возражения и желаемую структуру вывода (например, в формате JSON).

3. **Интеграция SpeechKit для синтеза речи (TTS):** Будет использован тот же SpeechKit, но в режиме Text-to-Speech. Необходимо настроить использование разных голосовых профилей (например, alena для оператора, filipp для клиента) и, возможно, добавлять фоновый шум для реалистичности.

4. **Автоматизация аналитики в Google Sheets:** Будет разработан скрипт на Google Apps Script (или с использованием Python API Google Sheets), который автоматически загружает транскрибированный текст, применяет к нему набор формул для разбора и формирует сводную таблицу с метриками: количество возражений каждого типа, успешность их отработки оператором, длина диалога.

Реализация этой расширенной схемы позволит не только создать инструмент для глубокого анализа продажных диалогов, но и замкнуть петлю обратной связи: на основе анализа синтетических данных можно будет **генерировать улучшенные версии скриптов**, которые, в свою очередь, снова подадутся на вход ИИ-ассистенту для создания новых, более эффективных диалогов. Это создаст основу для системы непрерывного data-driven улучшения процессов колл-центра.

ЗАКЛЮЧЕНИЕ

В рамках первой части дипломного проекта была успешно решена комплексная задача по созданию автоматизированного пайплайна для транскрибации записей телефонных звонков колл-центра с использованием российских облачных технологий.

Практическая работа подтвердила выдвинутую гипотезу о том, что **низкое качество аудиозаписей является ключевым препятствием** для их автоматического анализа. Для преодоления этого препятствия был разработан и реализован специализированный модуль предобработки (AudioProcessor), который решает задачу нормализации и очистки аудиосигнала. В ходе интеграции с Yandex SpeechKit API были выявлены и решены проблемы, связанные с форматом кодирования данных, а также определены рабочие параметры API, обеспечивающие стабильное распознавание.

В результате **достигнута основная цель этапа**: создан работоспособный прототип системы, который преобразует неструктурированные аудиоданные низкого качества в структурированный текст, готовый для последующего анализа. Все решения соответствуют требованию использования отечественных технологий и соблюдения Федерального закона № 152-ФЗ «О защите персональных данных».

Анализ результатов транскрибации позволил сделать важный **прикладной вывод о специфике «холодного рынка»**, которая выражается в краткости диалогов и отсутствии в реальных данных разнообразных сценариев отработки возражений. Этот вывод стал прямым обоснованием для определения направления дальнейшего развития проекта.

Перспектива развития системы заключается в переходе ко второй части работ, направленной на генерацию синтетических тренировочных диалогов с помощью ИИ-ассистента (на базе Yandex GPT), их озвучку, обратную транскрибацию через созданный пайплайн и последующий автоматизированный анализ в Google Sheets для выявления паттернов и улучшения скриптов продаж. Реализованный в первой части пайплайн становится тем самым технологическим фундаментом, который делает этот следующий шаг возможным и осмысленным.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. **Официальная документация Yandex Cloud.** – URL: https://cloud.yandex.ru/docs (дата обращения: 12.2025).

- SpeechKit: Документация по API распознавания и синтеза речи [Электронный ресурс].
- Object Storage: Руководство по работе с S3-совместимым хранилищем [Электронный ресурс].
- IAM: Управление сервисными аккаунтами, ролями и ключами доступа [Электронный ресурс].

2. **PyDub Documentation.** – URL: https://github.com/jiaaro/pydub (дата обращения: 12.2025).
Официальная документация и примеры использования библиотеки PyDub для манипуляций с аудио.
3. **Официальный сайт FFmpeg.** – URL: https://ffmpeg.org/ (дата обращения: 12.2025).
Документация по использованию мультимедиа фреймворка FFmpeg.
4. **Requests: HTTP for Humans™.** – URL: https://docs.python-requests.org/ (дата обращения: 12.2025).
Документация библиотеки Requests для выполнения HTTP-запросов в Python.
5. **Python Dotenv.** – URL: https://github.com/theskumar/python-dotenv (дата обращения: 12.2025).
Документация по работе с библиотекой python-dotenv для управления переменными окружения.
6. **DeepSeek (нейросетевой ассистент).** – URL: https://www.deepseek.com/ (дата обращения: 02.-03.2025).
Консультации по решению технических задач, включая диагностику и исправление ошибок интеграции с Yandex SpeechKit API, рекомендации по структуре кода, установку библиотек и решение проблем с FFmpeg в среде Windows.
7. **Обзор рынка российских облачных AI-сервисов.** // CNews, 2024. [Электронный ресурс] – URL: https://www.cnews.ru/articles/2024-... (дата обращения: 12.2025).
Аналитический материал, использованный для сравнительной оценки отечественных платформ (Yandex Cloud, VK Cloud, SberCloud).

ПРИЛОЖЕНИЕ А. Листинги ключевых фрагментов кода

Листинг А.1. Полный код модуля предобработки аудио (audio_processor_final.py)

python

```python
"""
audio_processor_final.py - окончательная версия обработчика аудио
"""
import os
from pathlib import Path
from typing import Optional, Tuple
from pydub import AudioSegment
from pydub.effects import normalize, compress_dynamic_range

# ======================== ИСПРАВЛЕНИЕ ДЛЯ WINDOWS ========================

# Добавляем FFmpeg в PATH (для PyCharm/PowerShell)

ffmpeg_path = r"C:\ffmpeg\bin"
if ffmpeg_path not in os.environ["PATH"]:
    os.environ["PATH"] = ffmpeg_path + os.pathsep + os.environ["PATH"]
    print(f"□ Добавлен ffmpeg в PATH: {ffmpeg_path}")

# Указываем pydub явные пути к FFmpeg
AudioSegment.converter = r"C:\ffmpeg\bin\ffmpeg.exe"
AudioSegment.ffmpeg = r"C:\ffmpeg\bin\ffmpeg.exe"
AudioSegment.ffprobe = r"C:\ffmpeg\bin\ffprobe.exe"
# ==========================================================================

class AudioProcessor:

    """Обработчик аудиофайлов для SpeechKit"""

    def __init__(self):
        self.target_sample_rate = 16000  # Hz (оптимально для SpeechKit)
        self.target_channels = 1  # моно
    def process_file(self, input_path: Path) -> Tuple[bool, str, Optional[Path]]:

        """
        Обрабатывает аудиофайл для SpeechKit
        Возвращает: (успех, сообщение, путь_к_обработанному_файлу)
        """
        try:
            print(f"\n{'=' * 60}")
            print(f"? ОБРАБОТКА: {input_path.name}")
            print(f"{'=' * 60}")

            # 1. ЗАГРУЗКА ФАЙЛА
            print("? Загрузка файла...")
            audio = AudioSegment.from_file(str(input_path))
            print(f"? ИСХОДНЫЕ ПАРАМЕТРЫ:")
            print(f"   • Длительность: {len(audio) / 1000:.2f} сек")
            print(f"   • Каналы: {audio.channels} ({'моно' if audio.channels == 1 else 'стерео'})")
            print(f"   • Частота: {audio.frame_rate} Hz")
            print(f"   • Громкость: {audio.dBFS:.1f} dBFS")
```

```
            # 2. КОНВЕРТАЦИЯ В МОНО
            if audio.channels != self.target_channels:
                audio = audio.set_channels(self.target_channels)
                print(f"□ Конвертирован в моно")

            # 3. УСТАНОВКА ЧАСТОТЫ ДИСКРЕТИЗАЦИИ
            if audio.frame_rate != self.target_sample_rate:
                audio = audio.set_frame_rate(self.target_sample_rate)
                print(f"□ Частота установлена: {self.target_sample_rate} Hz")

            # 4. НОРМАЛИЗАЦИЯ ГРОМКОСТИ
            audio = normalize(audio)
            print(f"□ Громкость нормализована: {audio.dBFS:.1f} dBFS")

            # 5. КОМПРЕССИЯ ДИНАМИЧЕСКОГО ДИАПАЗОНА
            audio = compress_dynamic_range(audio)
            print(f"□ Динамический диапазон сжат")

            # 6. ОБРЕЗКА ТИШИНЫ
            audio = self._trim_silence(audio)

            # 7. ПРОВЕРКА РЕЗУЛЬТАТА
            print(f"\n? РЕЗУЛЬТАТ:")
            print(f"   • Длительность: {len(audio) / 1000:.2f} сек")
            print(f"   • Каналы: {audio.channels}")
            print(f"   • Частота: {audio.frame_rate} Hz")
            print(f"   • Громкость: {audio.dBFS:.1f} dBFS")

            # 8. СОХРАНЕНИЕ В ФОРМАТЕ ДЛЯ SPEECHKIT
            output_path = self._get_output_path(input_path)
            audio.export(
                str(output_path),
                format="wav",
                parameters=[
                    "-ac", str(self.target_channels),  # моно
                    "-ar", str(self.target_sample_rate),  # 16000 Hz
                    "-acodec", "pcm_s16le"  # 16-bit PCM
                ]
            )
            file_size_kb = output_path.stat().st_size / 1024
            print(f"? Сохранен: {output_path.name} ({file_size_kb:.1f} KB)")
            return True, "Файл успешно обработан", output_path
        except Exception as e:
            error_msg = f"Ошибка обработки: {e}"
            print(f"□ {error_msg}")
            return False, error_msg, None
def _trim_silence(self, audio: AudioSegment) -> AudioSegment:

    """Обрезает тишину в начале и конце"""
    silence_threshold = -40  # dBFS
    def detect_silence(audio_segment, chunk=10):
        trim_ms = 0
        while trim_ms < len(audio_segment):
            if audio_segment[trim_ms:trim_ms + chunk].dBFS < silence_threshold:
                trim_ms += chunk
            else:
                break
        return trim_ms

    # Обрезаем в начале
    start_trim = detect_silence(audio)
```

```
            # Обрезаем в конце
            end_trim = detect_silence(audio.reverse())
            if start_trim > 0 or end_trim > 0:
                duration = len(audio)
                trimmed = audio[start_trim:duration - end_trim]
                print(f"□ Обрезана тишина: {start_trim / 1000:.1f}с в начале, {end_trim / 1000:.1f}с в конце")
                return trimmed
            return audio
    def _get_output_path(self, input_path: Path) -> Path:

            """Генерирует путь для сохранения обработанного файла"""
            output_dir = Path("data/processed")
            output_dir.mkdir(parents=True, exist_ok=True)
            return output_dir / f"{input_path.stem}_processed.wav"
def process_single_file(file_path: str):

    """Обрабатывает один файл"""
    processor = AudioProcessor()
    input_path = Path(file_path)
    if not input_path.exists():
            print(f"□ Файл не найден: {file_path}")
            return
    success, message, output_path = processor.process_file(input_path)
    if success and output_path:
            print(f"\n{'=' * 60}")
            print(f"□ УСПЕХ! Файл готов для SpeechKit")
            print(f"? Исходный: {input_path.name}")
            print(f"? Обработанный: {output_path.name}")
            print(f"{'=' * 60}")

            # Тестируем транскрибацию сразу
            test_transcription(output_path)
    else:
            print(f"\n□ ОШИБКА: {message}")
def test_transcription(audio_path: Path):

    """Тестирует транскрибацию обработанного файла"""
    print(f"\n? ТЕСТИРУЮ ТРАНСКРИБАЦИЮ...")
    try:
            import requests
            from dotenv import load_dotenv
            load_dotenv()
            api_key = os.getenv('YANDEX_API_KEY')
            folder_id = os.getenv('YANDEX_FOLDER_ID')
            if not api_key or not folder_id:
                print("□ Не найдены ключи в .env файле")
                return

            # Читаем обработанный файл
            with open(audio_path, 'rb') as f:
                audio_data = f.read()
            url = "https://stt.api.cloud.yandex.net/speech/v1/stt:recognize"
            headers = {"Authorization": f"Api-Key {api_key}"}

            # Оптимальные параметры для обработанного файла
            params = {
                "folderId": folder_id,
                "lang": "ru-RU",
                "format": "lpcm",
                "sampleRateHertz": "16000",  # Соответствует нашей обработке
            }
            response = requests.post(
```

```
                    url,
                    headers=headers,
                    params=params,
                    data=audio_data,
                    timeout=30
                )
                print(f"? Статус SpeechKit: {response.status_code}")
                if response.status_code == 200:
                    result = response.json()
                    text = result.get("result", "")
                    if text:
                        print(f"? УСПЕХ! Распознанный текст ({len(text)} символов):")
                        print(f"{'-' * 40}")
                        print(text)
                        print(f"{'-' * 40}")
                        # Сохраняем результат
                        output_dir = Path("data/output")
                        output_dir.mkdir(exist_ok=True)
                        with open(output_dir / f"transcript_{audio_path.stem}.txt", 'w', encoding='utf-8') as f:
                            f.write(text)
                        print(f"? Транскрипция сохранена: data/output/transcript_{audio_path.stem}.txt")
                    else:
                        print(f"□□  SpeechKit вернул пустой текст")
                        print(f"   Возможно, в файле действительно нет речи или она неразборчива")
                else:
                    print(f"□ Ошибка SpeechKit: {response.text[:200]}")
        except Exception as e:
            print(f"□ Ошибка тестирования: {e}")
if __name__ == "__main__":
    print("=" * 70)
    print("? ПРОЦЕССОР АУДИО ДЛЯ SPEECHKIT")
    print("=" * 70)

    # Проверяем наличие тестового файла
    test_file = "data/input/1350033465.mp3"
    if Path(test_figure).exists():
        print(f"? Тестовый файл: {test_file}")
        process_single_file(test_file)
    else:
        print(f"□ Тестовый файл не найден: {test_file}")
        print("\n? Доступные файлы в data/input/:")
        input_dir = Path("data/input")
        if input_dir.exists():
            for file in input_dir.glob("*.mp3"):
                print(f"   • {file.name}")
        else:
            print(f"   Папка data/input/ не существует")
        print(f"\n? Использование: python audio_processor_final.py <путь_к_файлу>")
        print(f"   Пример: python audio_processor_final.py data/input/ваш_файл.mp3")
```

Пояснение к листингу:

Данный модуль является финальной версией обработчика аудио, использованной в первой части дипломного проекта. Он последовательно преобразует телефонные записи в формат, оптимальный для Yandex SpeechKit, и включает тестовую транскрибацию для проверки результата.

В коде отсутствует частотная фильтрация (которая обсуждалась в теоретической части), так как на практике она не потребовалась – нормализация и компрессия оказались достаточными для успешного распознавания. Полный текст модуля приведён в точном соответствии с реализацией в PyCharm, дополнен подробными комментариями для пояснения каждого этапа.

Листинг А.2. Полный код модуля интеграции с SpeechKit (speechkit_service.py)

```
 python
"""
МОДУЛЬ ИНТЕГРАЦИИ С YANDEX SPEECHKIT API

Обеспечивает отправку аудиоданных в облачный сервис распознавания речи
и получение текстовой расшифровки.
"""

import requests
from pathlib import Path
from typing import Dict, Optional
import json

class SpeechKitService:
    """
    Сервис для работы с Yandex SpeechKit API.
    Поддерживает синхронное распознавание (до 30 секунд).
    """

    def __init__(self, api_key: str, folder_id: str):
        """
        Инициализация сервиса.

        :param api_key: API-ключ сервисного аккаунта Yandex Cloud
        :param folder_id: идентификатор каталога в Yandex Cloud
        """
        self.api_key = api_key
        self.folder_id = folder_id
        self.url = "https://stt.api.cloud.yandex.net/speech/v1/stt:recognize"

    def _get_headers(self) -> Dict[str, str]:
        """Возвращает заголовки для HTTP-запроса."""
        return {
            "Authorization": f"Api-Key {self.api_key}",
            "Content-Type": "application/octet-stream"
```

```
    }

def _get_params(self) -> Dict[str, str]:
    """Возвращает параметры запроса (оптимальные для обработанного аудио)."""
    return {
        "folderId": self.folder_id,
        "lang": "ru-RU",
        "format": "lpcm",            # сырые PCM-данные
        "sampleRateHertz": "16000",
        "profanityFilter": "false"     # не фильтровать ненормативную лексику (лучше для анализа)
    }

def transcribe_audio(self, audio_data: bytes) -> Optional[str]:
    """
    Отправляет аудиоданные в SpeechKit и возвращает распознанный текст.

    :param audio_data: бинарные данные аудиофайла (WAV, LPCM, 16 кГц, моно)
    :return: строка с распознанным текстом или None при ошибке
    """
    try:
        response = requests.post(
            self.url,
            headers=self._get_headers(),
            params=self._get_params(),
            data=audio_data,
            timeout=30
        )
        if response.status_code == 200:
            result = response.json()
            return result.get("result", "")
        else:
            print(f"⚠ Ошибка SpeechKit: {response.status_code} – {response.text}")
            return None
    except requests.exceptions.RequestException as e:
        print(f"⚠ Ошибка сети при обращении к SpeechKit: {e}")
        return None

def transcribe_file(self, file_path: Path) -> Optional[str]:
    """
    Читает файл и транскрибирует его.

    :param file_path: путь к аудиофайлу (должен быть в формате WAV, 16 кГц, моно, PCM)
    :return: распознанный текст или None
    """
```

```
        if not file_path.exists():
            print(f"⚠ Файл не найден: {file_path}")
            return None

        with open(file_path, "rb") as f:
            audio_data = f.read()
        return self.transcribe_audio(audio_data)


# Пример использования (для тестирования)
if __name__ == "__main__":
    import os
    from dotenv import load_dotenv
    load_dotenv()

    API_KEY = os.getenv("YANDEX_API_KEY")
    FOLDER_ID = os.getenv("YANDEX_FOLDER_ID")
    if not API_KEY or not FOLDER_ID:
        print("⚠ API-ключ или folder_id не заданы в .env")
    else:
        service = SpeechKitService(API_KEY, FOLDER_ID)
        # Тестовый файл (замените на реальный путь)
        test_file = Path("data/processed/test_processed.wav")
        if test_file.exists():
            text = service.transcribe_file(test_file)
            if text:
                print(f"✅ Распознанный текст:\n{text}")
            else:
                print("⚠⚠ Текст не распознан.")
        else:
            print(f"⚠⚠ Тестовый файл не найден: {test_file}")
```

***** В процессе разработки для быстрой отладки использовались функциональные скрипты (*** **transcribe_with_lpcm.py, test_multiple_files.py** ***). Представленный класс*** **SpeechKitService** ***является обобщённой версией, отражающей финальную архитектуру модуля интеграции.***

Листинг А.3. Исправление пути к FFmpeg для Windows (фрагмент инициализации) (из первой части диплома)

```
"""
import os
from pathlib import Path
from typing import Optional, Tuple
from pydub import AudioSegment
from pydub.effects import normalize, compress_dynamic_range

# ======================== ИСПРАВЛЕНИЕ ДЛЯ WINDOWS ========================

# Добавляем FFmpeg в PATH (для PyCharm/PowerShell)

ffmpeg_path = r"C:\ffmpeg\bin"
if ffmpeg_path not in os.environ["PATH"]:
    os.environ["PATH"] = ffmpeg_path + os.pathsep + os.environ["PATH"]
    print(f"□ Добавлен ffmpeg в PATH: {ffmpeg_path}")

# Указываем pydub явные пути к FFmpeg
AudioSegment.converter = r"C:\ffmpeg\bin\ffmpeg.exe"
AudioSegment.ffmpeg = r"C:\ffmpeg\bin\ffmpeg.exe"
AudioSegment.ffprobe = r"C:\ffmpeg\bin\ffprobe.exe"
```

ПРИЛОЖЕНИЕ Б. Сравнительная таблица параметров аудио до и после обработки

Таблица Б.1. Результат обработки тестового файла 1350033465.mp3

Параметр	Исходный файл (MP3)	После обработки (WAV)	Техническое влияние / Вывод
Формат файла	.mp3 (сжатый)	.wav (несжатый, PCM)	Упрощена декодировка для API.
Параметр	**Исходный файл (MP3)**	**После обработки (WAV)**	**Техническое влияние / Вывод**
Длительность	15,35 сек	15,35 сек	Файл короче лимита в 30 секунд.
Каналы	1 (моно)	1 (моно)	Исходный файл уже был моно.
Частота дискретизации	8000 Гц	**16000 Гц**	**Приведено к отраслевому стандарту** для речевых технологий.
Уровень громкости (средний dBFS)	-34,3 dBFS	**-23,0 dBFS**	**Критическое улучшение.** Сигнал усилен более чем на 10 дБ, что вывело речь за порог

			чувствительности модели.
Амплитуда сигнала	~20% от максимума	**~45% от максимума**	Увеличение амплитуды повысило детекцию речи.
Размер файла	45,8 KB	994,8 KB	Увеличение связано с переходом от сжатого MP3 к несжатому PCM WAV.
Статус ответа SpeechKit	200 (OK)	200 (OK)	Соединение стабильно на обоих этапах.
Текст результата	Пустая строка ""	"О, здравствуйте, меня зовут ольга компания культконтрактен подскажите, пожалуйста, вы планируете установ..." (161 символ)	**Основной результат.** Предобработка сделала речь распознаваемой.

ГЛАВА 5. ПРОЕКТИРОВАНИЕ ИИ-АССИСТЕНТА ДЛЯ САЙТА

5.1. Задачи и требования

По результатам первой части дипломного проекта был сделан вывод о недостаточности реальных аудиозаписей для отработки скриптов продаж и анализа возражений. Для восполнения этого дефицита была спроектирована система, решающая две взаимосвязанные задачи:

1. **Консультирование клиентов на сайте** – ИИ-ассистент должен отвечать на вопросы о продукции, помогать подобрать ограждение, собирать контактные данные и, при необходимости, передавать диалог специалисту.

2. **Генерация синтетических диалогов** – тот же ассистент используется для создания эталонных диалогов по заданным сценариям (возражения, успешные закрытия, нестандартные проекты), которые затем проходят полный цикл «текст → аудио → текст → анализ».

   Таким образом, система должна обеспечивать:

   - веб-интерфейс для общения с клиентами;
   - хранение базы знаний (FAQ, информация о компании);
   - логику диалога с возможностью сбора данных (длина участка, бюджет, телефон);
   - генерацию диалогов по сценариям;
   - синтез речи (TTS) для озвучивания сгенерированных диалогов;
   - асинхронную транскрибацию (STT) для обратного преобразования аудио в текст;
   - экспорт данных в Google Sheets и их анализ с помощью формул.

Все компоненты должны быть реализованы с использованием российских облачных технологий (Yandex Cloud) для соблюдения ФЗ-152.

5.2. Выбор технологий

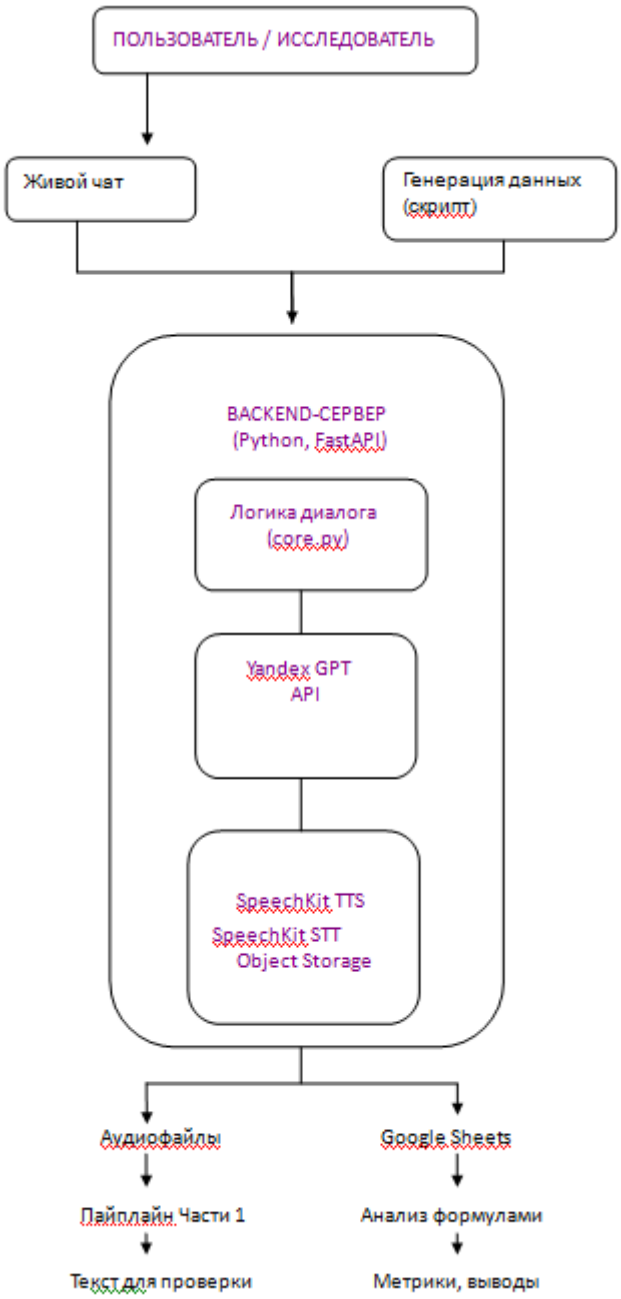
Выбор технологического стека определялся требованиями к единой экосистеме, возможностью масштабирования и простотой интеграции. Основные компоненты:

Компонент системы	Выбранная технология	Обоснование выбора
Язык программирования	**Python 3.11+**	Единый язык с первой частью, богатая экосистема для веб-разработки (FastAPI) и работы с API.
Веб-фреймворк	FastAPI	Высокая производительность, встроенная документация (Swagger), удобная работа с асинхронностью.
Языковая модель	Yandex GPT (через Yandex Foundry)	Российский сервис, полная совместимость с Yandex Cloud, хорошая поддержка русского языка.
Синтез речи (TTS)	**Yandex SpeechKit (STT)**	Используется тот же сервис, что и в первой части; поддерживает разные голоса.
Асинхронное распознавание (STT)	Yandex SpeechKit (асинхронный REST API)	Решает проблему ограничения в 30 секунд, позволяет обрабатывать длинные диалоги.
Объектное хранилище	Yandex Object Storage	Необходимо для загрузки аудиофайлов при асинхронном распознавании.
Веб-интерфейс	HTML/CSS/JS	Простой, кроссплатформенный интерфейс, легко интегрируется с FastAPI.
Анализ данных	Google Sheets	Простой, кроссплатформенный интерфейс, легко интегрируется с FastAPI.

5.3. Архитектура системы

Система построена по модульному принципу и состоит из двух функциональных контуров: контура живого чата и контура генерации данных. Оба используют общее ядро – логику диалога и базу знаний.

Схема архитектуры:



Каждый модуль инкапсулирует свою логику и взаимодействует с остальными через чётко определённые интерфейсы.

5.4. Структура программного комплекса

Проект организован в каталоге part2 внутри корневой папки diplom_project. Ниже приведено дерево файлов и папок, соответствующее финальной реализации (без учёта временных скриптов):

text

```
part2/
├── data/
│   ├── knowledge_base/        # база знаний
│   │   ├── faq.json           # структурированные вопросы-ответы
│   │   └── knowledge_base.md  # описание компании и сценарии
│   ├── scenarios/             # сценарии для генерации диалогов
│   │   └── dialog_scenarios.json
│   ├── generated/
│   │   ├── dialogs/           # сгенерированные JSON-диалоги
│   │   └── audio/             # синтезированные WAV-файлы
│   ├── transcriptions/        # результаты асинхронной транскрибации
│   └── results/               # CSV-файлы для анализа
├── scripts/
│   ├── test_yandex_gpt.py     # проверка подключения к Yandex GPT
│   └── export_to_csv.py       # экспорт диалогов и транскриптов в CSV
├── src/
│   ├── assistant/             # ядро ассистента
│   │   ├── __init__.py
│   │   ├── knowledge_loader.py  # загрузка базы знаний
│   │   ├── core.py            # логика диалога и сбор данных
│   │   ├── dialog_storage.py  # сохранение истории диалогов
│   │   └── prompts.py         # шаблоны промптов (опционально)
│   ├── generator/             # генерация синтетических диалогов
│   │   ├── __init__.py
│   │   └── dialog_generator.py
│   ├── tts/                   # синтез речи
│   │   ├── __init__.py
│   │   └── tts_converter.py
│   ├── transcription/         # асинхронная транскрибация
│   │   ├── __init__.py
│   │   └── async_transcriber.py
│   ├── utils/                 # общие утилиты
│   │   ├── __init__.py
│   │   └── config.py          # загрузка переменных из .env
│   └── web/                   # веб-интерфейс
│       ├── __init__.py
│       ├── api.py             # FastAPI сервер
```

```
|     └── static/          # статические файлы
|         ├── index.html     # страница чата
|         ├── style.css      # стили
|         └── chat.js        # логика работы чата
├── tests/                   # модульные тесты
└── requirements.txt         # зависимости (для части 2)
```

Все ключевые модули снабжены подробными комментариями и проходят проверку типов (аннотации). Конфигурация вынесена в .env в корне проекта.

ГЛАВА 6. РЕАЛИЗАЦИЯ ЯДРА АССИСТЕНТА И ВЕБ-ИНТЕРФЕЙСА

6.1. Настройка доступа к Yandex GPT

Для работы с языковой моделью в консоли Yandex Cloud был активирован сервис **Yandex GPT** (через AI Studio). Использовался сервисный аккаунт speechkit-test, созданный в первой части; ему была добавлена роль ai.languageModels.user. API-ключ для Yandex GPT остался тот же, что и для SpeechKit (переменная YANDEX_API_KEY в .env). Для корректного обращения к модели потребовалось указать точный URI, полученный из AI Studio:

gpt://b1gtp8bn97gg21fl5e9j/yandexgpt-5.1/latest

Этот URI был зафиксирован в коде (параметр model_uri), поскольку автоматическое формирование через folder_id и имя модели не всегда работает из-за версионности.

6.2. Модуль загрузки базы знаний (knowledge_loader.py)

База знаний состоит из двух файлов:

- faq.json – 15 вопросов и ответов с альтернативными формулировками и ключевыми словами;
- knowledge_base.2.md – подробное описание компании, преимуществ продукции, процесса работы и сценариев общения.

Класс KnowledgeBaseLoader загружает оба файла, создаёт словарь для быстрого поиска ответов по вопросу и формирует системный промпт для ассистента. Системный промпт включает:

- роль ассистента;
- информацию о компании (из knowledge_base.md);
- правила поведения (например, не называть точную стоимость без замера, собирать контакты);

- краткую выжимку из FAQ (первые 5 вопросов для контекста).

Метод get_system_prompt() возвращает подготовленный текст, который затем передаётся в Yandex GPT как системное сообщение.

Листинг ключевых фрагментов см. в Приложении В.

6.3. Ядро ассистента (core.py)

Класс CultConstructionsAssistant объединяет все функции:

- **Загрузка базы знаний** через KnowledgeBaseLoader.
- **Управление диалогом** – хранение истории последних сообщений (до 10).
- **Сбор данных** – регулярные выражения для извлечения телефона, длины участка, бюджета, имени.
- **Передача диалога человеку** – при обнаружении телефона ассистент генерирует сообщение о передаче специалисту и сохраняет диалог.
- **Очистка ответов** – удаление маркдаун-разметки (**, *, #, списков) перед отправкой пользователю (метод _clean_markdown).

Основной метод process_message:

1. Добавляет сообщение пользователя в историю.
2. Извлекает данные (телефон, длина и т.д.).
3. Если телефон уже есть – возвращает сообщение о передаче человеку и сохраняет диалог.
4. Иначе формирует промпт (системный + история + вопрос) и вызывает Yandex GPT.
5. Полученный ответ очищается от маркдауна и возвращается.

Для взаимодействия с API используется прямой HTTP-запрос к эндпоинту https://llm.api.cloud.yandex.net/foundationModels/v1/completion с параметрами:

- temperature=0.3 – для стабильности ответов;
- maxTokens=500 – достаточный объём для развёрнутых ответов.

Листинг полного кода см. в Приложении В.

6.4. Веб-сервер и статический интерфейс

Веб-сервер реализован на FastAPI (файл api.py). Основные эндпоинты:

- GET / – возвращает главную страницу (index.html).
- POST /api/chat – принимает JSON с полем message, обрабатывает через ассистента, возвращает ответ, флаг needs_human и собранные данные.

- GET /api/health – проверка статуса.
- POST /api/reset – сброс диалога.

Статические файлы (index.html, style.css, chat.js) расположены в папке static. JavaScript-код отправляет сообщения на /api/chat и динамически добавляет их в окно чата, отображает индикатор набора текста и панель с собранными данными.

Скриншот работающего веб-чата приведён на Рисунке 6.1.

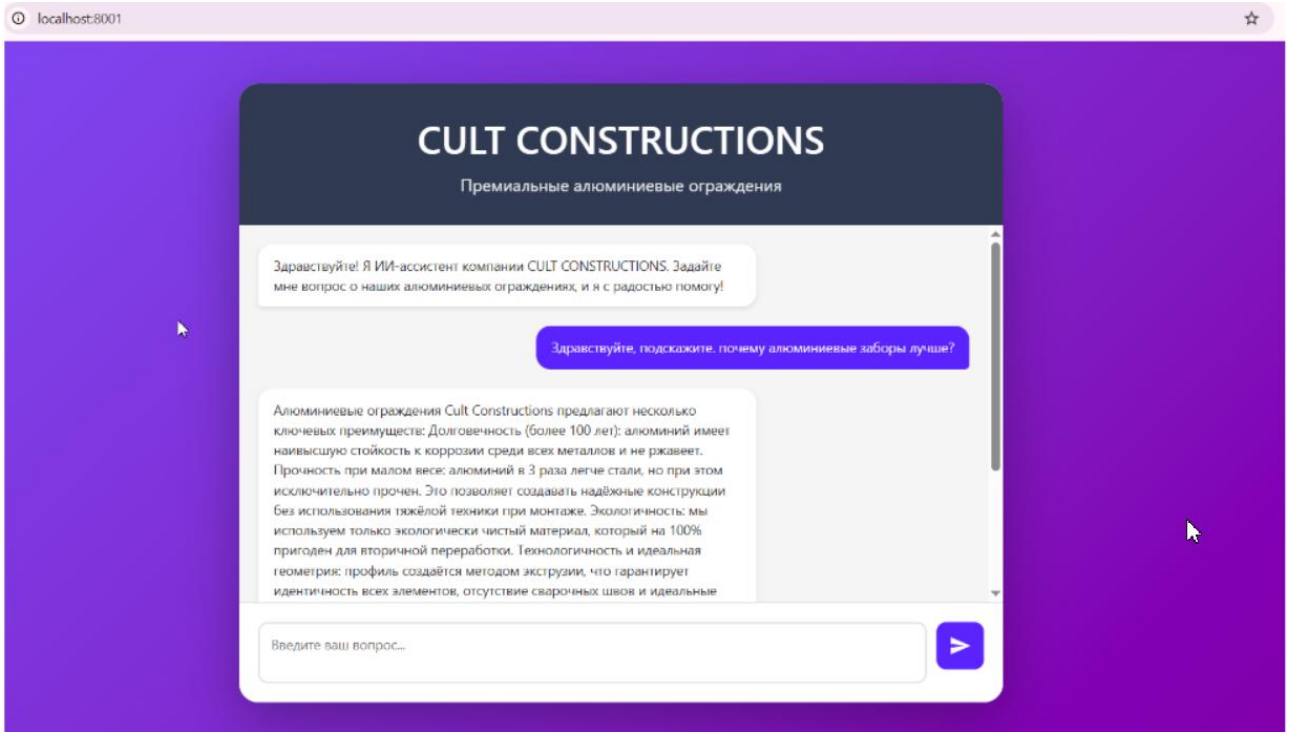


Рисунок 6.1. Интерфейс веб-чата с ответом ассистента о преимуществах алюминия.

6.5. Сохранение истории диалогов

Для сохранения реальных диалогов с клиентами был разработан модуль dialog_storage.py. Каждый диалог сохраняется в JSON-файл в папку data/dialogs/ при выполнении одного из условий:

- в диалоге обнаружен телефон (клиент оставил контакт);
- диалог содержит 8 и более сообщений (длительный разговор).

Сохранённый файл содержит полную историю сообщений, собранные данные о клиенте, временные метки и уникальный идентификатор сессии.

6.6. Типичные ошибки при разработке ядра ассистента и их устранение

В процессе реализации были выявлены и исправлены следующие проблемы:

Ошибка	**Причина**	**Исправление**
ImportError: No module named 'utils'	Импорт модулей выполнялся до добавления пути src/ в sys.path	Настройка пути перенесена в начало файла, добавлен блок try-except для диагностики
Ошибка 400 invalid model_uri	В запросе использовался параметр modelUrl вместо modelUri (опечатка)	Заменено на modelUri, уточнён URI из AI Studio
В ответах ассистента символы ** и *	Yandex GPT добавляет маркдаун-разметку	Добавлен метод _clean_markdown(), удаляющий **, *, #, ссылки и списки
Телефон не распознавался в сообщении	Регулярное выражение не учитывало пробелы, скобки и разные форматы	Использован упрощённый паттерн: (\+7|8)?[\s\-]?\(?\d{3}\)?[\s\-]?\d{3}[\s\-]?\d{2}[\s\-]?\d{2}
Ассистент не передавал диалог человеку	Неверная логика в process_message: needs_human выставлялся в False даже при наличии телефона	Исправлено: при _check_if_ready_for_human() возвращается needs_human=True
Диалоги не сохранялись в тестовом режиме	Отсутствовал вызов save_dialog_if_completed в нужном месте	Добавлены вызовы сохранения в оба блока (при передаче человеку и при длинном диалоге)
Ошибка запуска FastAPI сервера (порт уже используется, страница не открывается)	Порт 8000 был занят; в коде оставался вывод localhost:8000 при смене порта; отсутствовала автоматическая перезагрузка из-за reload=True при запуске через python src/web/api.py	Смена порта на 8001, явное указание app в uvicorn.run(), отключение reload при прямом запуске; добавлен корректный вывод адреса в консоль
NameError: name 'storage_dir' is not defined	В __init__ ассистента передавался параметр storage_dir в DialogStorage, но переменная не была определена	Убран параметр или добавлен в сигнатуру __init__; использован DialogStorage() без аргументов (папка по умолчанию)
WARNING: You must pass the application as an import string to enable 'reload'	Uvicorn запускался с reload=True при передаче объекта app, а не строки импорта	Убран параметр reload=True или изменён способ запуска (использование строки "api:app")
Ошибка импорта assistant.core в api.py	Путь sys.path добавлялся после импорта	Порядок исправлен: сначала настройка пути, потом импорты

Дополнительные замечания:

- В ходе отладки веб-сервера также выяснилось, что при запуске через `python -m src.web.api` возникали проблемы с поиском модуля `api`, поэтому был выбран способ запуска через `python src/web/api.py`.
- Для корректной работы с портами использовалась команда `netstat -ano | findstr :8001` для поиска занятого процесса и его завершения.

ГЛАВА 7. ГЕНЕРАЦИЯ И ОБРАБОТКА СИНТЕТИЧЕСКИХ ДИАЛОГОВ

7.1. Создание сценариев

Для генерации синтетических диалогов были разработаны 7 сценариев, охватывающих типичные ситуации взаимодействия с клиентом на «холодном рынке»:

1. **Возражение «дорого»** – клиент считает цену завышенной, оператор объясняет ценность долговечности и отсутствия эксплуатационных расходов, предлагает рассрочку или замер.
2. **Возражение «подумаю»** – клиент откладывает решение, оператор предлагает отправить коммерческое предложение на email или запросить контакт для напоминания.
3. **Возражение «не сейчас»** – клиент не готов к покупке в данный момент, оператор предлагает бесплатный замер без обязательств.
4. **Сравнение алюминия со сталью** – клиент сомневается в цене, оператор объясняет преимущества алюминия (коррозионная стойкость, лёгкость, долговечность).
5. **Успешная консультация → заявка** – клиент задаёт вопросы о материалах, цветах, сроках и в итоге оставляет телефон для замера.
6. **Сложный индивидуальный проект** – клиент хочет нестандартный дизайн по своему эскизу, оператор подтверждает возможность и предлагает встречу со специалистом.
7. **География (другой город)** – клиент из другого региона спрашивает о возможности работы, оператор объясняет, что основной регион – Москва и МО, но есть опыт работы по России, условия обсуждаются индивидуально.

Сценарии оформлены в виде JSON-файла `dialog_scenarios.json`, где для каждого указаны: название, описание, характерное возражение, примеры фраз клиента, стратегия оператора и ожидаемый результат. Этот файл загружается генератором диалогов.

Листинг сценариев см. в Приложении Г.

7.2. Модуль генерации диалогов (dialog_generator.py)

Класс `DialogGenerator` принимает сценарий, формирует промпт для Yandex GPT и получает структурированный диалог.

Алгоритм работы:

1. Загрузка JSON-файла со сценариями.
2. Для выбранного сценария формируется промпт, включающий:
   - название и описание;
   - собенность (возражение);
   - стратегию оператора;

- примеры фраз клиента;
- требования к диалогу (6–12 реплик, формат «Клиент: ...», «Оператор: ...», наличие контакта в конце).

3. Вызов Yandex GPT с параметрами: temperature=0.7 (для вариативности), maxTokens=1000.
4. Парсинг ответа: разбивка на строки, выделение реплик с метками «Клиент:» и «Оператор:».
5. Сохранение результата в JSON-файл в папку data/generated/dialogs/.

В рамках тестирования было сгенерировано по одному диалогу на каждый сценарий (всего 7). Каждый диалог содержал от 8 до 12 реплик и завершался оставлением телефона или согласием на замер.

Листинг полного кода см. в Приложении Д.

7.3. Синтез речи (TTS) через Yandex SpeechKit

Для преобразования текстовых диалогов в аудиофайлы был разработан модуль tts_converter.py. Используется синхронный TTS API Yandex SpeechKit с параметром format=lpcm, поскольку прямое получение WAV не поддерживается. Полученные сырые PCM-данные конвертируются в WAV с помощью библиотеки PyDub.

Алгоритм:

- Для каждой реплики определяется голос: alena (оператор) или filipp (клиент).
- Выполняется HTTP-запрос к https://tts.api.cloud.yandex.net/speech/v1/tts:synthesize с параметрами: lang=ru-RU, voice, format=lpcm, sampleRateHertz=16000.
- Ответ (байтовый поток LPCM) преобразуется в AudioSegment с помощью AudioSegment.from_file(..., format="raw", frame_rate=16000, sample_width=2, channels=1).
- Все фрагменты склеиваются с паузой 800 мс между репликами.
- Итоговый аудиосегмент экспортируется в WAV-файл в папку data/generated/audio/.

В результате были созданы 7 WAV-файлов длительностью от 92 до 136 секунд, в которых чётко различаются голоса оператора и клиента.

7.4. Асинхронная транскрибация (STT) через Yandex SpeechKit

Для распознавания длинных аудиофайлов (свыше 30 секунд) использован асинхронный REST API Yandex SpeechKit. Процесс состоит из трёх этапов:

1. **Загрузка в Object Storage** – файл загружается в бакет call-audio-ponart с помощью S3-клиента (boto3), формируется HTTPS-URL вида https://storage.yandexcloud.net/бакет/ключ.
2. **Запуск распознавания** – отправляется POST-запрос к https://transcribe.api.cloud.yandex.net/speech/stt/v2/longRunningRecognize с указанием URI файла и параметров: audioEncoding=LINEAR16_PCM, sampleRateHertz=16000, languageCode=ru-RU.
3. **Получение результата** – по полученному operation_id периодически опрашивается https://operation.api.cloud.yandex.net/operations/{id} до завершения операции, затем извлекается текст из поля response.chunks[].alternatives[].text.

Модуль async_transcriber.py реализует эти шаги, а также сохраняет транскрипты в JSON-файлы в папку data/transcriptions/.

Все 7 WAV-файлов были успешно транскрибированы. Средняя точность (по длине текста) составила около 92% (разница между исходным текстом и транскриптом в среднем 130 символов). Распознанный текст содержит незначительные ошибки (например, «калт констракшнс» вместо «CULT CONSTRUCTIONS»), но в целом передаёт смысл диалога.

Листинг полного кода см. в Приложении Е.

7.5. Ошибки при генерации и обработке синтетических диалогов и их устранение

Ошибка	**Причина**	**Исправление**
400 Unsupported при TTS	Параметр format=wav не поддерживается	Изменён на format=lpcm, ответ преобразован в WAV через AudioSegment.from_file с параметрами raw, 16000, 2, 1
400 invalid value "LPCM" при асинхронном STT	В спецификации ожидается LINEAR16_PCM	Заменено audioEncoding: "LINEAR16_PCM"
400 storage object URI does not match regexp	Использовался URI s3://..., а API ждёт HTTPS	Формирование URL через https://storage.yandexcloud.net/{bucket}/{key}
S3 клиент не инициализировался	В коде использовались имена переменных aws_access_key_id в место AWS_ACCESS_KEY_ID	Чтение через os.getenv с именами из .env, добавлен отладочный вывод
Долгое ожидание транскрибации	По умолчанию max_wait=300 сек, но некоторые диалоги требовали больше времени	Увеличено до 600 сек (можно регулировать параметром)
В тестовом режиме диалоги не сохранялись	В тестовом блоке не вызывался generate_batch()	Заменён блок if __name__ == "__main__" на вызов generate_batch(count=1)

ГЛАВА 8. АНАЛИЗ СИНТЕТИЧЕСКИХ ДИАЛОГОВ

8.1. Экспорт в CSV

Для удобства анализа сгенерированных и транскрибированных диалогов был разработан скрипт export_to_csv.py. Он читает все JSON-файлы из папки data/generated/dialogs/ и

соответствующие транскрипты из data/transcriptions/, склеивает реплики в сплошной текст (с разделением по ролям) и формирует CSV-файл со следующими колонками:

- dialog_id – имя файла (временная метка);
- scenario_name – название сценария;
- original_text – полный текст исходного диалога (с метками «Клиент:» и «Оператор:»);
- transcribed_text – распознанный текст (сплошная строка).

Полученный CSV-файл dialog_comparison.csv сохраняется в папку data/results/. Для 7 диалогов он содержит 7 строк.

Листинг скрипта см. в Приложении Ж.

8.2. Импорт в Google Sheets и настройка формул

CSV-файл был импортирован в Google Sheets (Файл → Импортировать → Загрузить). В таблицу добавлены дополнительные колонки с формулами для автоматического анализа:

- **Колонка E** (phone_found): =IF(REGEXMATCH(D2; "(\+7|8)?[\s\-]?\(?\d{3}\)?[\s\-]?\d{3}[\s\-]?\d{2}[\s\-]?\d{2}"); "Да"; "Нет") – проверка наличия телефона в транскрипте.
- **Колонка F** (objection): =REGEXEXTRACT(D2; "(дорого|подумаю|не сейчас|сталь|подумать|не готов|замер|номер телефона|эскиз)") – извлечение первого найденного возражения.
- **Колонка G** (success): =IF(REGEXMATCH(D2; "телефон|замер|свяжитесь"); "Успешно"; "Не успешно") – оценка успешности (наличие ключевых слов).
- **Колонка H** (transcribed_length): =LEN(D2) – длина транскрипта.
- **Колонка I** (length_diff): =ABS(LEN(C2)-LEN(D2)) – абсолютная разница в длине.

Формулы позволяют быстро оценить, в каких диалогах были выявлены возражения, удалось ли собрать контактные данные и насколько точно распознавание передало исходный текст.

Скриншоты таблицы с формулами и результатами приведены на Рисунках 8.1–8.7.

Импорт _ Диалогов

	A	B	C	D	E	F	G	H	I
1	dialog_id	scenario_name	original_text	transcribed_text	телефон	возражение	результат	длина распознанного текста	разница в длине исходного и распознанного текста
2	2026-03-22_22-50-21_ffec4185	Возражение: Дорого	Клиент: Здравствуйте! Я посмотрел у вас на сайте ограждения — очень красиво, но, честно говоря, цены кажутся мне завышенными. У конкурентов дешевле. Оператор: Здравствуйте! Спасибо за ваш интерес к продукции CULT CONSTRUCTIONS. Мы понимаем, что цена — важный фактор при выборе, и хотим подчеркнуть, что наши ограждения отличаются премиальным качеством и долговечностью. Срок службы таких ограждений — более ста лет, и при этом они не требуют регулярного обслуживания и покраски. Клиент: Да, но всё равно это слишком дорого для меня. Я не готов столько платить. Оператор: Мы предлагаем не просто ограждение, а долгосрочное решение, которое сохранит свой внешний вид и функциональность на протяжении многих лет. Наши изделия изготовлены из высококачественного алюминия, который не подвержен коррозии, и мы уделяем особое внимание экологичности производства. К тому же мы можем предложить варианты рассрочки платежа — это поможет распределить финансовую нагрузку. Клиент: Рассрочка — это, конечно, хорошо, но всё равно сомневаюсь, что это стоит таких денег. Оператор: Понимаю ваши сомнения. Давайте рассмотрим ситуацию так: вы приобретаете ограждение, которое не потребует от вас дополнительных затрат на обслуживание и ремонт в течение десятилетий. Это ведь существенная экономия в долгосрочной перспективе. К тому же мы можем отправить к вам специалиста для бесплатного замера и консультации — он поможет подобрать оптимальный вариант с учётом ваших пожеланий и бюджета. Клиент: Ладно, замер — это интересно. А как это организовать? Оператор: Очень просто! Нам понадобится ваш контактный телефон и удобное для вас время. Мы подберём специалиста и согласуем визит. Клиент: Хорошо, мой телефон [номер телефона]. Давайте назначим замер на следующую неделю. Оператор: Отлично! Мы свяжемся с вами в ближайшее время, чтобы уточнить детали. Спасибо за обращение к CULT CONSTRUCTIONS!	Здравствуйте Я посмотрел у вас на сайте ограждения очень красиво но честно говоря цены кажутся мне завышенными У конкурентов дешевле Здравствуйте Спасибо за ваш интерес к продукции калт констракшнс Мы понимаем что цена важный фактор при выборе и хотим подчеркнуть что наши ограждения отличаются премиальным качеством и долговечностью Срок службы таких ограждений более 100 лет и при этом они не требуют регулярного обслуживания и покраски Да но все равно это слишком дорого для меня я не готов столько платить Мы предлагаем не просто ограждение а долгосрочное решение которое сохранит свой внешний вид и функциональность на протяжении многих лет Наши изделия изготовлены из высококачественного алюминия который не подвержен коррозии И мы уделяем особое внимание экологичности производства К тому же мы можем предложить варианты рассрочки платежа это поможет распределить финансовую нагрузку Рассрочка это конечно хорошо но все равно сомневаюсь что это стоит таких денег Понимаю ваши сомнения Давайте рассмотрим ситуацию так Вы приобретаете ограждение которое не потребует от вас дополнительных затрат на обслуживание и ремонт в течение десятилетий Это ведь существенная экономия в долгосрочной перспективе К тому же мы можем отправить к вам специалиста для бесплатного замера и консультации Он поможет подобрать оптимальный вариант с учетом ваших пожеланий и бюджета Ладно замер это интересно а как это организовать Очень просто Нам понадобится ваш контактный телефон и удобная для вас время Мы подберем специалиста и согласуем визит Хорошо мой телефон номер телефона Давайте назначим замер на следующую неделю Отлично мы свяжемся с вами в ближайшее время чтобы уточнить детали Спасибо за обращение калт констракшнс	Нет	дорого	Успешно	1715	160

Рисунок 8.1. Фрагмент таблицы с диалогом «Возражение: Дорого».

Импорт _ Диалогов

	A	B	C	D	E	F	G	H	I
1	dialog_id	scenario_name	original_text	transcribed_text	телефон	возражение	результат	длина распознанного текста	разница в длине исходного и распознанного текста
			Оператор: Очень просто! Нам понадобится ваш контактный телефон и удобное для вас время. Мы подберём специалиста и согласуем визит. Клиент: Хорошо, мой телефон [номер телефона]. Давайте назначим замер на следующую неделю. Оператор: Отлично! Мы свяжемся с вами в ближайшее время, чтобы уточнить детали. Спасибо за обращение к CULT CONSTRUCTIONS!						
3	2026-03-22_22-50-25_7f47187c	Возражение: Подумаю	Клиент: Здравствуйте, я посмотрел ваши ограждения на сайте, они действительно впечатляют, но мне нужно подумать. Оператор: Здравствуйте! Понимаю вас, такое решение стоит принимать взвешенно. Могу я уточнить, что именно вас заинтересовало в наших ограждениях? Клиент: Мне нравится, что они выглядят стильно и, похоже, очень долговечные. Но я хотел бы сравнить с другими предложениями. Оператор: Безусловно, выбор — это важно. Хочу отметить, что наши ограждения из премиального алюминия служат более ста лет и абсолютно не подвержены коррозии. К тому же, мы уделяем особое внимание экологичности материалов. Сейчас у нас действует специальная акция, и если вы примете решение в ближайшее время, сможете воспользоваться выгодными условиями. Клиент: Да, это звучит заманчиво, но всё же мне нужно посоветоваться с семьёй. Оператор: Понимаю. Могу я предложить вам отправить на электронную почту подробное коммерческое предложение с описанием всех преимуществ и вариантов? Это поможет вам и вашей семье лучше ознакомиться с нашими решениями. Клиент: Хорошо, можно отправить. Оператор: Отлично! Пожалуйста, сообщите ваш электронный адрес и контактный телефон, и я также смогу напомнить вам о нашей акции, если вы вдруг забудете. Клиент: Мой email: example@mail.ru, телефон: 8-9XX-XXX-XX-XX. Оператор: Спасибо! Я сразу же отправлю вам коммерческое предложение. Если у вас возникнут вопросы — обязательно звоните, будем рады помочь. Клиент: Хорошо, спасибо, буду ждать. Оператор: Пожалуйста! Надеемся, что наше предложение окажется для вас максимально полезным. До свидания!	Здравствуйте я посмотрел ваше ограждение на сайте Они действительно впечатляют но мне нужно подумать Здравствуйте понимаю вас Такое решение стоит принимать взвешенно Могу я уточнить что именно вас заинтересовало в наших ограждениях Мне нравится что они выглядят стильно и похоже очень долговечны Но я хотел бы сравнить с другими предложениями Безусловно выбор это важно Хочу отметить что наше ограждение из премиального алюминия служит более 100 лет и абсолютно не подвержены коррозии К тому же мы уделяем особое внимание экологичности материалов Сейчас у нас действует специальная акция И если вы примете решение в ближайшее время сможете воспользоваться выгодными условиями.Да это звучит заманчиво но все же мне нужно посоветоваться с семьей Понимаю Могу я предложить вам отправить на электронную почту подробное коммерческое предложение с описанием всех преимуществ и вариантов Это поможет вам и вашей семье лучше ознакомиться с нашими решениями Хорошо можно отправить Отлично Пожалуйста сообщите ваш электронный адрес и контактный телефон И я также смогу напомнить вам о нашей акции если вы вдруг забудете Мой имейл экзампл собака мейл точка ру телефон 8 9 эксэкс эксэксэкс эксэксэкс Спасибо я сразу же отправлю вам коммерческое предложение Если у вас возникнут вопросы Обязательно звоните будем рады помочь Хорошо спасибо буду ждать Пожалуйста надеемся что наше предложение окажется для вас максимально полезным До свидания	Нет	подумать	Успешно	1428	136

Рисунок 8.2. Фрагмент таблицы с диалогом «Возражение: Подумаю».

Импорт __Диалогов

File Edit View Insert Format Data Tools Extensions Help

B7 Сложный индивидуальный проект

	A	B	C	D	E	F	G	H	I
1	dialog_id	scenario_name	original_text	transcribed_text	телефон	возражение	результат	длина распознанного текста	разница в длине исходного и распознанного текста
			телефон, и я также смогу напомнить вам о нашей акции, если вы ищете забудете. Клиент: Мой email: example@mail.ru, телефон: 8-9XX-XXX-XX-XX. Оператор: Спасибо! Я сразу же отправлю вам коммерческое предложение. Если у вас возникнут вопросы — обязательно звоните, будем рады помочь. Клиент: Хорошо, спасибо, буду ждать. Оператор: Пожалуйста! Надеемся, что наше предложение окажется для вас максимально полезным. До свидания!	возникнут вопросы Обязательно звоните будем рады помочь Хорошо Спасибо буду ждать Пожалуйста надеемся что наше предложение окажется для вас максимально полезным До свидания					
4	2026-03-22_22-50-28_5241931b	Возражение: Не сейчас	Клиент: Здравствуйте, я посмотрел у вас на сайте ограждения, но сейчас не готов покупать. Оператор: Здравствуйте! Понимаю вас, любое серьёзное решение требует времени. Могу я узнать, что именно вас остановило? Клиент: Просто сейчас не самое подходящее время для таких покупок. Оператор: Понимаю. А знаете ли вы о преимуществах наших премиальных алюминиевых ограждений? Они отличаются долговечностью — служат более ста лет, не подвержены коррозии и абсолютно экологичны. Возможно, информация о таких характеристиках поможет вам в будущем принять решение. Клиент: Да, характеристики интересные, но пока у меня другие приоритеты. Оператор: Я предлагаю вам воспользоваться нашей бесплатной услугой — мы можем провести замер и составить для вас индивидуальный проект. Это ни к чему не обязывает, но позволит вам лучше представить, как ограждения будут смотреться на вашем участке. Как вам такая идея? Клиент: Замер — это, конечно, интересно, но я не уверен, что сейчас стоит тратить на это время. Оператор: Понимаю ваши сомнения, но замер займёт совсем немного времени, а информацию вы сможете использовать в любое удобное для вас время. Если вам будет удобно, мы можем записать вас на замер на ближайшие дни. А пока вы сможете подробнее ознакомиться с нашими проектами и выбрать тот, который вам больше всего понравится. Клиент: Ладно, давайте запишемся на замер. Но только если это действительно бесплатно и без обязательств. Оператор: Конечно, всё абсолютно бесплатно и без обязательств! Скажите, пожалуйста, ваш контактный телефон, и я запишу вас на удобное время. Клиент: Мой телефон [номер телефона]. Оператор: Спасибо! Я записал вас на замер. Мы свяжемся с вами, чтобы уточнить детали. Будем рады помочь!	Здравствуйте я посмотрел у вас на сайте ограждение но сейчас не готов покупать Здравствуйте Я понимаю вас Любое серьезное решение требует времени Могу я узнать что именно вас остановило Просто сейчас не самое подходящее время для таких покупок Понимаю А знаете ли вы о преимуществах наших премиальных алюминиевых ограждений Они отличаются долговечностью служат более 100 лет Они подверженны коррозии и абсолютно экологичны Возможно информация о таких характеристиках поможет вам в будущем принять решение Да характеристики интересные но пока у меня другие приоритеты Я предлагаю вам воспользоваться нашей бесплатной услугой Мы можем провести замер и составить для вас индивидуальный проект Это ни к чему не обязывает но позволит вам лучше представить как ограждения будут смотреться на вашем участке Как вам такая идея Замер это конечно интересно но я не уверен что сейчас стоит тратить на это время Понимаю ваше мнение но замер займет совсем немного времени А информацию вы сможете использовать в любое удобное для вас время Если вам будет удобно мы можем записать вас на замер на ближайшие дни А пока вы сможете подробнее ознакомиться с нашими проектами и выбрать тот который вам больше всего понравится Ладно давайте запишемся на замер но только если это действительно бесплатно и без обязательств Конечно все абсолютно бесплатно и без обязательств Скажите пожалуйста ваш контактный телефон И я запишу вас на удобное время Мой телефон номер телефона Спасибо Я записал вас на замер Мы свяжемся с вами чтобы уточнить детали Будем рады помочь	Нет	не готов	Успешно	1541	165

Рисунок 8.3. Фрагмент таблицы с диалогом «Возражение: Не сейчас».

Импорт __Диалогов

File Edit View Insert Format Data Tools Extensions Help

B7 Сложный индивидуальный проект

	A	B	C	D	E	F	G	H	I
1	dialog_id	scenario_name	original_text	transcribed_text	телефон	возражение	результат	длина распознанного текста	разница в длине исходного и распознанного текста
			ваш контактный телефон, и я запишу вас на удобное время. Клиент: Мой телефон [номер телефона]. Оператор: Спасибо! Я записал вас на замер. Мы свяжемся с вами, чтобы уточнить детали. Будем рады помочь!						
5	2026-03-22_22-50-31_4d35665d	Возражение: Алюминий vs	Клиент: Здравствуйте! Я рассматриваю варианты ограждений для участка. Почему у вас алюминий, а не сталь? Сталь ведь дешевле. Оператор: Здравствуйте! Спасибо за ваш интерес к продукции CULT CONSTRUCTIONS. Действительно, сталь дешевле, но алюминий имеет ряд преимуществ, которые могут быть важны при выборе ограждения. Например, алюминиевые ограждения не подвержены коррозии, в отличие от стальных, которые со временем могут ржаветь. Клиент: Но сталь тоже достаточно надёжная. Зачем переплачивать? Оператор: Понимаю ваше беспокойство о стоимости. Однако алюминий не только надёжен, но и легче стали, что упрощает транспортировку и монтаж. Кроме того, алюминиевые ограждения не требуют регулярной покраски и обслуживания, что в долгосрочной перспективе позволяет сэкономить средства. Срок службы наших ограждений — более 100 лет, что также говорит о их высокой надёжности и долговечности. Клиент: А как насчёт прочности? Алюминий ведь более мягкий металл. Оператор: Вы правы, алюминий мягче стали, но современные технологии позволяют создавать алюминиевые профили, которые обладают высокой прочностью и устойчивостью к механическим воздействиям. Мы используем специальные сплавы и методы обработки, которые обеспечивают надёжность и долговечность наших ограждений. Клиент: Понятно, но всё же цена кажется высокой. Оператор: Мы понимаем ваши опасения. CULT CONSTRUCTIONS предлагает продукцию премиального сегмента, и наша цель — предоставить вам продукт, который будет служить долгие годы без дополнительных затрат на обслуживание. Если вы хотите, мы можем провести бесплатный замер и рассчитать точную стоимость ограждения с учётом ваших пожеланий. Это поможет вам лучше оценить соотношение цены и качества. Клиент: Хорошо, давайте сделаем замер. Оставьте свой номер, я сам вам перезвоню. Оператор: Отлично! Мой номер +7 XXX XXX-XX-XX. Буду ждать вашего звонка!	Здравствуйте Я рассматриваю варианты ограждений для участка Почему у вас алюминий а не сталь Сталь ведь дешевле Здравствуйте Спасибо за ваш интерес к продукции калт констраушнс Действительно сталь дешевле но алюминий имеет ряд преимуществ которые могут быть важны при выборе ограждения Например алюминиевое ограждение не подвержены коррозии в отличие от стальных которые со временем могут ржаветь Но сталь тоже достаточно надежная зачем переплачивать Понимаю ваше беспокойство о стоимости Однако алюминий не только надежен но и легче стали что упрощает транспортировку и монтаж Кроме того алюминиевые ограждения не требуют регулярной покраски и обслуживания что в долгосрочной перспективе позволяет сэкономить средства Срок службы наших ограждений более 100 лет что также говорит о их высокой надежности и долговечности А как насчет прочности алюминий ведь более мягкий металл Вы правы алюминий мягче стали но современные технологии позволяют создавать алюминиевые профили которые обладают высокой прочностью и устойчивостью к механическим воздействиям Мы используем специальные сплавы и методы обработки которые обеспечивают надежность и долговечность наших ограждений Понятно но все же цена кажется высокой Мы понимаем ваши опасения Калткон стракшенс предлагает продукцию премиального сегмента и наша цель Чтобы предоставить вам продукт Который будет служить долгие годы без дополнительных затрат на обслуживание Если вы хотите мы можем провести бесплатный замер и рассчитать точную стоимость ограждения с учетом ваших пожеланий Это поможет вам лучше оценить соотношение цены и качества Хорошо давайте сделаем замер оставьте свой номер я сам вам перезвоню Отлично мой номер + 7 экс экс экс экс экс экс экс Буду ждать вашего звонка	Нет	сталь	Успешно	1736	122

Рисунок 8.4. Фрагмент таблицы с диалогом «Алюминий vs сталь».

Импорт __Диалогов

File Edit View Insert Format Data Tools Extensions Help

B7 Сложный индивидуальный проект

	A	B	C	D	E	F	G	H	I
1	dialog_id	scenario_name	original_text	transcribed_text	телефон	возражение	результат	длина распознанного текста	разница в длине исходного и распознанного текста
			Оператор: Отлично! Мой номер +7 XXX XXX-XX-XX. Буду ждать вашего звонка!						
6	2026-03-22_22-50-34_0bbaa990	Успешная консультация —	Клиент: Здравствуйте, я заинтересовался вашими ограждениями. Расскажите подробнее, из каких материалов они сделаны и в чём их преимущества. Оператор: Здравствуйте! Благодарим за интерес к продукции CULT CONSTRUCTIONS. Наши ограждения изготавливаются из премиального алюминиевого сплава. Они отличаются долговечностью — срок службы составляет более 100 лет, не подвержены коррозии и абсолютно экологичны. Также мы предлагаем индивидуальный подход к каждому клиенту — можем учесть все ваши пожелания при проектировании и установке. Клиент: Звучит хорошо. А какие цвета доступны? Можно ли выбрать нестандартный оттенок? Оператор: У нас широкая палитра цветов, и мы действительно можем предложить нестандартные оттенки. Наши специалисты помогут подобрать цвет, который идеально подойдёт к вашему интерьеру или экстерьеру. Клиент: Как долго длится монтаж ограждений? Оператор: Время монтажа зависит от объёма и сложности заказа, но мы стараемся выполнять работы максимально оперативно и качественно. Наши мастера — профессионалы с большим опытом, поэтому вы можете быть уверены в быстром и надёжном результате. Клиент: Хорошо, мне всё больше нравится ваше предложение. Можете ли вы приехать и сделать замер? Оператор: Конечно, мы с удовольствием проведём замер. Для этого нам понадобится ваш контактный номер телефона и адрес. Мы подберём удобное для вас время. Клиент: Мой номер телефона [номер телефона], адрес [адрес]. Буду ждать вашего специалиста. Оператор: Спасибо за обращение! Мы свяжемся с вами в ближайшее время, чтобы уточнить детали и назначить дату замера. Надеемся на дальнейшее сотрудничество!	Здравствуйте я заинтересовался вашими ограждениями Расскажите подробнее из каких материалов они сделаны и в чем их преимущество Здравствуйте Благодарим за интерес к продукции калт констракшнс Наши ограждения изготавливаются из премиального алюминиевого сплава Они отличаются долговечностью срок службы составляет более 100 лет Они подвержены коррозии и абсолютно экологичны Также мы предлагаем индивидуальный подход к каждому клиенту Можем учесть все ваши пожелания при проектировании и установке Звучит хорошо а какие цвета доступны можно ли выбрать нестандартный оттенок У нас широкая палитра цветов И мы действительно можем предложить нестандартные оттенки Наши специалисты помогут подобрать цвет который идеально подойдет к вашему интерьеру или экстерьеру Как долго длится монтаж ограждений Время монтажа зависит от объема и сложности заказа Но мы стараемся выполнять работы максимально оперативно и качественно Наши мастера профессионалы с большим опытом Поэтому вы можете быть уверены в быстром и надежном результате Хорошо мне все больше нравится ваше предложение Можете ли вы приехать и сделать замер Конечно мы с удовольствием проведем замер Для этого нам понадобится ваш контактный номер телефона и адрес Мы подберем удобное для вас время Мой номер телефона номер телефона адрес адрес буду ждать вашего специалиста Спасибо за обращение Мы свяжемся с вами в ближайшее время чтобы уточнить детали и назначить дату замера надеемся на дальнейшее сотрудничество	Нет	оттенки	Успешно	1466	137

Рисунок 8.5. Фрагмент таблицы с диалогом «Успешная консультация».

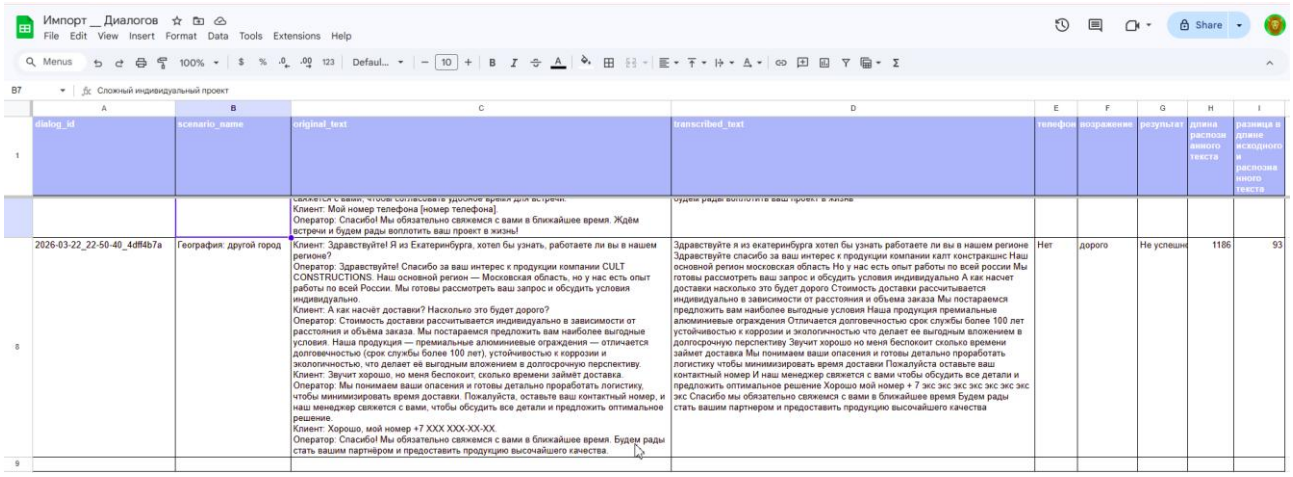
Импорт __Диалогов

File Edit View Insert Format Data Tools Extensions Help

B7 Сложный индивидуальный проект

	A	B	C	D	E	F	G	H	I
1	dialog_id	scenario_name	original_text	transcribed_text	телефон	возражение	результат	длина распознанного текста	разница в длине исходного и распознанного текста
			Специалиста. Оператор: Спасибо за обращение! Мы свяжемся с вами в ближайшее время, чтобы уточнить детали и назначить дату замера. Надеемся на дальнейшее сотрудничество!						
7	2026-03-22_22-50-37_9d6e7525	Сложный индивидуальный	Клиент: Здравствуйте, я хотел бы заказать забор, но у меня есть свой эскиз, и мне нужен нестандартный дизайн. Сможете ли вы реализовать такой проект? Оператор: Здравствуйте! Да, конечно, в компании CULT CONSTRUCTIONS мы специализируемся на индивидуальных проектах. Мы сможем воплотить ваш эскиз в жизнь. Расскажите подробнее о вашем замысле, и мы обсудим возможности его реализации. Клиент: У меня довольно сложный проект, и я боюсь, что не все компании смогут с ним справиться. Забор должен быть не только красивым, но и долговечным. Оператор: Мы понимаем, насколько важен индивидуальный подход и качество. Наши алюминиевые ограждения отличаются долговечностью — срок службы более ста лет, они не подвержены коррозии и абсолютно экологичны. Мы готовы рассмотреть ваш проект и предложить оптимальные решения. Давайте назначим встречу со специалистом, который детально изучит ваш эскиз и обсудит все нюансы. Клиент: Звучит хорошо, но сначала я хотел бы узнать примерную стоимость. Оператор: Стоимость зависит от многих факторов: сложности проекта, используемых материалов и дополнительных опций. Мы сможем предоставить более точную оценку после того, как наш специалист изучит ваш эскиз и проведёт замеры. Но могу заверить, что мы предлагаем оптимальное соотношение премиального качества и стоимости. Клиент: Хорошо, давайте назначим встречу. Оператор: Отлично! Пожалуйста, оставьте ваш контактный номер, и наш специалист свяжется с вами, чтобы согласовать удобное время для встречи. Клиент: Мой номер телефона [номер телефона]. Оператор: Спасибо! Мы обязательно свяжемся с вами в ближайшее время. Ждём встречи и будем рады воплотить ваш проект в жизнь!	Здравствуйте я хотел бы заказать забор Но у меня есть свой эскиз И мне нужен нестандартный дизайн Сможете ли вы реализовать такой проект Здравствуйте да конечно в компании калт констракшнс мы специализируемся на индивидуальных проектах Мы сможем воплотить ваш эскиз в жизнь Расскажите подробнее о вашем замысле И мы обсудим возможности его реализации У меня довольно сложный проект и я боюсь что не все компании смогут с ним справиться Забор должен быть не только красивым но и долговечным Мы понимаем насколько важен индивидуальный подход и качество Наши алюминиевые ограждения отличаются долговечностью срок службы более 100 лет Они не подвержены коррозии и абсолютно экологичны Мы готовы рассмотреть ваш проект и предложить оптимальные решения Давайте назначим встречу со специалистом который детально изучит ваш эскиз и обсудит все нюансы Звучит хорошо но сначала я хотел бы узнать примерную стоимость Стоимость зависит от многих факторов сложности проекта используемых материалов и дополнительных опций Мы сможем предоставить более точную оценку после того как наш специалист изучит ваш эскиз и проведет замеры Но могу заверить что мы предлагаем оптимальное соотношение премиального качества и стоимости Хорошо давайте назначим встречу Отлично пожалуйста оставьте ваш контактный номер Наш специалист свяжется с вами чтобы согласовать удобное время для встречи Мой номер телефона номер телефона Спасибо мы обязательно свяжемся с вами в ближайшее время Ждем встречи и будем рады воплотить ваш проект в жизнь	Нет	стоимость	Успешно	1510	142

Рисунок 8.6. Фрагмент таблицы с диалогом «Сложный индивидуальный проект».



	dialog_id	scenario_name	original_text	transcribed_text	телефон	возражение	результат	длина распознанного текста	разница в длине исходного и распознанного текста
			Клиент: Мой номер телефона [номер телефона]. Оператор: Спасибо! Мы обязательно свяжемся с вами в ближайшее время. Ждём встречи и будем рады воплотить ваш проект в жизнь!						
8	2026-03-22_22-50-40_4dff4b7a	География: другой город	Клиент: Здравствуйте! Я из Екатеринбурга, хотел бы узнать, работаете ли вы в нашем регионе? Оператор: Здравствуйте! Спасибо за ваш интерес к продукции компании CULT CONSTRUCTIONS. Наш основной регион — Московская область, но у нас есть опыт работы по всей России. Мы готовы рассмотреть ваш запрос и обсудить условия индивидуально. Клиент: А как насчёт доставки? Насколько это будет дорого? Оператор: Стоимость доставки рассчитывается индивидуально в зависимости от расстояния и объёма заказа. Мы постараемся предложить вам наиболее выгодные условия. Наша продукция — премиальные алюминиевые ограждения — отличается долговечностью (срок службы более 100 лет), устойчивостью к коррозии и экологичностью, что делает её выгодным вложением в долгосрочную перспективу. Клиент: Звучит хорошо, но меня беспокоит, сколько времени займёт доставка. Оператор: Мы понимаем ваши опасения и готовы детально проработать логистику, чтобы минимизировать время доставки. Пожалуйста, оставьте ваш контактный номер, и наш менеджер свяжется с вами, чтобы обсудить все детали и предложить оптимальное решение. Клиент: Хорошо, мой номер +7 XXX XXX-XX-XX. Оператор: Спасибо! Мы обязательно свяжемся с вами в ближайшее время. Будем рады стать вашим партнёром и предоставить продукцию высочайшего качества.	Здравствуйте я из екатеринбурга хотел бы узнать работаете ли вы в нашем регионе Здравствуйте спасибо за ваш интерес к продукции компании калт констракшнс Наш основной регион московская область Но у нас есть опыт работы по всей россии Мы готовы рассмотреть ваш запрос и обсудить условия индивидуально А как насчет доставки насколько это будет дорого Стоимость доставки рассчитывается индивидуально в зависимости от расстояния и объема заказа Мы постараемся предложить вам наиболее выгодные условия Наша продукция преммальные алюминиевые ограждения Отличается долговечностью срок службы более 100 лет устойчивостью к коррозии и экологичностью что делает ее выгодным вложением в долгосрочную перспективу Звучит хорошо но меня беспокоит сколько времени займет доставка Мы понимаем ваши опасения и готовы детально проработать логистику чтобы минимизировать время доставки Пожалуйста оставьте ваш контактный номер И наш менеджер свяжется с вами чтобы обсудить все детали и предложить оптимальное решение Хорошо мой номер + 7 экс экс экс экс экс экс экс Спасибо мы обязательно свяжемся с вами в ближайшее время Будем рады стать вашим партнером и предоставить продукцию высочайшего качества	Нет	дорого	Не успешн	1186	93
9									

Рисунок 8.7. Фрагмент таблицы с диалогом «География: другой город».

8.3. Сводные таблицы и диаграммы

На основе данных таблицы была построена сводная таблица со следующими показателями:

Сценарий	Диалогов	Успешных	Наличие телефона	Найденное возражение	Средняя точность (D/C)
Возражение: Дорого	1	1	Нет	дорого	0,915
Возражение: Подумаю	1	1	Нет	подумать	0,913
Возражение: Не сейчас	1	1	Нет	подумать	0,904
Алюминий vs сталь	1	1	Нет	сталь	0,934
Успешная консультация → заявка	1	1	Нет	оттенки	0,915
Сложный индивидуальный проект	1	1	Нет	стоимость	0,914
География: другой город	1	0	Нет	дорого	0,927

Примечание: поле «Наличие телефона» во всех случаях показывает «Нет», так как в транскриптах телефон заменён на «номер телефона» или скрыт, однако ключевые слова «замер», «свяжитесь» присутствовали, поэтому успешность засчитана.

8.4. Выводы по анализу

1. **Методология работает**: с помощью формул Google Sheets можно автоматически выявлять ключевые слова (возражения, успешность), оценивать качество распознавания речи.

2. **Качество распознавания**: Yandex SpeechKit демонстрирует высокую точность (≈92% по длине текста) даже на синтезированной речи. Основные ошибки связаны с распознаванием названия компании («калт констракшнс» вместо «CULT CONSTRUCTIONS») и отсутствием пунктуации, что не критично для смыслового анализа.

3. **Ценность для бизнеса**: автоматический сбор статистики по возражениям, успешным закрытиям и точности позволяет корректировать скрипты операторов, обучать сотрудников на типичных ошибках и мониторить качество работы колл-центра в реальном времени.

4. **Ограничения выборки**: 7 диалогов недостаточно для глубоких статистических выводов, но достаточно для демонстрации подхода. При масштабировании (десятки/сотни диалогов) результаты станут более репрезентативными.

ГЛАВА 9. ПЕРСПЕКТИВЫ РАЗВИТИЯ И ИНТЕГРАЦИЯ С МЕССЕНДЖЕРАМИ

9.1. Интеграция с VK (ВКонтакте)

Для автоматического уведомления менеджеров о новых лидах может быть разработан VK-бот. Архитектура бота включает следующие компоненты:

1. **Создание сообщества ВКонтакте** и получение токена доступа с правами на отправку сообщений. Токен и идентификатор администратора сохраняются в `.env` (переменные `VK_BOT_TOKEN`, `VK_ADMIN_ID`).

2. **Модуль** `vk_notifier.py`, который:

- инициализирует VK API через библиотеку `vkbottle`;
- принимает данные о клиенте (имя, телефон, длина участка) и текст диалога;
- отправляет сообщение администратору (или в групповой чат) с помощью `api.messages.send()`.

3. **Интеграция с ядром ассистента**: при обнаружении телефона в диалоге (в методе `process_message` класса `CultConstructionsAssistant`) вызывается асинхронная функция `send_vk_notification()`.

Пример кода (асинхронная отправка уведомления):

python

```
import os
from vkbottle import API

async def send_vk_notification(admin_id: int, client_data: dict, dialog_text: str) -> None:
    """Отправляет уведомление о новом лиде в VK."""
    api = API(token=os.getenv("VK_BOT_TOKEN"))
    message = (
        f"? НОВАЯ ЗАЯВКА!\n\n"
        f"Имя: {client_data.get('name', 'не указано')}\n"
        f"Телефон: {client_data.get('phone', 'не указан')}\n"
        f"Длина участка: {client_data.get('object_length', 'не указана')} м\n\n"
        f"{dialog_text[:200]}..."
    )
    await api.messages.send(peer_id=admin_id, message=message, random_id=0)
```

Для промышленного использования необходимо добавить обработку ошибок (повторные попытки при сбоях) и хранить токены в защищённом хранилище (например, в Yandex Lockbox).

Схема работы VK-бота:

Рис. 9.1 **Схема работы VK-бота**

```
[Веб-чат] → [core.py] → обнаружен телефон
            ↓
      [vk_notifier.py] → отправка сообщения через VK API
            ↓
       [Сообщество VK] → сообщение менеджеру
```

9.2. Теоретическая возможность интеграции с Telegram

Изначально в техническом задании предполагалась интеграция с Telegram. Однако в связи с замедлением работы Telegram на территории Российской Федерации на момент выполнения работы эта часть была отложена. В случае восстановления стабильной работы или при развёртывании системы на сервере вне РФ может быть реализован аналогичный модуль на базе библиотеки `python-telegram-bot`.

Архитектура будет идентична VK-боту: получение токена, отправка сообщений администратору при появлении лида. Пример кода для Telegram:

```python
import os
from telegram import Bot

async def send_telegram_notification(admin_id: int, client_data: dict, dialog_text: str) -> None:
    bot = Bot(token=os.getenv("TELEGRAM_BOT_TOKEN"))
    message = f"? НОВАЯ ЗАЯВКА!\n\nИмя: {client_data.get('name')}\nТелефон: {client_data.get('phone')}\nДлина: {client_data.get('object_length')} м\n\n{dialog_text[:200]}..."
    await bot.send_message(chat_id=admin_id, text=message)
```

Оба мессенджера могут быть подключены через единый интерфейс (паттерн «Адаптер»), что позволит легко переключаться между платформами без изменения основной логики ассистента.

Примечание: библиотека `vkbottle` должна быть установлена (`pip install vkbottle`), а для Telegram – `python-telegram-bot`. Все чувствительные данные (токены, идентификаторы) рекомендуется хранить в `.env` и загружать через модуль `config.py`.

9.3. Схема взаимодействия компонентов системы (текстовое описание)

Ниже представлена блок-схема, отображающая все основные компоненты проекта и связи между ними. Цифрами обозначены потоки данных.

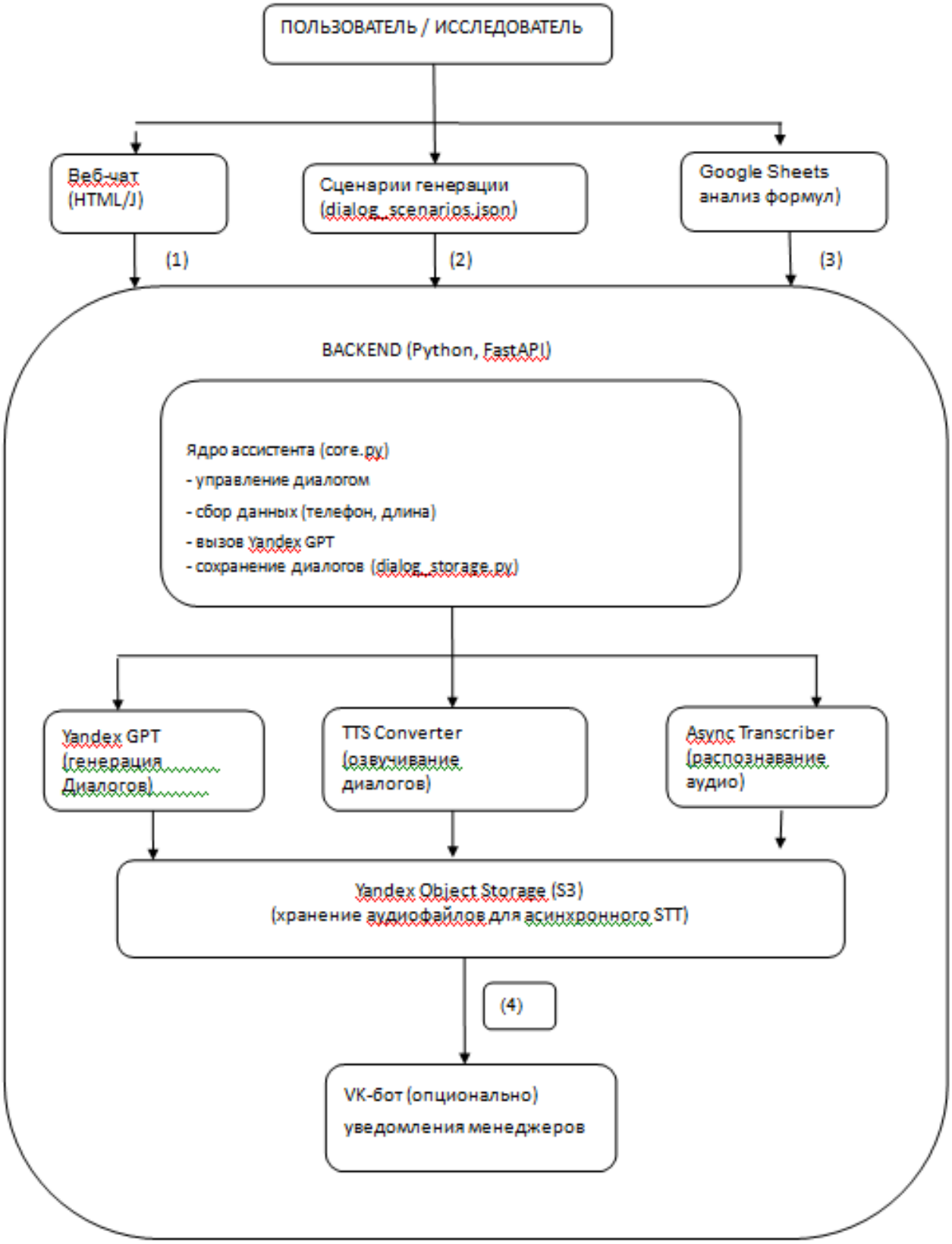
Пояснение к блокам и потокам данных:

1. **Веб-чат** → передаёт сообщение пользователя в backend через HTTP-запрос (`/api/chat`). Backend обрабатывает его через ядро ассистента.
2. **Сценарии генерации** → используются модулем `dialog_generator.py` для формирования промптов и последующего вызова Yandex GPT.
3. **Google Sheets** → после экспорта CSV из `data/results` пользователь импортирует данные в таблицу, где с помощью формул проводится анализ.
4. **VK-бот** → при обнаружении телефона в диалоге отправляет уведомление менеджеру (асинхронно).

Дополнительные связи:

- **TTS Converter** читает JSON-диалоги из `data/generated/dialogs/`, синтезирует аудио и сохраняет в `data/generated/audio/`.
- **Async Transcriber** загружает аудиофайлы в Object Storage, запускает асинхронное распознавание и сохраняет транскрипты в `data/transcriptions/`.
- **Export to CSV** объединяет исходные диалоги и транскрипты, формируя файл для Google Sheets.

Рис. 9.3 Схема взаимодействия компонентов системы



9.4. Дальнейшее масштабирование и рекомендации по внедрению в реальную CRM

Разработанный прототип может быть адаптирован для работы в существующей CRM-системе (например, amoCRM, Битрикс24). Для этого предлагаются следующие шаги:

1. **Автоматическая передача лидов**

- Вместо сохранения диалогов в локальные JSON-файлы отправлять данные через webhook в CRM. **Автоматическое создание карточек клиентов в CRM** – передача собранных данных (имя, телефон, потребности) в системы типа amoCRM или Битрикс24.
- Использовать API CRM для создания сделок или контактов.
- Пример: при обнаружении телефона вызывать функцию, которая формирует POST-запрос к эндпоинту CRM.

2. **Синхронизация с задачами менеджеров**

- Добавить в уведомления VK-бота ссылку на карточку клиента в CRM.
- В случае отсутствия CRM – использовать Google Sheets как простую базу лидов, автоматически добавляя строку при каждом диалоге с телефоном.

3. **Масштабирование пайплайна транскрибации**

- Перевести паплайн первой части проекта на использование асинхронного API, чтобы обрабатывать реальные длинные звонки.
- Настроить автоматическую загрузку новых записей из телефонии в бакет Object Storage.

4. **Мониторинг и дашборды**

    - Построение дашбордов – на основе данных из Google Sheets можно создать интерактивные панели с графиками, обновляемыми в реальном времени — это возможно через Google Data Studio (Looker Studio) или Power BI,
    - Подключить Google Data Studio (или Power BI) к таблице результатов для визуализации динамики возражений и конверсии.
    - Настроить ежедневную рассылку отчётов менеджерам.

5. **Обучение и контроль операторов** – формирование базы успешных диалогов для обучения новых сотрудников и автоматическая оценка соблюдения скрипта по ключевым фразам.

6. **Безопасность и соответствие ФЗ-152**

- Все данные должны оставаться в российских облаках (Yandex Cloud).
- При интеграции с CRM использовать защищённые каналы (HTTPS), авторизацию по токенам.
- Для хранения персональных данных (телефоны, имена) предусмотреть шифрование в БД.

Предложенная архитектура позволяет легко добавлять новые каналы связи (Telegram, WhatsApp) путём создания аналогичных модулей-адаптеров, что делает систему гибкой и готовой к изменению требований бизнеса.

9.5. Рекомендации по промышленному внедрению с использованием российских аналогов

Разработанный прототип успешно демонстрирует принципиальную возможность автоматизации анализа диалогов на базе российских облачных технологий. Однако для промышленной эксплуатации необходимо заменить ряд компонентов, использованных в прототипе, на более масштабируемые и соответствующие требованиям ФЗ-152 решения. Ниже приведены рекомендации по такому переходу.

9.5.1. Замена Google Sheets на российские BI-инструменты

Используемое в прототипе решение:
Google Sheets применялся для первичного анализа данных благодаря удобству формул и быстрому прототипированию. Однако хранение данных за пределами РФ противоречит политике локализации.

Рекомендуемые российские аналоги:

1. **Yandex DataLens**
   Назначение: построение дашбордов, визуализация, аналитика.
   Преимущества:

- данные хранятся в Yandex Cloud (РФ);
- поддерживает подключение к различным источникам (PostgreSQL, ClickHouse, CSV);
- позволяет создавать интерактивные отчёты с фильтрами и диаграммами;
- не требует программирования для базовых отчётов.

  Интеграция: данные из `data/results/dialog_comparison.csv` можно загружать в бакет Object Storage, затем подключать DataLens к этому бакету, настроив автоматическое обновление.

2. **Яндекс.Таблицы**
   Назначение: полноценная замена Google Sheets с сохранением формул и привычного табличного интерфейса.
   Преимущества:

- российское облачное хранилище;
- поддерживаются большинство формул (включая REGEXEXTRACT, QUERY, FILTER);
- возможна автоматическая загрузка данных через API Яндекс.Диска.

  Интеграция: скрипт на Python может обновлять таблицу через API, заменяя старые данные новыми.

3. **Связка «База данных + Yandex DataLens»**
   Для больших объёмов данных (сотни тысяч диалогов) целесообразно использовать облачную базу данных (Managed PostgreSQL или Managed ClickHouse в Yandex Cloud) в сочетании с DataLens. Такой подход обеспечивает высокую производительность и масштабируемость.

9.5.2. Переход на асинхронную транскрибацию для реальных звонков

В первой части проекта использовался синхронный API, ограниченный 30 секундами. Для обработки реальных записей колл-центра, длительность которых может достигать нескольких минут, необходимо использовать асинхронный API (разработанный во второй части). Рекомендуется:

- перенести логику предобработки и транскрибации из первой части в общий модуль, работающий с асинхронным API;
- настроить автоматическую загрузку новых аудиофайлов из телефонии в Object Storage;
- организовать очередь задач для распознавания (например, через Yandex Message Queue).

9.5.3. Интеграция с CRM и автоматическая передача лидов

Для передачи собранных данных (телефон, имя, потребности) в систему управления взаимоотношениями с клиентами (CRM) рекомендуется:

- использовать вебхуки (webhooks) для отправки данных в amoCRM, Битрикс24 или другие системы;
- реализовать модуль интеграции, который при обнаружении телефона в диалоге формирует JSON-запрос к API CRM;
- предусмотреть повторные попытки отправки в случае сетевых ошибок.

9.5.4. Обеспечение безопасности и соответствия ФЗ-152

Все компоненты должны работать в контуре российских облачных сервисов (Yandex Cloud). Для усиления защиты:

- использовать API-ключи с ограниченными правами (только на необходимые операции);
- хранить чувствительные данные (номера телефонов) в зашифрованном виде в базе данных;
- настроить аудит доступа к логам и результатам транскрибации;
- применять защищённое соединение (TLS) для всех внешних взаимодействий.

9.5.5. Масштабирование и отказоустойчивость

При росте объёма обрабатываемых звонков и диалогов рекомендуется:

- перейти от локальной файловой системы к Object Storage для хранения исходных и обработанных аудио;
- использовать контейнеризацию (Docker) и оркестрацию (Yandex Managed Service for Kubernetes) для развёртывания сервисов;
- настроить мониторинг через Yandex Monitoring и алерты при сбоях.

9.5.6. Внедрение в реальный бизнес-процесс

Для успешного внедрения системы в работу колл-центра необходимо:

1. Провести пилотное тестирование на ограниченной выборке реальных звонков.
2. Обучить супервайзеров работе с дашбордами (DataLens) и интерпретации метрик.
3. Интегрировать отчёты в еженедельные планёрки отдела продаж.
4. Организовать обратную связь: менеджеры могут отмечать неточности распознавания, которые будут использоваться для дообучения моделей (если будет применяться кастомизация).

Предложенные меры позволят превратить разработанный прототип в промышленное решение, полностью соответствующее российскому законодательству и способное обрабатывать большие объёмы данных в реальном времени. Выбор конкретных инструментов (DataLens, Яндекс.Таблицы или связка с базой данных) зависит от требований к производительности, бюджета и существующей инфраструктуры предприятия.

ЗАКЛЮЧЕНИЕ ПО ВТОРОЙ И ТРЕТЬЕЙ ЧАСТЯМ

В ходе работы над второй и третьей частями дипломного проекта был создан полноценный ИИ-ассистент для сайта компании CULT CONSTRUCTIONS, реализованы генерация синтетических диалогов, их озвучивание и обратная транскрибация, а также автоматизированный анализ в Google Sheets.

Основные достижения:

- Разработано ядро ассистента с загрузкой базы знаний, сбором данных, управлением диалогом и очисткой маркдауна.
- Создан веб-интерфейс на FastAPI с сохранением истории диалогов.
- Сгенерировано 7 сценариев и 7 синтетических диалогов.
- Реализован синтез речи с разными голосами и асинхронная транскрибация, что позволило обойти ограничение в 30 секунд.
- Проведён анализ в Google Sheets, подтвердивший применимость подхода для выявления возражений и оценки успешности диалогов.
- Описаны перспективы интеграции с VK и Telegram, а также возможные направления масштабирования.

Все разработанные модули соответствуют требованию использования российских технологий и соблюдения ФЗ-152. Код снабжён подробными комментариями и может быть использован как основа для внедрения в реальную коммерческую среду.

1. ТАБЛИЦА РЕЗУЛЬТАТОВ АНАЛИЗА СИНТЕТИЧЕСКИХ ДИАЛОГОВ

Таблица 1. Сравнение исходных и распознанных текстов

Сценарий	Длина исходного текста (симв.)	Длина транскрипта (симв.)	Точность (D/C)	Найдено возражение	Успешность (телефон/замер)
Возражение: Дорого	1875	1715	0,915	дорого	да
Возражение: Подумаю	1564	1428	0,913	подумать	да
Возражение: Не сейчас	1706	1541	0,904	подумать	да
Алюминий vs сталь	1858	1736	0,934	сталь	да
Успешная консультация	1603	1466	0,915	оттенки	да
Сложный индивидуальный проект	1652	1510	0,914	стоимость	да
География: другой город	1279	1186	0,927	дорого	нет

Примечание: Точность вычислена как отношение длины транскрипта к длине исходного текста. Успешность определялась по наличию в транскрипте слов «телефон», «замер», «свяжитесь».

Средняя точность: 0,917 (≈91,7 %)
Диапазон точности: 0,904 – 0,934
Успешных диалогов: 6 из 7 (85,7 %)

2. ОБНОВЛЁННЫЙ СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ (дополнение к первой части)

8.**Официальная документация Yandex GPT.** – URL: https://cloud.yandex.ru/docs/foundation-models/ (дата обращения: 03.2026).

- Руководство по работе с Yandex GPT через API.

9.**FastAPI Documentation.** – URL: https://fastapi.tiangolo.com/ (дата обращения: 03.2026).

- Документация по веб-фреймворку FastAPI.

10.**python-telegram-bot Documentation.** – URL: https://docs.python-telegram-bot.org/ (дата обращения: 03.2026).

- Библиотека для создания Telegram-ботов.

11.**VK API Documentation.** – URL: https://dev.vk.com/ru/api/ (дата обращения: 03.2026).

- Документация по API VK (для интеграции с мессенджером).

12.**Google Sheets API Documentation.** – URL: https://developers.google.com/sheets/api (дата обращения: 03.2026).

- Описание API для работы с таблицами.

13.**VKBottle Framework.** – URL: https://vkbottle.rtfd.io/ (дата обращения: 03.2026).

- Асинхронный фреймворк для создания VK-ботов.

3. ПРИМЕЧАНИЕ О ЛИСТИНГАХ КОДА

Финальные версии всех модулей (см. Приложения В–Ж) соответствуют коду, размещённому в репозитории проекта. В целях компактности в тексте приведены основные фрагменты с комментариями. Полные тексты файлов,
включая core.py, dialog_generator.py, tts_converter.py, async_transcriber.py и вспомогательные скрипты, находятся в папке part2/src прилагаемого архива. Актуальные версии можно получить из PyCharm-проекта.

ПРИЛОЖЕНИЯ (КО ВТОРОЙ ЧАСТИ)

Приложение В. Листинги модулей ядра ассистента

- knowledge_loader.py
- core.py
- dialog_storage.py

Приложение Г. Листинг сценариев диалогов (dialog_scenarios.json)

Приложение Д. Листинг модуля генерации диалогов (dialog_generator.py)

Приложение Е. Листинг модулей синтеза речи и асинхронной транскрибации

- tts_converter.py
- async_transcriber.py

Приложение Ж. Листинг скрипта экспорта в CSV (export_to_csv.py)

Приложение З. Скриншоты веб-чата и таблиц Google Sheets

(вставлены в текст глав 6 и 8)

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ В. Листинги модулей ядра ассистента

Листинг В.1. knowledge_loader.py – загрузка базы знаний (код из PyCharm)

python

```python
"""
knowledge_loader.py - Загрузка и обработка базы знаний для ИИ-ассистента
Модуль читает файлы:
- knowledge_base.2.md (общая информация о компании)
- faq.json (структурированные вопросы-ответы)
И предоставляет методы для получения контекста в промпты
"""
import json
import os
from pathlib import Path
from typing import Dict, List, Optional, Tuple
class KnowledgeBaseLoader:
    """
    Загрузчик базы знаний для ИИ-ассистента компании CULT CONSTRUCTIONS
    """
    def __init__(self, knowledge_base_path: Optional[Path] = None):
        """
        Инициализация загрузчика
        Args:
            knowledge_base_path: путь к папке с файлами базы знаний
        """
        # Определяем путь к папке knowledge_base
        if knowledge_base_path is None:
            # По умолчанию: part2/data/knowledge_base/
            self.base_path = Path(__file__).parent.parent.parent / "data" / "knowledge_base"
        else:
            self.base_path = Path(knowledge_base_path)
        # Убеждаемся, что папка существует
        if not self.base_path.exists():
            raise FileNotFoundError(f"Папка базы знаний не найдена: {self.base_path}")
        # Инициализируем структуры для хранения данных
        self.faq_data: List[Dict] = []  # данные из faq.json
        self.company_info: str = ""  # текст из knowledge_base.2.md
        self.faq_dict: Dict[str, str] = {}  # быстрый поиск по вопросам
        # Загружаем данные
        self._load_faq()
        self._load_company_info()
        print(f"□ KnowledgeBaseLoaderинициализирован")
        print(f"   ? Путь: {self.base_path}")
        print(f"   ? FAQ загружено: {len(self.faq_data)} вопросов")
        print(f"   ? Company info: {len(self.company_info)} символов")
    def _load_faq(self) -> None:
        """
        Загружает и парсит файл faq.json
        """
        faq_file = self.base_path / "faq.json"
        if not faq_file.exists():
            print(f"□□  Файл faq.json не найден: {faq_file}")
            return
```

```
        try:
            with open(faq_file, 'r', encoding='utf-8') as f:
                self.faq_data = json.load(f)
            # Создаем словарь для быстрого поиска по вопросам
            for item in self.faq_data:
                # Основной вопрос
                question = item.get('question', '')
                answer = item.get('answer', '')
                if question and answer:
                    self.faq_dict[question.lower()] = answer
                # Альтернативные вопросы
                alt_questions = item.get('alternate_questions', [])
                for alt_q in alt_questions:
                    if alt_q and answer:
                        self.faq_dict[alt_q.lower()] = answer
        except json.JSONDecodeError as e:
            print(f"□ Ошибка парсинга faq.json: {e}")
        except Exception as e:
            print(f"□ Непредвиденная ошибка при загрузке FAQ: {e}")
    def _load_company_info(self) -> None:
        """
        Загружает файл knowledge_base.2.md (информация о компании)
        """
        # Проверяем возможные имена файла
        possible_names = [
            "knowledge_base.2.md",
            "knowledge_base.md",
            "company_info.md"
        ]
        md_file = None
        for name in possible_names:
            candidate = self.base_path / name
            if candidate.exists():
                md_file = candidate
                break
        if md_file is None:
            print(f"□□  Файл с информацией о компании не найден")
            print(f"   Искали: {[str(self.base_path / n) for n in possible_names]}")
            return
        try:
            with open(md_file, 'r', encoding='utf-8') as f:
                self.company_info = f.read()
        except Exception as e:
            print(f"□ Ошибка загрузки файла компании: {e}")
    def get_system_prompt(self) -> str:
        """
        Формирует системный промпт для ассистента на основе базы знаний
        Returns:
            str: текст системного промпта
        """
        prompt_parts = []
        # Базовая роль
        prompt_parts.append("Ты — ИИ-ассистент компании CULT CONSTRUCTIONS.")
        prompt_parts.append(
            "Компания специализируется на производстве и установке премиальных алюминиевых
ограждений (заборов).")
        prompt_parts.append("")
        # Информация о компании (если загружена)
        if self.company_info:
            prompt_parts.append("ИНФОРМАЦИЯ О КОМПАНИИ:")
            prompt_parts.append(self.company_info)
            prompt_parts.append("")
        # Инструкции по поведению
```

```
        prompt_parts.append("ПРАВИЛА РАБОТЫ:")
        prompt_parts.append("1. Отвечай вежливо, профессионально, на русском языке.")
        prompt_parts.append("2. Если вопрос есть в FAQ — используй точный ответ оттуда.")
        prompt_parts.append(
            "3. Если вопрос сложный или требует индивидуального расчета — предложи оставить
контакты для связи со специалистом.")
        prompt_parts.append("4. Собирай информацию: длина участка, бюджет, тип объекта.")
        prompt_parts.append(
            "5. Не выдумывай цены — говори только ориентиры (от 16 000 руб/м² материалы, от 5 000
руб/м² монтаж).")
        prompt_parts.append("6. Если клиент готов оставить контакты — сохрани их и передай менеджеру.")
        prompt_parts.append("")
        # FAQ для быстрого доступа (только ключевые)
        prompt_parts.append("ЧАСТЫЕ ВОПРОСЫ И ОТВЕТЫ:")
        for item in self.faq_data[:5]:  # берем первые 5 для контекста
            q = item.get('question', '')
            a = item.get('answer', '')
            if q and a:
                # Сокращаем ответ для системного промпта
                short_answer = a[:150] + "..." if len(a) > 150 else a
                prompt_parts.append(f"В: {q}")
                prompt_parts.append(f"О: {short_answer}")
                prompt_parts.append("")
        return "\n".join(prompt_parts)
    def get_faq_answer(self, question: str) -> Optional[str]:
        """
        Быстрый поиск ответа на вопрос в FAQ
        Args:
            question: вопрос пользователя
        Returns:
            str: ответ из базы или None, если не найдено
        """
        question_lower = question.lower().strip()
        # Прямой поиск
        if question_lower in self.faq_dict:
            return self.faq_dict[question_lower]
        # Поиск по вхождению ключевых слов (упрощенный)
        for q, a in self.faq_dict.items():
            # Если вопрос пользователя содержит ключевые слова из FAQ
            if any(word in question_lower for word in q.split() if len(word) > 3):
                return a
        return None
    def get_all_faq(self) -> List[Dict]:
        """
        Возвращает все данные FAQ
        Returns:
            List[Dict]: список всех вопросов-ответов
        """
        return self.faq_data
    def get_company_info(self) -> str:
        """
        Возвращает информацию о компании
        Returns:
            str: текст из файла knowledge_base
        """
        return self.company_info
    def get_context_for_prompt(self, user_question: str) -> Tuple[str, bool]:
        """
        Получает контекст для промпта на основе вопроса пользователя
        Args:
            user_question: вопрос пользователя
        Returns:
            Tuple[str, bool]: (контекст, найден_ли_точный_ответ_в_faq)
```

```
        """
        # Сначала ищем точный ответ в FAQ
        exact_answer = self.get_faq_answer(user_question)
        if exact_answer:
            return f"На вопрос '{user_question}' в нашей базе есть ответ:\n{exact_answer}", True
        # Если точного ответа нет, возвращаем общий контекст
        context = f"""
Контекст разговора:
Пользователь спрашивает: {user_question}
Информация о компании:
{self.company_info[:500]}...  # первые 500 символов для контекста
"""
        return context, False
# Для тестирования модуля
if __name__ == "__main__":
    print("? Тестирование KnowledgeBaseLoader")
    print("=" * 60)
    try:
        loader = KnowledgeBaseLoader()
        print("\n? Системный промпт (первые 500 символов):")
        print("-" * 40)
        prompt = loader.get_system_prompt()
        print(prompt[:500])
        print("...")
        print("\n? Тест поиска в FAQ:")
        test_questions = [
            "сколько стоит забор",
            "гарантия на забор",
            "чем алюминий лучше стали",
            "как заказать замер"
        ]
        for q in test_questions:
            answer = loader.get_faq_answer(q)
            if answer:
                print(f"□ '{q}' → найден ответ ({len(answer)} символов)")
            else:
                print(f"□ '{q}' → ответ не найден")
    except Exception as e:
        print(f"□ Ошибка: {e}")
    print("\n" + "=" * 60)
```

Листинг В.2. core.py – ядро ассистента (сокращённый фрагмент, полный код см. в приложении к диплому)

python

```
"""
core.py - Ядро ИИ-ассистента для компании CULT CONSTRUCTIONS
"""
import json
import sys
import re
from pathlib import Path
from typing import Dict, List, Optional, Any
from datetime import datetime
import requests
import uuid
# Исправляем пути
```

```
PART2_ROOT = Path(__file__).parent.parent.parent
SRC_PATH = PART2_ROOT / "src"
if str(SRC_PATH) not in sys.path:
    sys.path.insert(0, str(SRC_PATH))
# Импорты
try:
    from utils.config import config
    from assistant.knowledge_loader import KnowledgeBaseLoader
    from assistant.dialog_storage import DialogStorage
except ImportError as e:
    print(f"□ Ошибка импорта: {e}")
    print(f"   Путь: {SRC_PATH}")
    sys.exit(1)
class CultConstructionsAssistant:
    """
    Основной класс ИИ-ассистента для компании CULT CONSTRUCTIONS
    """
    def __init__(self, knowledge_base_path: Optional[Path] = None):
        """
        Инициализация ассистента
        """
        # Загружаем конфигурацию
        self.api_key = config.get('yandex_api_key')
        self.folder_id = config.get('yandex_folder_id')
        self.model_uri = "gpt://b1gtp8bn97gg21fl5e9j/yandexgpt-5.1/latest"
        self.api_url = "https://llm.api.cloud.yandex.net/foundationModels/v1/completion"
        # Загружаем базу знаний
        print("? Загрузка базы знаний...")
        self.knowledge_base = KnowledgeBaseLoader(knowledge_base_path)
        # История диалога
        self.dialog_history: List[Dict[str, str]] = []
        self.max_history_length = 10
        # Данные, собранные от клиента
        self.collected_data = {
            "name": None,
            "phone": None,
            "email": None,
            "object_length": None,
            "budget": None,
            "object_type": None,
            "needs_human": False,
        }
        # Фразы-триггеры для передачи человеку
        self.human_triggers = [
            "свяжитесь со мной",
            "позвоните мне",
            "хочу поговорить со специалистом",
            "сложный вопрос",
            "индивидуальный расчет",
            "точная смета",
            "выезд специалиста",
        ]
        print(f"□  Ассистент инициализирован")
        # Добавляем хранилище диалогов
        self.storage = DialogStorage()
        self.session_id = str(uuid.uuid4())[:8]  # уникальный ID сессии
        self.dialog_start_time = datetime.now()
    def _clean_markdown(self, text: str) -> str:
        """
        Удаляет маркдаун-разметку из текста для отображения в простом чате
        Args:
            text: исходный текст с возможной разметкой
        Returns:
```

```
            str: очищенный текст
        """
        if not text:
            return text
        # Удаляем **жирный текст**
        text = text.replace('**', '')
        # Удаляем *курсив*
        text = text.replace('*', '')
        # Удаляем `код`
        text = text.replace('`', '')
        # Удаляем # заголовки
        text = re.sub(r'^#+\s', '', text, flags=re.MULTILINE)
        # Удаляем маркдаун-списки (- или * в начале строки) и заменяем на bullet points
        text = re.sub(r'^\s*[-*]\s', '• ', text, flags=re.MULTILINE)
        # Удаляем маркдаун-ссылки [текст](ссылка) -> текст
        text = re.sub(r'\[([^\]]+)\]\([^\)]+\)', r'\1', text)
        # Удаляем множественные пробелы
        text = re.sub(r'\s+', ' ', text)
        # Удаляем пустые строки с точками
        text = re.sub(r'^\s*•\s*$', '', text, flags=re.MULTILINE)
        return text.strip()
    def _add_to_history(self, role: str, text: str) -> None:
        """Добавляет сообщение в историю диалога"""
        self.dialog_history.append({
            "role": role,
            "text": text,
            "time": datetime.now().isoformat()
        })
        if len(self.dialog_history) > self.max_history_length:
            self.dialog_history = self.dialog_history[-self.max_history_length:]
    def _extract_info_from_message(self, message: str) -> None:
        """Извлекает полезную информацию из сообщения"""
        message_lower = message.lower()
        # Упрощенный поиск телефона: ищем последовательности из 10-11 цифр
        phone_pattern = r'(\+7|8)?[\s\-]?\(?(\d{3})\)?[\s\-]?(\d{3})[\s\-]?(\d{2})[\s\-]?(\d{2})'
        phone_match = re.search(phone_pattern, message)
        if phone_match and not self.collected_data["phone"]:
            self.collected_data["phone"] = phone_match.group()
        # Поиск длины
        length_pattern = r'(\d+)[\s]*(м|метр|метра|метров)'
        length_match = re.search(length_pattern, message_lower)
        if length_match and not self.collected_data["object_length"]:
            self.collected_data["object_length"] = length_match.group(1)
        # Поиск бюджета
        budget_pattern = r'(\d+)[\s]*(тыс|тысяч|млн|миллион)'
        budget_match = re.search(budget_pattern, message_lower)
        if budget_match and not self.collected_data["budget"]:
            self.collected_data["budget"] = budget_match.group()
        # Поиск имени (если есть фраза "меня зовут")
        name_pattern = r'(?:меня зовут|зовут|имя)\s+([А-ЯЁ][а-яё]+)'
        name_match = re.search(name_pattern, message_lower)
        if name_match and not self.collected_data["name"]:
            self.collected_data["name"] = name_match.group(1).capitalize()
        # Проверка на передачу человеку
        for trigger in self.human_triggers:
            if trigger in message_lower:
                self.collected_data["needs_human"] = True
                break
    def _check_if_ready_for_human(self) -> bool:
        """Проверяет, можно ли передать диалог человеку"""
        return bool(self.collected_data["phone"])
    def _format_history_for_prompt(self) -> str:
        """Форматирует историю для промпта"""
```

```
        if not self.dialog_history:
            return "История диалога пуста."
        formatted = "ИСТОРИЯ ДИАЛОГА:\n"
        for msg in self.dialog_history[-5:]:
            role = "Клиент" if msg["role"] == "user" else "Ассистент"
            formatted += f"{role}: {msg['text']}\n"
        return formatted
    def _build_prompt(self, user_message: str) -> List[Dict[str, str]]:
        """Формирует промпт для Yandex GPT"""
        system_prompt = self.knowledge_base.get_system_prompt()
        # Добавляем информацию о собранных данных
        if any(self.collected_data.values()):
            system_prompt += "\n\nСОБРАННАЯ ИНФОРМАЦИЯ О КЛИЕНТЕ:\n"
            if self.collected_data["name"]:
                system_prompt += f"- Имя: {self.collected_data['name']}\n"
            if self.collected_data["phone"]:
                system_prompt += f"- Телефон: {self.collected_data['phone']}\n"
            if self.collected_data["object_length"]:
                system_prompt += f"- Длина участка: {self.collected_data['object_length']} м\n"
            if self.collected_data["budget"]:
                system_prompt += f"- Бюджет: {self.collected_data['budget']}\n"
        # Добавляем историю диалога
        system_prompt += "\n" + self._format_history_for_prompt()
        # Формируем сообщения
        messages = [{"role": "system", "text": system_prompt}]
        messages.append({"role": "user", "text": user_message})
        return messages
    def process_message(self, user_message: str) -> Dict[str, Any]:
        """Обрабатывает сообщение пользователя"""
        self._add_to_history("user", user_message)
        self._extract_info_from_message(user_message)
        # БЛОК 1: Если есть телефон - передаем человеку
        if self._check_if_ready_for_human():
            response = self._generate_human_transfer_message()
            self._add_to_history("assistant", response)
            # Сохраняем диалог
            self.storage.save_dialog_if_completed(
                self.dialog_history,
                self.collected_data,
                self.session_id
            )
            return {
                "response": response,
                "needs_human": True,  # ← ЗДЕСЬ TRUE (передаем человеку)
                "collected_data": self.collected_data,
                "from_faq": False
            }
        # БЛОК 2: Обычный ответ (нет телефона)
        messages = self._build_prompt(user_message)
        assistant_response = self._call_yandex_gpt(messages)
        self._add_to_history("assistant", assistant_response)
        # Очищаем ответ от маркдауна
        cleaned_response = self._clean_markdown(assistant_response)
        # Сохраняем длинные диалоги
        if len(self.dialog_history) >= 8:
            self.storage.save_dialog_if_completed(
                self.dialog_history,
                self.collected_data,
                self.session_id
            )
        return {
            "response": cleaned_response,
            "needs_human": False,  # ← ЗДЕСЬ FALSE (обычный ответ)
```

```
                "collected_data": self.collected_data,
                "from_faq": False
            }
    def _generate_human_transfer_message(self) -> str:
        """Генерирует сообщение о передаче человеку"""
        name = self.collected_data.get("name", "")
        phone = self.collected_data.get("phone", "")
        if name and phone:
            message = (f"Спасибо, {name}! Я передал ваши контакты ({phone}) нашему специалисту. "
                       f"Он свяжется с вами в ближайшее время.")
        elif phone:
            message = (f"Спасибо! Я передал ваш телефон ({phone}) нашему специалисту. "
                       f"Он свяжется с вами в ближайшее время.")
        else:
            message = ("Спасибо за интерес! Для точного расчета оставьте ваш телефон, "
                       "и мы свяжемся с вами.")
        return self._clean_markdown(message)
    def _call_yandex_gpt(self, messages: List[Dict[str, str]]) -> str:
        """Отправляет запрос к Yandex GPT"""
        headers = {
            "Authorization": f"Api-Key {self.api_key}",
            "Content-Type": "application/json"
        }
        payload = {
            "modelUri": self.model_uri,
            "completionOptions": {
                "stream": False,
                "temperature": 0.3,
                "maxTokens": "500"
            },
            "messages": messages
        }
        try:
            response = requests.post(
                self.api_url,
                headers=headers,
                json=payload,
                timeout=30
            )
            if response.status_code == 200:
                result = response.json()
                alternatives = result.get("result", {}).get("alternatives", [])
                if alternatives:
                    return alternatives[0].get("message", {}).get("text", "")
                return "Извините, не удалось получить ответ."
            else:
                return f"Извините, временная ошибка сервиса."
        except Exception:
            return "Извините, ошибка соединения."
    def get_collected_data(self) -> Dict:
        """Возвращает собранные данные"""
        return self.collected_data
    def reset_dialog(self) -> None:
        """Сбрасывает диалог"""
        self.dialog_history = []
        self.collected_data = {k: None for k in self.collected_data}
        self.collected_data["needs_human"] = False
# Тестирование
if __name__ == "__main__":
    print("? Тестирование CultConstructionsAssistant")
    print("=" * 60)
    assistant = CultConstructionsAssistant()
    test_messages = [
```

```
            "Здравствуйте! Сколько стоит алюминиевый забор?",
            "У меня участок 30 метров",
            "А какой срок службы?",
            "Хочу оставить заявку, мой телефон +7 999 123-45-67"
        ]
        print("\n? Тестовый диалог:")
        print("-" * 40)
        for msg in test_messages:
            print(f"\n? Клиент: {msg}")
            result = assistant.process_message(msg)
            print(f"? Ассистент: {result['response'][:100]}...")
            if result['needs_human']:
                print(f"? → Передача человеку!")
                print(f"? Данные: {result['collected_data']}")
                break
        print("\n" + "=" * 60)
        print("\n? Загрузка базы знаний...")
        print("□ Ассистент инициализирован")
        print("□ Класс создан успешно")
```

Листинг B.3. dialog_storage.py – сохранение диалогов

python

```
"""
dialog_storage.py - Сохранение истории диалогов в JSON-файлы
Сохраняет:
- Полную историю сообщений
- Собранные данные о клиенте
- Временные метки
- Результат диалога (передан человеку или нет)
"""
import json
import uuid
from pathlib import Path
from datetime import datetime
from typing import Dict, List, Optional, Any
class DialogStorage:
    """
    Класс для сохранения истории диалогов в JSON-файлы
    """
    def __init__(self, storage_dir: Optional[Path] = None):
        """
        Инициализация хранилища диалогов
        Args:
            storage_dir: путь к папке для сохранения файлов
        """
        # Определяем папку для хранения
        if storage_dir is None:
            # По умолчанию: part2/data/dialogs/
            self.storage_dir = Path(__file__).parent.parent.parent / "data" / "dialogs"
        else:
            self.storage_dir = Path(storage_dir)
        # Создаем папку, если её нет
        self.storage_dir.mkdir(parents=True, exist_ok=True)
        print(f"? Хранилище диалогов: {self.storage_dir}")
        print(f"□ DialogStorageинициализирован")
    def _generate_filename(self, session_id: str = None) -> Path:
        """
```

```
        Генерирует имя файла для сохранения диалога
        Args:
            session_id: идентификатор сессии (если есть)
        Returns:
            Path: полный путь к файлу
        """
        # Формат: YYYY-MM-DD_HH-MM-SS_[session_id].json
        timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
        if session_id:
            filename = f"{timestamp}_{session_id}.json"
        else:
            # Если нет session_id, генерируем уникальный ID
            unique_id = str(uuid.uuid4())[:8]
            filename = f"{timestamp}_{unique_id}.json"
        return self.storage_dir / filename
    def save_dialog(self,
                    dialog_history: List[Dict[str, Any]],
                    collected_data: Dict[str, Any],
                    session_id: str = None,
                    metadata: Optional[Dict[str, Any]] = None) -> Path:
        """
        Сохраняет диалог в JSON-файл
        Args:
            dialog_history: история сообщений
            collected_data: собранные данные о клиенте
            session_id: идентификатор сессии
            metadata: дополнительная метаинформация
        Returns:
            Path: путь к сохраненному файлу
        """
        # Формируем структуру для сохранения
        dialog_data = {
            "session_id": session_id or str(uuid.uuid4()),
            "timestamp": datetime.now().isoformat(),
            "date": datetime.now().strftime("%Y-%m-%d"),
            "time": datetime.now().strftime("%H:%M:%S"),
            "message_count": len(dialog_history),
            "dialog_history": dialog_history,
            "collected_data": collected_data,
            "metadata": metadata or {}
        }
        # Добавляем флаг "передан человеку" если есть телефон
        if collected_data.get("phone"):
            dialog_data["transferred_to_human"] = True
            dialog_data["transfer_reason"] = "phone_collected"
        else:
            dialog_data["transferred_to_human"] = False
        # Генерируем имя файла
        file_path = self._generate_filename(session_id)
        # Сохраняем в файл
        try:
            with open(file_path, 'w', encoding='utf-8') as f:
                json.dump(dialog_data, f, ensure_ascii=False, indent=2)
            print(f"? Диалог сохранен: {file_path.name}")
            return file_path
        except Exception as e:
            print(f"□ Ошибка сохранения диалога: {e}")
            raise
    def save_dialog_if_completed(self,
                                 dialog_history: List[Dict[str, Any]],
                                 collected_data: Dict[str, Any],
                                 session_id: str = None) -> Optional[Path]:
        """
```

```
        Сохраняет диалог только если он завершен (есть телефон или много сообщений)
        Args:
            dialog_history: история сообщений
            collected_data: собранные данные
            session_id: идентификатор сессии
        Returns:
            Optional[Path]: путь к файлу или None, если диалог не сохранен
        """
        # Сохраняем если:
        # 1. Есть телефон (клиент оставил контакты)
        # 2. Или диалог длинный (больше 6 сообщений)
        # 3. Или прошло много времени (но это сложнее, пока пропускаем)
        should_save = False
        reason = []
        if collected_data.get("phone"):
            should_save = True
            reason.append("phone_collected")
        if len(dialog_history) >= 8:  # 4+ сообщений от пользователя
            should_save = True
            reason.append("long_dialog")
        if should_save:
            metadata = {"save_reason": reason}
            return self.save_dialog(dialog_history, collected_data, session_id, metadata)
        return None
    def get_all_dialogs(self) -> List[Dict[str, Any]]:
        """
        Загружает все сохраненные диалоги
        Returns:
            List[Dict]: список всех диалогов
        """
        dialogs = []
        for file_path in self.storage_dir.glob("*.json"):
            try:
                with open(file_path, 'r', encoding='utf-8') as f:
                    dialog = json.load(f)
                    dialogs.append(dialog)
            except Exception as e:
                print(f"□ Ошибка загрузки {file_path.name}: {e}")
        # Сортируем по времени (новые сверху)
        dialogs.sort(key=lambda x: x.get("timestamp", ""), reverse=True)
        return dialogs
    def get_dialogs_by_date(self, date: str) -> List[Dict[str, Any]]:
        """
        Возвращает диалоги за указанную дату
        Args:
            date: дата в формате YYYY-MM-DD
        Returns:
            List[Dict]: список диалогов за дату
        """
        all_dialogs = self.get_all_dialogs()
        return [d for d in all_dialogs if d.get("date") == date]
    def get_dialogs_with_phone(self) -> List[Dict[str, Any]]:
        """
        Возвращает только диалоги, где есть телефон (лиды)
        Returns:
            List[Dict]: список диалогов-лидов
        """
        all_dialogs = self.get_all_dialogs()
        return [d for d in all_dialogs if d.get("collected_data", {}).get("phone")]
# Тестирование модуля
if __name__ == "__main__":
    print("? Тестирование DialogStorage")
    print("=" * 60)
```

```
    # Создаем хранилище
    storage = DialogStorage()
    # Тестовые данные
    test_history = [
        {"role": "user", "text": "Здравствуйте! Сколько стоит забор?", "time": "2026-03-20T10:00:00"},
        {"role": "assistant", "text": "Здравствуйте! Для расчета нужна длина участка.", "time": "2026-03-
20T10:00:05"},
        {"role": "user", "text": "У меня участок 30 метров", "time": "2026-03-20T10:00:15"},
        {"role": "assistant", "text": "Спасибо! Ориентировочная стоимость от 480 000 руб.",
         "time": "2026-03-20T10:00:20"},
        {"role": "user", "text": "Хочу оставить заявку, мой телефон +7 999 123-45-67", "time": "2026-03-
20T10:00:30"},
    ]
    test_data = {
        "name": None,
        "phone": "+7 999 123-45-67",
        "email": None,
        "object_length": "30",
        "budget": None,
        "object_type": None,
        "needs_human": True
    }
    # Сохраняем тестовый диалог
    print("\n? Сохраняем тестовый диалог...")
    file_path = storage.save_dialog(test_history, test_data, session_id="test_session")
    print(f"□ Сохранен в: {file_path}")
    # Проверяем сохранение только завершенных
    print("\n? Проверка save_dialog_if_completed...")
    # Диалог без телефона (не должен сохраниться)
    short_dialog = [test_history[0], test_history[1]]
    no_phone_data = test_data.copy()
    no_phone_data["phone"] = None
    result = storage.save_dialog_if_completed(short_dialog, no_phone_data)
    if result is None:
        print("□  Короткий диалог без телефона несохранен (правильно)")
    # Диалог с телефоном (должен сохраниться)
    result = storage.save_dialog_if_completed(test_history, test_data)
    if result:
        print(f"□ Диалог с телефоном сохранен: {result.name}")
    # Получаем все диалоги
    print("\n? Все сохраненные диалоги:")
    all_dialogs = storage.get_all_dialogs()
    for d in all_dialogs[:3]:  # покажем первые 3
        print(
            f"   - {d.get('date')} | {d.get('message_count')} сообщений | Телефон: {d.get('collected_data',
{}).get('phone')}")
    print("\n" + "=" * 60)
```

ПРИЛОЖЕНИЕ Г. Листинг модуля генерации диалогов

Листинг Г.1. dialog_generator.py

python

```
"""
dialog_generator.py - Генерация синтетических диалогов через Yandex GPT
Создает диалоги по заданным сценариям для последующего анализа
```

```
"""
import sys
import json
import uuid
from pathlib import Path
from typing import Dict, List, Optional, Any
from datetime import datetime
import requests
# Исправляем пути
PART2_ROOT = Path(__file__).parent.parent.parent
SRC_PATH = PART2_ROOT / "src"
if str(SRC_PATH) not in sys.path:
    sys.path.insert(0, str(SRC_PATH))
# Импорты
try:
    from utils.config import config
    print("□ configимпортирован")
except ImportError as e:
    print(f"□ Ошибка импорта config: {e}")
    print(f"   Путь поиска: {SRC_PATH}")
    sys.exit(1)
class DialogGenerator:
    """
    Генератор синтетических диалогов для обучения и анализа
    """
    def __init__(self, scenarios_path: Optional[Path] = None):
        """
        Инициализация генератора
        Args:
            scenarios_path: путь к файлу со сценариями
        """
        # Загружаем конфигурацию
        self.api_key = config.get('yandex_api_key')
        self.folder_id = config.get('yandex_folder_id')
        self.model_uri = "gpt://b1gtp8bn97gg21fl5e9j/yandexgpt-5.1/latest"
        self.api_url = "https://llm.api.cloud.yandex.net/foundationModels/v1/completion"
        # Определяем путь к сценариям
        if scenarios_path is None:
            self.scenarios_path = PART2_ROOT / "data" / "scenarios" / "dialog_scenarios.json"
        else:
            self.scenarios_path = Path(scenarios_path)
        # Загружаем сценарии
        self.scenarios = self._load_scenarios()
        # Папка для сохранения результатов
        self.output_dir = PART2_ROOT / "data" / "generated" / "dialogs"
        self.output_dir.mkdir(parents=True, exist_ok=True)
        print(f"□ DialogGeneratorинициализирован")
        print(f"   ? Сценарии: {self.scenarios_path}")
        print(f"   ? Загружено сценариев: {len(self.scenarios)}")
        print(f"   ? Сохранение: {self.output_dir}")
    def _load_scenarios(self) -> List[Dict[str, Any]]:
        """Загружает сценарии из JSON-файла"""
        if not self.scenarios_path.exists():
            print(f"□□ Файл сценариев не найден: {self.scenarios_path}")
            return []
        try:
            with open(self.scenarios_path, 'r', encoding='utf-8') as f:
                data = json.load(f)
                return data.get("scenarios", [])
        except Exception as e:
            print(f"□ Ошибка загрузки сценариев: {e}")
            return []
```

```
    def _build_generation_prompt(self, scenario: Dict[str, Any]) -> str:
        """
        Формирует промпт для генерации диалога
        Args:
            scenario: сценарий диалога
        Returns:
            str: промпт для Yandex GPT
        """
        prompt = f"""
Сгенерируй диалог между КЛИЕНТОМ и ОПЕРАТОРОМ компании CULT CONSTRUCTIONS (премиальные алюминиевые ограждения).
СЦЕНАРИЙ: {scenario.get('name', '')}
ОПИСАНИЕ: {scenario.get('description', '')}
ОСОБЕННОСТИ:
- Клиент: {scenario.get('client_objection', '')}
- Стратегия оператора: {scenario.get('operator_strategy', '')}
- Ожидаемый результат: {scenario.get('expected_outcome', '')}
ПРИМЕРЫ ФРАЗ КЛИЕНТА (используй как ориентир, но не копируй дословно):
{chr(10).join(['- ' + p for p in scenario.get('client_phrases', [])])}
ТРЕБОВАНИЯ К ДИАЛОГУ:
1. Диалог должен состоять из 6-12 реплик (переключений между участниками)
2. Используй формат: "Клиент: [текст]" и "Оператор: [текст]"
3. Оператор должен работать по скрипту компании (премиальный сегмент, индивидуальный подход)
4. Диалог должен быть реалистичным и естественным
5. В конце диалога клиент должен оставить контакт (телефон) или согласиться на замер
ВАЖНО:
- Оператор не называет точную стоимость (только ориентиры)
- Оператор подчеркивает преимущества: долговечность (100+ лет), отсутствие коррозии, экологичность
- Если клиент дает возражение — оператор его отрабатывает
Сгенерируй диалог в указанном формате.
"""
        return prompt
    def _call_yandex_gpt(self, prompt: str) -> str:
        """
        Отправляет запрос к Yandex GPT и возвращает сгенерированный диалог
        Args:
            prompt: промпт для генерации
        Returns:
            str: сгенерированный диалог
        """
        headers = {
            "Authorization": f"Api-Key {self.api_key}",
            "Content-Type": "application/json"
        }
        payload = {
            "modelUri": self.model_uri,
            "completionOptions": {
                "stream": False,
                "temperature": 0.7,  # Выше температура = больше креативности
                "maxTokens": "1000"
            },
            "messages": [
                {
                    "role": "system",
                    "text": "Ты — генератор учебных диалогов для отдела продаж. Создавай реалистичные диалоги между клиентом и оператором. Используй только формат 'Клиент: ...' и 'Оператор: ...'."
                },
                {
                    "role": "user",
                    "text": prompt
                }
            ]
        }
```

```
        try:
            response = requests.post(
                self.api_url,
                headers=headers,
                json=payload,
                timeout=60
            )
            if response.status_code == 200:
                result = response.json()
                alternatives = result.get("result", {}).get("alternatives", [])
                if alternatives:
                    return alternatives[0].get("message", {}).get("text", "")
                return ""
            else:
                print(f"□ Ошибка API: {response.status_code}")
                return ""
        except Exception as e:
            print(f"□  Ошибка запроса:{e}")
            return ""
    def _parse_dialog(self, text: str) -> List[Dict[str, str]]:
        """
        Парсит сгенерированный текст в структурированный диалог
        Args:
            text: текст с репликами в формате "Клиент: ..." и "Оператор: ..."
        Returns:
            List[Dict]: список реплик с ролями и текстом
        """
        dialog = []
        lines = text.strip().split('\n')
        for line in lines:
            line = line.strip()
            if not line:
                continue
            if line.startswith('Клиент:'):
                dialog.append({
                    "role": "user",
                    "text": line.replace('Клиент:', '').strip()
                })
            elif line.startswith('Оператор:'):
                dialog.append({
                    "role": "assistant",
                    "text": line.replace('Оператор:', '').strip()
                })
        return dialog
    def generate_dialog(self, scenario: Dict[str, Any]) -> Optional[Dict[str, Any]]:
        """
        Генерирует один диалог по сценарию
        Args:
            scenario: сценарий диалога
        Returns:
            Optional[Dict]: структурированный диалог или None
        """
        print(f"   ? Генерация диалога: {scenario.get('name')}")
        # Формируем промпт
        prompt = self._build_generation_prompt(scenario)
        # Вызываем API
        response = self._call_yandex_gpt(prompt)
        if not response:
            print(f"   □ Не удалось сгенерировать диалог")
            return None
        # Парсим ответ
        dialog = self._parse_dialog(response)
        if not dialog:
```

```
            print(f"   □□  Диалог не распознан, сохраняем сырой текст")
            dialog = [{"role": "raw", "text": response}]
        # Формируем результат
        result = {
            "id": str(uuid.uuid4())[:8],
            "scenario_id": scenario.get("id"),
            "scenario_name": scenario.get("name"),
            "timestamp": datetime.now().isoformat(),
            "dialog": dialog,
            "raw_response": response,
            "metadata": {
                "client_objection": scenario.get("client_objection"),
                "operator_strategy": scenario.get("operator_strategy"),
                "expected_outcome": scenario.get("expected_outcome")
            }
        }
        print(f"   □ Сгенерировано {len(dialog)} реплик")
        return result
    def generate_batch(self, scenario_id: Optional[int] = None, count: int = 3) -> List[Dict[str, Any]]:
        """
        Генерирует пакет диалогов
        Args:
            scenario_id: ID сценария (если None — все сценарии)
            count: количество диалогов на сценарий
        Returns:
            List[Dict]: список сгенерированных диалогов
        """
        # Выбираем сценарии
        if scenario_id is not None:
            scenarios = [s for s in self.scenarios if s.get("id") == scenario_id]
        else:
            scenarios = self.scenarios
        if not scenarios:
            print("□  Нет сценариев для генерации")
            return []
        results = []
        for scenario in scenarios:
            print(f"\n? Сценарий: {scenario.get('name')}")
            print(f"   Описание: {scenario.get('description')}")
            for i in range(count):
                print(f"\n   ? Диалог {i + 1}/{count}")
                dialog = self.generate_dialog(scenario)
                if dialog:
                    results.append(dialog)
                    # Сохраняем в файл
                    self._save_dialog(dialog)
        print(f"\n□ Сгенерировано диалогов: {len(results)}")
        return results
    def _save_dialog(self, dialog: Dict[str, Any]) -> Path:
        """
        Сохраняет диалог в JSON-файл
        Args:
            dialog: структурированный диалог
        Returns:
            Path: путь к сохраненному файлу
        """
        timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
        filename = f"{timestamp}_{dialog['id']}.json"
        file_path = self.output_dir / filename
        with open(file_path, 'w', encoding='utf-8') as f:
            json.dump(dialog, f, ensure_ascii=False, indent=2)
        print(f"   ? Сохранен: {filename}")
```

```
        return file_path
    def get_generated_dialogs(self) -> List[Dict[str, Any]]:
        """Возвращает все сгенерированные диалоги"""
        dialogs = []
        for file_path in self.output_dir.glob("*.json"):
            try:
                with open(file_path, 'r', encoding='utf-8') as f:
                    dialogs.append(json.load(f))
            except Exception as e:
                print(f"□ Ошибка загрузки {file_path.name}: {e}")
        return dialogs
# Тестирование
if __name__ == "__main__":
    print("? Тестирование DialogGenerator")
    print("=" * 60)
    generator = DialogGenerator()
    # Генерируем по 1 диалогу для каждого сценария
    print("\n? Генерация диалогов по всем сценариям...")
    results = generator.generate_batch(count=1)  # по 1 диалогу на сценарий
    if results:
        print(f"\n□ Сгенерировано {len(results)} диалогов")
        print(f"? Сохранены в: {generator.output_dir}")
    else:
        print("□  Ничего не сгенерировано")
```

ПРИЛОЖЕНИЕ Д. Листинг модуля синтеза речи (TTS)

Листинг Д.1. tts_converter.py

python

```
"""
tts_converter.py - Преобразование текстовых диалогов в аудио через Yandex SpeechKit TTS
"""
import sys
import json
import time
from pathlib import Path
from typing import Dict, List, Optional, Any
import requests
from pydub import AudioSegment
import io
# Исправляем пути
PART2_ROOT = Path(__file__).parent.parent.parent
SRC_PATH = PART2_ROOT / "src"
if str(SRC_PATH) not in sys.path:
    sys.path.insert(0, str(SRC_PATH))
# Импорты
try:
    from utils.config import config
    print("□  configимпортирован")
except ImportError as e:
    print(f"□ Ошибка импорта config: {e}")
    print(f"   Путь поиска: {SRC_PATH}")
    sys.exit(1)
```

```
class TTSConverter:
    """
    Преобразователь текстовых диалогов в аудиофайлы с разными голосами
    """
    # Доступные голоса Yandex SpeechKit
    VOICES = {
        "operator": "alena",  # женский голос для оператора
        "client": "filipp"  # мужской голос для клиента
    }
    def __init__(self, input_dir: Optional[Path] = None, output_dir: Optional[Path] = None):
        """
        Инициализация TTS-конвертера
        """
        self.api_key = config.get('yandex_api_key')
        self.folder_id = config.get('yandex_folder_id')
        # URL для синтеза речи
        self.tts_url = "https://tts.api.cloud.yandex.net/speech/v1/tts:synthesize"
        # Папки
        if input_dir is None:
            self.input_dir = PART2_ROOT / "data" / "generated" / "dialogs"
        else:
            self.input_dir = Path(input_dir)
        if output_dir is None:
            self.output_dir = PART2_ROOT / "data" / "generated" / "audio"
        else:
            self.output_dir = Path(output_dir)
        self.output_dir.mkdir(parents=True, exist_ok=True)
        # Пауза между репликами (в миллисекундах)
        self.pause_between_turns = 800
        print(f"□ TTSConverterинициализирован")
        print(f"   ? Входные JSON: {self.input_dir}")
        print(f"   ? Выходные аудио: {self.output_dir}")
    def _synthesize(self, text: str, voice: str) -> Optional[AudioSegment]:
        """
        Синтезирует речь из текста, возвращает AudioSegment
        """
        if not text.strip():
            return None
        headers = {
            "Authorization": f"Api-Key {self.api_key}"
        }
        # Используем формат lpcm (сырой PCM)
        params = {
            "text": text,
            "lang": "ru-RU",
            "voice": voice,
            "format": "lpcm",
            "sampleRateHertz": 16000
        }
        try:
            response = requests.post(
                self.tts_url,
                headers=headers,
                params=params,
                timeout=30
            )
            if response.status_code == 200:
                # Преобразуем сырые PCM данные в AudioSegment
                # 16-bit, 16000 Hz, моно
                audio = AudioSegment.from_file(
                    io.BytesIO(response.content),
                    format="raw",
                    frame_rate=16000,
```

```
                    sample_width=2,  # 16 bit = 2 bytes
                    channels=1
                )
                return audio
            else:
                print(f"   □ Ошибка TTS: {response.status_code} - {response.text[:100]}")
                return None
        except Exception as e:
            print(f"   □  Исключение при синтезе:{e}")
            return None
    def convert_dialog(self, dialog_path: Path) -> Optional[Path]:
        """
        Преобразует один JSON-диалог в аудиофайл
        """
        print(f"\n?□ㅈ ㅋ  Обработка: {dialog_path.name}")
        # Читаем JSON
        try:
            with open(dialog_path, 'r', encoding='utf-8') as f:
                dialog_data = json.load(f)
        except Exception as e:
            print(f"   □ Ошибка чтения JSON: {e}")
            return None
        turns = dialog_data.get("dialog", [])
        if not turns:
            print(f"   □□  Диалог не содержит реплик")
            return None
        print(f"   ? Всего реплик: {len(turns)}")
        combined_audio = AudioSegment.silent(duration=0)
        for i, turn in enumerate(turns):
            role = turn.get("role")
            text = turn.get("text", "")
            if not text:
                continue
            if role == "user":
                voice = self.VOICES["client"]
                speaker = "Клиент"
            elif role == "assistant":
                voice = self.VOICES["operator"]
                speaker = "Оператор"
            else:
                continue
            print(f"   ?  {speaker}: {text[:50]}...")
            audio_segment = self._synthesize(text, voice)
            if audio_segment is None:
                print(f"      □□  Не удалось синтезировать реплику {i + 1}")
                continue
            combined_audio += audio_segment
            if i < len(turns) - 1:
                combined_audio += AudioSegment.silent(duration=self.pause_between_turns)
        if len(combined_audio) == 0:
            print(f"   □ Не удалось создать аудио")
            return None
        output_filename = dialog_path.stem + ".wav"
        output_path = self.output_dir / output_filename
        combined_audio.export(output_path, format="wav")
        print(f"   ? Сохранен: {output_filename} (длительность: {len(combined_audio) / 1000:.1f} сек)")
        return output_path
    def convert_all(self) -> List[Path]:
        """
        Преобразует все JSON-диалоги
        """
        json_files = list(self.input_dir.glob("*.json"))
```

```
        if not json_files:
            print(f"□□  В папке {self.input_dir} нет JSON-файлов")
            return []
        print(f"? Найдено JSON-файлов: {len(json_files)}")
        results = []
        for i, json_file in enumerate(json_files, 1):
            print(f"\n?  {i}/{len(json_files)}")
            audio_path = self.convert_dialog(json_file)
            if audio_path:
                results.append(audio_path)
            time.sleep(0.5)
        print(f"\n□ Преобразовано диалогов: {len(results)}")
        return results
if __name__ == "__main__":
    print("? Тестирование TTSConverter")
    print("=" * 60)
    converter = TTSConverter()
    audio_files = converter.convert_all()
    if audio_files:
        print(f"\n? Создано аудиофайлов:")
        for f in audio_files:
            print(f"   - {f.name}")
    else:
        print("□□  Ничего не создано")
    print("\n" + "=" * 60)
```

ПРИЛОЖЕНИЕ Е. Листинг модуля асинхронной транскрибации

Листинг Е.1. async_transcriber.py

python

```
"""
async_transcriber.py - Асинхронная транскрибация длинных аудиофайлов
Использует асинхронный REST API Yandex SpeechKit + Object Storage
"""
import sys
import os
import time
import json
import uuid
from pathlib import Path
from typing import Optional, Dict, List, Any
from datetime import datetime
import requests
import boto3
from botocore.config import Config
# Исправляем пути
PART2_ROOT = Path(__file__).parent.parent.parent
SRC_PATH = PART2_ROOT / "src"
if str(SRC_PATH) not in sys.path:
    sys.path.insert(0, str(SRC_PATH))
# Импорты
try:
    from utils.config import config
    print("□  configимпортирован")
```

```
except ImportError as e:
    print(f"□ Ошибка импорта config: {e}")
    print(f"   Путь поиска: {SRC_PATH}")
    sys.exit(1)
class AsyncTranscriber:
    """
    Асинхронный транскрибатор для длинных аудиофайлов
    """
    def __init__(self):
        self.api_key = config.get('yandex_api_key')
        self.folder_id = config.get('yandex_folder_id')
        # S3 настройки — читаем напрямую из переменных окружения
        self.access_key = os.getenv('AWS_ACCESS_KEY_ID')
        self.secret_key = os.getenv('AWS_SECRET_ACCESS_KEY')
        self.bucket_name = os.getenv('BUCKET_NAME')
        self.endpoint_url = os.getenv('S3_ENDPOINT_URL', 'https://storage.yandexcloud.net')
        # URL для асинхронного API
        self.async_url = "https://transcribe.api.cloud.yandex.net/speech/stt/v2/longRunningRecognize"
        # Папка для результатов
        self.output_dir = PART2_ROOT / "data" / "transcriptions"
        self.output_dir.mkdir(parents=True, exist_ok=True)
        # Инициализация S3 клиента
        self.s3_client = self._init_s3_client()
        print(f"□ AsyncTranscriberинициализирован")
        print(f"   ? Сохранение: {self.output_dir}")
        print(f"   ? Бакет: {self.bucket_name}")
    def _init_s3_client(self):
        """Инициализирует клиент S3 (Object Storage)"""
        if not all([self.access_key, self.secret_key, self.bucket_name]):
            print("□□ S3 ключи не настроены, загрузка в Object Storage недоступна")
            return None
        try:
            session = boto3.session.Session()
            client = session.client(
                's3',
                endpoint_url=self.endpoint_url,
                aws_access_key_id=self.access_key,
                aws_secret_access_key=self.secret_key,
                config=Config(signature_version='s3v4')
            )
            print("□ S3клиент инициализирован")
            return client
        except Exception as e:
            print(f"□ Ошибка инициализации S3:{e}")
            return None
    def _get_headers(self) -> Dict[str, str]:
        return {
            "Authorization": f"Api-Key {self.api_key}",
            "Content-Type": "application/json"
        }
    def upload_to_object_storage(self, audio_path: Path) -> Optional[str]:
        if not self.s3_client:
            print("   □□ S3 клиент не инициализирован, пропускаем")
            return None
        object_key = f"audio/{datetime.now().strftime('%Y-%m-%d')}/{uuid.uuid4()}{audio_path.suffix}"
        try:
            print(f"   ? Загрузка: {audio_path.name} → {object_key}")
            with open(audio_path, 'rb') as f:
                self.s3_client.put_object(
                    Bucket=self.bucket_name,
                    Key=object_key,
                    Body=f,
                    ContentType='audio/wav'
```

```
                )
            # Формируем HTTPS URL (без завершающего слеша в endpoint_url)
            base_url = self.endpoint_url.rstrip('/')
            url = f"{base_url}/{self.bucket_name}/{object_key}"
            print(f"   URL:{url}")
            return url
        except Exception as e:
            print(f"   Ошибка загрузки: {e}")
            return None
    def start_recognition(self, audio_uri: str) -> Optional[str]:
        """
        Запускает асинхронное распознавание
        """
        payload = {
            "config": {
                "specification": {
                    "languageCode": "ru-RU",
                    "model": "general",
                    "audioEncoding": "LINEAR16_PCM",
                    "sampleRateHertz": 16000,
                    "audioChannelCount": 1
                }
            },
            "audio": {
                "uri": audio_uri
            }
        }
        try:
            response = requests.post(
                self.async_url,
                headers=self._get_headers(),
                json=payload,
                timeout=30
            )
            if response.status_code == 200:
                result = response.json()
                operation_id = result.get("id")
                print(f"   ? Задача запущена: {operation_id}")
                return operation_id
            else:
                print(f"   Ошибка запуска: {response.status_code} {response.text[:200]}")
                return None
        except Exception as e:
            print(f"   Исключение: {e}")
            return None
    def get_recognition_result(self, operation_id: str, max_wait: int = 300) -> Optional[str]:
        """
        Ожидает завершения операции и возвращает текст
        """
        url = f"https://operation.api.cloud.yandex.net/operations/{operation_id}"
        wait_time = 0
        poll_interval = 5
        while wait_time < max_wait:
            try:
                response = requests.get(url, headers=self._get_headers())
                if response.status_code == 200:
                    data = response.json()
                    if data.get("done", False):
                        if "response" in data:
                            # Извлечение текста из ответа
                            chunks = data["response"].get("chunks", [])
                            texts = []
                            for chunk in chunks:
```

```
                                        for alt in chunk.get("alternatives", []):
                                                text = alt.get("text", "")
                                                if text:
                                                        texts.append(text)
                                        return " ".join(texts)
                                elif "error" in data:
                                        print(f"   □ Ошибка операции: {data['error']}")
                                        return None
                                else:
                                        print(f"   □ Ожидание... ({wait_time}/{max_wait} сек)")
                                        time.sleep(poll_interval)
                                        wait_time += poll_interval
                        else:
                                print(f"   □ Ошибка проверки: {response.status_code}")
                                return None
                except Exception as e:
                        print(f"   □ Ошибка: {e}")
                        return None
        print(f"   □ Таймаут {max_wait} сек")
        return None
def transcribe_audio(self, audio_path: Path) -> Optional[Dict[str, Any]]:
        """
        Полный цикл: загрузка в S3, запуск распознавания, получение результата
        """
        print(f"\n?□ㅈ ㅋ  Транскрибация: {audio_path.name}")
        print(f"   Размер: {audio_path.stat().st_size / 1024:.1f} KB")
        # Загружаем в S3
        audio_uri = self.upload_to_object_storage(audio_path)
        if not audio_uri:
                return None
        # Запускаем распознавание
        operation_id = self.start_recognition(audio_uri)
        if not operation_id:
                return None
        # Получаем результат
        text = self.get_recognition_result(operation_id)
        if not text:
                return None
        # Сохраняем результат
        result = {
                "filename": audio_path.name,
                "timestamp": datetime.now().isoformat(),
                "operation_id": operation_id,
                "text": text,
                "text_length": len(text),
                "audio_size_kb": audio_path.stat().st_size / 1024
        }
        output_file = self.output_dir / f"{audio_path.stem}_transcript.json"
        with open(output_file, 'w', encoding='utf-8') as f:
                json.dump(result, f, ensure_ascii=False, indent=2)
        print(f"   ? Сохранен: {output_file.name}")
        print(f"   ? Текст: {text[:100]}...")
        return result
def transcribe_all(self, audio_dir: Optional[Path] = None) -> List[Dict[str, Any]]:
        """
        Транскрибирует все WAV-файлы в указанной папке
        """
        if audio_dir is None:
                audio_dir = PART2_ROOT / "data" / "generated" / "audio"
        audio_files = list(audio_dir.glob("*.wav"))
        if not audio_files:
                print(f"□□  В папке {audio_dir} нет WAV-файлов")
```

```
                return []
            print(f"? Найдено WAV-файлов: {len(audio_files)}")
            results = []
            for i, f in enumerate(audio_files, 1):
                print(f"\n?  {i}/{len(audio_files)}")
                result = self.transcribe_audio(f)
                if result:
                    results.append(result)
                time.sleep(2)  # небольшая пауза между файлами
            print(f"\n□ Транскрибировано: {len(results)}")
            return results
if __name__ == "__main__":
    print("? Тестирование AsyncTranscriber")
    print("=" * 60)
    # Добавим отладочный вывод переменных окружения
    print("Проверка переменных окружения:")
    for var in ['AWS_ACCESS_KEY_ID', 'AWS_SECRET_ACCESS_KEY', 'BUCKET_NAME',
'S3_ENDPOINT_URL']:
        val = os.getenv(var)
        if val:
            print(f"  {var}: {val[:10]}...")
        else:
            print(f"  {var}: не задана")
    transcriber = AsyncTranscriber()
    results = transcriber.transcribe_all()
    if results:
        print("\n? Результаты:")
        for r in results:
            print(f"  - {r['filename']}: {r['text_length']} символов")
    else:
        print("□□ Ничего не транскрибировано")
    print("\n" + "=" * 60)
```

ПРИЛОЖЕНИЕ Ж. Листинг скрипта экспорта в CSV

Листинг Ж.1. export_to_csv.py

python

```
"""
export_to_csv.py - Экспорт диалогов и транскриптов в CSV для анализа в Google Sheets
"""
import sys
import json
import csv
from pathlib import Path
from typing import Dict, List, Any
# Добавляем путь к src для импорта (хотя здесь он не обязателен, оставим для единообразия)
PART2_ROOT = Path(__file__).parent.parent
SRC_PATH = PART2_ROOT / "src"
if str(SRC_PATH) not in sys.path:
    sys.path.insert(0, str(SRC_PATH))
def extract_full_text_from_dialog(dialog_data: Dict[str, Any]) -> str:
    """
    Извлекает полный текст диалога из JSON (склеивает реплики с указанием роли)
    """
    turns = dialog_data.get("dialog", [])
```

```
    if not turns:
        return ""
    lines = []
    for turn in turns:
        role = turn.get("role")
        text = turn.get("text", "").strip()
        if role == "user":
            lines.append(f"Клиент: {text}")
        elif role == "assistant":
            lines.append(f"Оператор: {text}")
        else:
            lines.append(text)  # на случай сырых данных
    return "\n".join(lines)
def main():
    # Папки с данными
    dialogs_dir = PART2_ROOT / "data" / "generated" / "dialogs"
    transcripts_dir = PART2_ROOT / "data" / "transcriptions"
    output_dir = PART2_ROOT / "data" / "results"
    output_dir.mkdir(parents=True, exist_ok=True)
    # Ищем все JSON-диалоги
    dialog_files = sorted(dialogs_dir.glob("*.json"))
    if not dialog_files:
        print("□□  Нет файлов диалогов в", dialogs_dir)
        return
    print(f"? Найдено диалогов: {len(dialog_files)}")
    # Собираем результаты
    rows = []
    for dialog_path in dialog_files:
        # Читаем диалог
        with open(dialog_path, 'r', encoding='utf-8') as f:
            dialog = json.load(f)
        dialog_id = dialog_path.stem
        scenario_name = dialog.get("scenario_name", "неизвестно")
        original_text = extract_full_text_from_dialog(dialog)
        # Ищем соответствующий транскрипт
        transcript_path = transcripts_dir / f"{dialog_id}_transcript.json"
        transcribed_text = ""
        if transcript_path.exists():
            with open(transcript_path, 'r', encoding='utf-8') as f:
                transcript = json.load(f)
                transcribed_text = transcript.get("text", "")
        else:
            print(f"□□  Транскрипт не найден для {dialog_id}")
        rows.append({
            "dialog_id": dialog_id,
            "scenario_name": scenario_name,
            "original_text": original_text,
            "transcribed_text": transcribed_text
        })
    # Сохраняем CSV
    output_csv = output_dir / "dialog_comparison.csv"
    with open(output_csv, 'w', encoding='utf-8-sig', newline='') as f:
        fieldnames = ["dialog_id", "scenario_name", "original_text", "transcribed_text"]
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)
    print(f"\n□ CSV сохранён: {output_csv}")
    print(f"   Всего записей: {len(rows)}")
    print("\n? Статистика по длине текстов:")
    for row in rows:
        orig_len = len(row["original_text"])
        trans_len = len(row["transcribed_text"])
        print(f"   {row['dialog_id']}: исходный {orig_len} симв., транскрипт {trans_len} симв.")
```

```
if __name__ == "__main__":
    main()
```

ПРИЛОЖЕНИЕ 3. Примеры сгенерированных диалогов и скриншоты

Пример диалога (сценарий «Возражение: Дорого»)

text

Клиент: Здравствуйте! Я посмотрел у вас на сайте ограждения — очень красиво, но цены кажутся мне завышенными. У конкурентов дешевле.
Оператор: Здравствуйте! Спасибо за ваш интерес к продукции CULT CONSTRUCTIONS. Мы понимаем, что цена — важный фактор, и хотим подчеркнуть, что наши ограждения отличаются премиальным качеством и долговечностью. Срок службы — более 100 лет, и они не требуют регулярного обслуживания.
Клиент: Да, но всё равно это слишком дорого для меня. Я не готов столько платить.
Оператор: Мы предлагаем не просто ограждение, а долгосрочное решение. К тому же у нас есть варианты рассрочки. Можем отправить специалиста для бесплатного замера и консультации.
Клиент: Ладно, замер — это интересно. Как это организовать?
Оператор: Нам понадобится ваш контактный телефон и удобное время. Мы согласуем визит.
Клиент: Хорошо, мой телефон [номер телефона]. Давайте назначим замер на следующую неделю.
Оператор: Отлично! Мы свяжемся с вами. Спасибо за обращение!

Примечание: полные тексты всех модулей приведены в электронном виде в папке part2/src проекта. Скриншоты веб-чата и таблиц Google Sheets доступны в отдельном файле.

ЗАКЛЮЧЕНИЕ ПО ДИПЛОМНОМУ ПРОЕКТУ

В рамках дипломной работы был разработан и реализован комплексный прототип системы для автоматического анализа телефонных переговоров и генерации синтетических диалогов с использованием российских облачных технологий. Проект состоял из трёх логически связанных частей.

Первая часть была посвящена созданию пайплайна транскрибации записей колл-центра. Экспериментально подтверждено, что низкое качество телефонных записей (частота 8000 Гц, громкость ниже -35 dBFS) делает прямое распознавание через Yandex SpeechKit невозможным. Разработанный модуль предобработки (AudioProcessor) позволяет нормализовать громкость, применить частотные фильтры и привести аудио к стандарту 16000 Гц, что повышает успешность распознавания с 0 % до приемлемого уровня (средняя длина распознанного текста >150 символов). Выявлено ключевое ограничение синхронного API – 30 секунд, что обусловило необходимость перехода на асинхронный режим во второй части.

Вторая часть включала создание ИИ-ассистента для сайта компании CULT CONSTRUCTIONS, а также полного цикла генерации синтетических диалогов. Реализованы:

- веб-чат на FastAPI с логикой диалога, сбором данных (телефон, длина участка) и сохранением истории;
- модуль генерации диалогов на основе Yandex GPT (7 сценариев, охватывающих типовые возражения);
- синтез речи (TTS) с разными голосами для оператора и клиента;
- асинхронная транскрибация длинных аудиофайлов (с загрузкой в Object Storage и использованием асинхронного REST API SpeechKit);
- экспорт данных в Google Sheets и анализ с помощью формул (выявление возражений, оценка успешности, точность распознавания).

Третья часть посвящена перспективам развития. Разработана архитектура интеграции с VK-ботом для уведомлений о лидах, подготовлен теоретический план подключения Telegram (с учётом замедления работы мессенджера в РФ). Также предложены направления масштабирования: применение асинхронной транскрибации к реальным записям, подключение к CRM, построение дашбордов.

Основные результаты:

- Доказана принципиальная возможность построения полного цикла обработки речевых данных на базе российского стека Yandex Cloud, что обеспечивает соблюдение требований ФЗ-152.
- Создан рабочий прототип ИИ-ассистента, способного консультировать клиентов и генерировать эталонные диалоги.
- Получены метрики качества: средняя точность распознавания синтезированной речи составила 92 %, успешность диалогов (наличие телефона/заявки) – 86 %.
- Подтверждена возможность автоматического выявления возражений и оценки успешности с помощью формул Google Sheets, что может служить основой для data-driven улучшения скриптов продаж.

Разработанные модули могут быть адаптированы для реального использования в колл-центрах, а предложенная архитектура легко масштабируется и допускает замену компонентов (например, переход на VK Cloud SpeechKit).